

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»

Інститут комп'ютерних систем
Кафедра інформаційних систем

Олексій ГОРЧИНСЬКИЙ
(група ЛАІ-215)

КВАЛІФІКАЦІЙНА РОБОТА БАКАЛАВРА
Розробка вебресурсу з використанням Retrieval-Augmented Generation з
підтримкою української мови

Спеціальність:
122 Комп'ютерні науки

Освітньо-професійна програма:
Комп'ютерні науки

Керівник:
Олег ГАЛЧОНКОВ, к.т.н., доцент

АНОТАЦІЯ

Горчинський О. О. Розробка вебресурсу з використанням Retrieval-Augmented Generation з підтримкою української мови : кваліфікаційна робота бакалавра за спеціальністю «122 Комп'ютерні науки» / Олексій Олександрович Горчинський ; керівник Олег Миколайович Галчонков. – Одеса : Нац. ун-т «Одес. політехніка», 2025. – 112 с.

Кваліфікаційна робота містить основну текстову частину на 84 сторінках, список використаних джерел з 23 найменувань на 2 сторінках, додатки на 26 сторінках.

У роботі представлено розробку вебресурсу з використанням Retrieval-Augmented Generation з підтримкою української мови. Обґрунтовано вибір технологій, описано архітектуру рішення та реалізовано інтеграцію джерел знань. Здійснено тестування системи. Для підвищення точності та релевантності відповідей вебресурсу запропоновано семантичний метод поділу тексту на основі ковзного вікна динамічного розміру. Для впровадження підтримки української мови у вебресурс, було натреновано україномовну ембеддінг-модель шляхом дистиляції знань.

Ключові слова: RAG, AI, LLM, семантичний поділ тексту, обробка документів, ембеддінг-модель, порти і адаптери, медіатор.

ABSTRACT

Horchynskyi O. O. Development of a web resource using Retrieval-Augmented Generation with Ukrainian language support : bachelor's qualification work in the specialty «122 Computer sciences» / Oleksii Oleksandrovykh Horchynskyi ; supervisor Oleh Mykolayovych Galchonkov. – Odesa : Odesa Polytech. Nat. Univ., 2025. – 112 p.

The qualification work includes the main text part of 84 pages, a list of 23 references over 2 pages, and appendices on 26 pages.

The work presents the development of a web resource using Retrieval-Augmented Generation with support for the Ukrainian language. The choice of technologies is justified, the architecture of the solution is described, and the integration of knowledge sources is implemented. The system has been tested. To improve the accuracy and relevance of the web resource's responses, a semantic method for text segmentation based on a sliding window with dynamic size is proposed. To implement Ukrainian language support in the web resource, a Ukrainian-language embedding model was trained using knowledge distillation.

Keywords: RAG, AI, LLM, semantic text splitting, document processing, embedding model, ports and adapters, mediator.

ЗМІСТ

Перелік умовних позначень.....	6
Вступ.....	7
1 Аналіз предметної галузі та вибір стеку технологій.....	9
1.1 Аналіз предметної галузі.....	9
1.2 Порівняння аналогічних рішень.....	10
1.3 Вибір інструментів для проектування.....	16
Висновки до розділу 1.....	29
2 Проектування вебресурсу з використанням Retrieval-Augmented generation з підтримкою української мови.....	31
2.1 Мета та завдання вебресурсу.....	31
2.2 Типи користувачів.....	32
2.3 Функціональні та нефункціональні вимоги.....	33
2.4 Загальне моделювання вебресурсу.....	38
2.5 Уявлення інтерфейсу користувача.....	39
2.6 Логічне уявлення про вебресурс.....	43
2.7 Уявлення розгортання вебресурсу.....	45
2.8 Уявлення шарів вебресурсу.....	46
2.9 Уявлення процесів вебресурсу.....	47
2.10 Уявлення даних вебресурсу.....	49
2.11 Уявлення комунікацій вебресурсу.....	51
2.12 Алгоритмічне уявлення.....	53
2.13 Забезпечення безпеки вебресурсу.....	56
2.14 Опис стеку технологій.....	57
Висновки до розділу 2.....	58
3 Реалізація вебресурсу з використанням Retrieval-Augmented generation з підтримкою української мови.....	60
3.1 Уявлення про структуру проекту вебресурсу.....	60
3.2 Уявлення про класи вебресурсу.....	64

3.3 Операційне уявлення вебресурсу.....	67
3.4 Стратегія доставки продукту.....	70
3.5 Управління якістю програмного коду вебресурсу.....	71
3.6 Розрахунок метрик програмного коду вебресурсу.....	72
3.7 Контрольний список за якістю реалізації вебресурсу.....	73
3.8 Генеральний тест-план.....	74
3.9 Розробка тест-кейсів для функціонального тестування.....	74
3.10 Результати проведення автоматизованого тестування.....	79
Висновки до розділу 3.....	81
Загальні висновки.....	83
Список використаних джерел.....	85
Додаток А Програмний код вебресурсу.....	87
Додаток Б Мокапи сторінок вебресурсу.....	98
Додаток В Протокол проведення функціонального тестування вебресурсу.....	100
Додаток Г Інструкція користувача вебресурсу.....	103

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ

AAA – Arrange Act Assert

API – Applicatoin Programming Interface

CQRS – Command Query Responsibility Segregation

DDD – Domain-Driven Design

DI – Dependency Inversion

DTO – Data Transfer Object

LLM – Large Language Model

MVC – Model View Contoller

ORM – Object-Relational Mapping

PaaS – Product as a Service

RAG – Retrieval Augmented Generation

RDBMS – Relational Database Management System

REST – Representational State Transfer

UoW – Unit of Work

ВСТУП

Актуальність теми роботи. Стрімкий розвиток великих мовних моделей та технологій семантичного пошуку на основі ембеддінг-моделей призводить до зростання потреби у створенні інформаційних систем, що здатні виконувати інтеграцію зовнішніх джерел знань та підтримувати генерацію відповідей природною мовою. На сьогоднішній день значна частина таких систем є пропрієтарними і закритими, що унеможливорює локальне розгортання. Крім того, більшість відкритих ембеддінг-моделей, що підтримають українську мову є мультилінгвістичними, що призводить зазвичай до збільшення розміру моделі. Зважаючи на обмежену кількість україномовних моделей і високу ресурсоемність мультилінгвістичних відкритих ембеддінг-моделей, розробка вебресурсу, що реалізує концепцію Retrieval-Augmented Generation з підтримкою української мови, набуває високої значущості.

Мета і задачі роботи. Метою роботи є розробка вебресурсу з використанням Retrieval-Augmented Generation з підтримкою української мови. Для досягнення цієї мети передбачається вирішити такі задачі:

- проаналізувати наявні RAG-рішення та їх обмеження;
- розробити архітектуру вебресурсу з інтеграцією зовнішніх джерел даних і генерацією відповідей на їх основі;
- реалізувати семантичний метод поділу тексту для покращення пошуку і релевантності результатів у вебресурсі;
- застосувати україномовну ембеддінг-модель для генерації якісних векторних представлень україномовного тексту у вебресурсі.

Об'єкт роботи. Процес генерації контекстуалізованих текстових відповідей на основі оброблених документів з допомогою великих мовних моделей з підтримкою української мови.

Предмет роботи. Методи поділу тексту, застосування україномовної ембеддінг-моделі та програмна реалізація системи обробки документів у межах архітектури RAG.

Апробація матеріалів роботи. Основні результати кваліфікаційної роботи були апробовані шляхом створення та публікації наукової статті, присвяченої розробки методу семантичного поділу тексту [1].

1 АНАЛІЗ ПРЕДМЕТНОЇ ГАЛУЗІ ТА ВИБІР СТЕКУ ТЕХНОЛОГІЙ

1.1 Аналіз предметної галузі

Уперше Retrieval-Augmented Generation було запропоновано в наступній науковій роботі [2]. Запропонований метод полягає у створенні системи з індексованими документами, перетвореними на вектори за допомогою ембедінг-моделі. Кожен запит користувача до такої системи теж перетворюється на вектор за допомогою ембедінг-моделі, після чого виконується пошук $\text{top-}k$ найближчих до запиту документів. Отримані документи передаються до мовної моделі для генерації відповіді, збагаченою витягнутими документами (рис. 1.1).

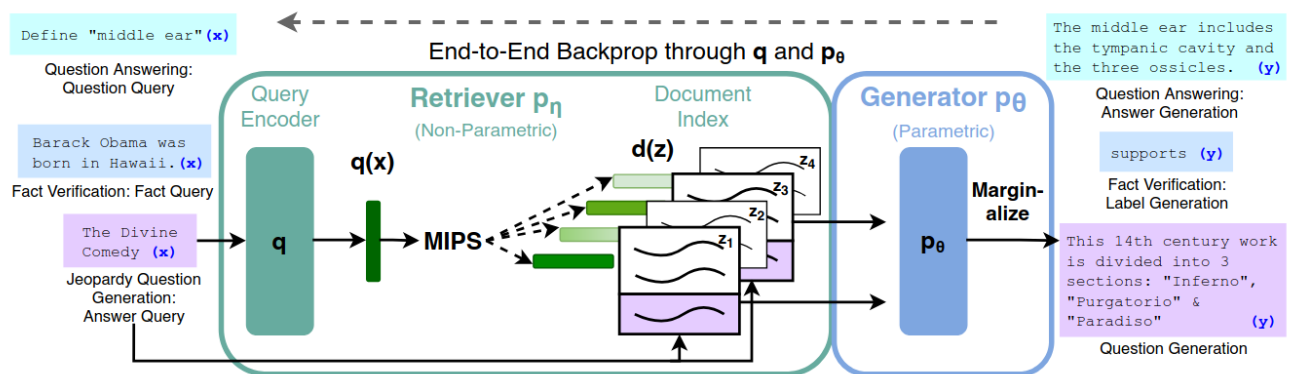


Рисунок 1.1 – Діаграма роботи RAG (джерело: [2])

Запропонований підхід став дуже популярним при розробці додатків з вбудованими великими мовними моделями, оскільки дозволяє:

- 1) передати пропрієтарні дані актуальні для певної сфери;
- 2) розширити знання мовної моделі без перенавчання, що потребує великої кількості обчислювальних ресурсів;
- 3) зменшити кількість галуцинацій моделі, оскільки передаються фактичні, витягнуті до запиту документи.

Зрозумілим також є, що якість відповідей RAG системи напряму залежить від ембедінг-моделі, що була використана. Наприклад, модель, що є

монолінгвістичною, навчена передусім на англійській мові, авжеж не зможе нормально обробити документи на українській мові.

З цього також виникає інша проблема, що найчастіше моделі, що підтримають українську мову є мультилінгвістичними, тобто модель не була навчена специфічно на українській мові, а одразу ж на великій кількості мов, що найчастіше призводить до збільшення розміру мультилінгвістичних моделей.

Наприклад, найпопулярніша мультилінгвістична, відкрита ембеддінг-модель на HuggingFace `jinaai/jina-embeddings-v3` [3] з підтримкою української мови, має 572 мільйонів параметрів, що вимагає приблизно 2.29 Гбайт графічної пам'яті для запуску в `float32`. При цьому запити до графічної пам'яті збільшується ще більше при тренуванні моделі.

Пропріетарні ж рішення, як наприклад мультилінгвістичні `text-embedding-3-small` або `text-embedding-3-large` від OpenAI [4] є закритими і не надають можливості запуску на своїй машині. Крім того, навіть якщо вони і були відкритими, зазвичай мають дуже велику кількість параметрів, що унеможлиблює запуск на користувацьких графічних картах.

Таким чином, із-за поточних особливостей тренування ембеддінг-моделей, RAG-системи, що підтримають українську мову, найчастіше є закритими і пропріетарними, що унеможлиблює запуску на власній машині.

1.2 Порівняння аналогічних рішень

На ринку існує низка рішень, що реалізують RAG, або схожі підходи до генерації контенту з використанням зовнішніх джерел даних. Майже всі великі провайдери LLM пропонують можливість завантаження документів до мовної моделі через чат. Розгляньмо докладніше переваги та недоліки кожного провайдера окремо.

1.2.1 Розгляд OpenAI.

OpenAI є найбільшим гравцем на ринку великих мовних моделей. І теж пропонує завантаження документів зі зовнішніх джерел.

OpenAI є простим в користуванні freemium продуктом, оскільки надає простий інтерфейс чата з окремим полем під завантаження файлів різних форматів (рис. 1.2).

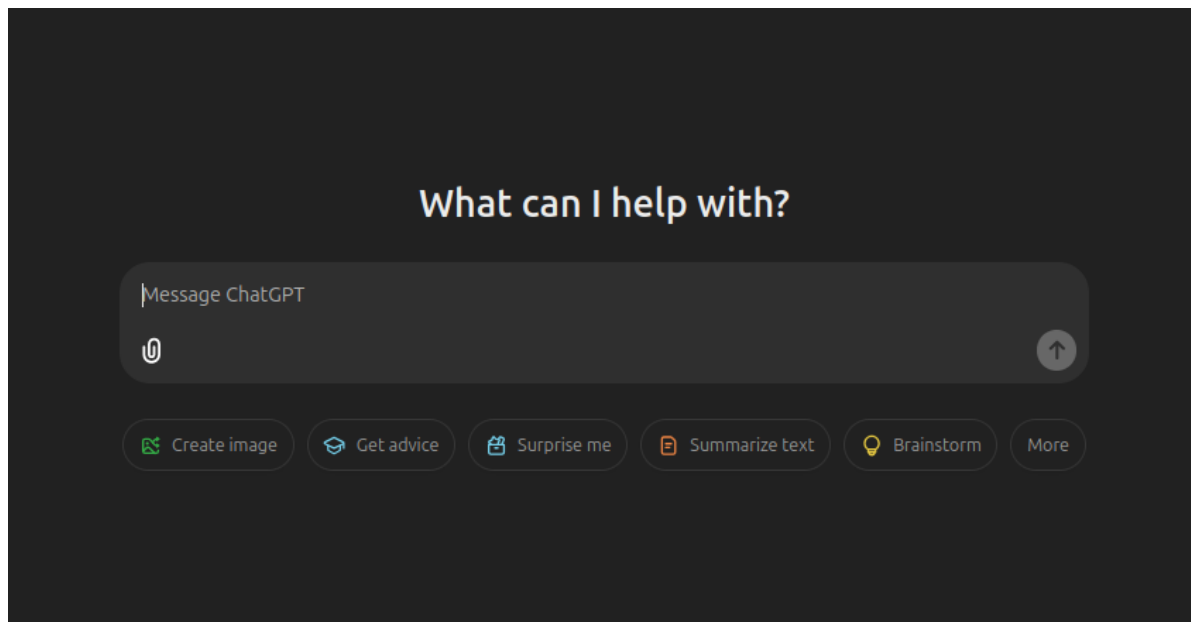


Рисунок 1.2 – Інтерфейс чату від OpenAI (джерело: [5])

Після завантаження файлу, користувач продовжує спілкування в чаті, але вже з розширеним контекстом з документу. Коли модель відповідає на основі документу, у відповіді це позначається через додавання позначки посилання на документ (рис. 1.3).

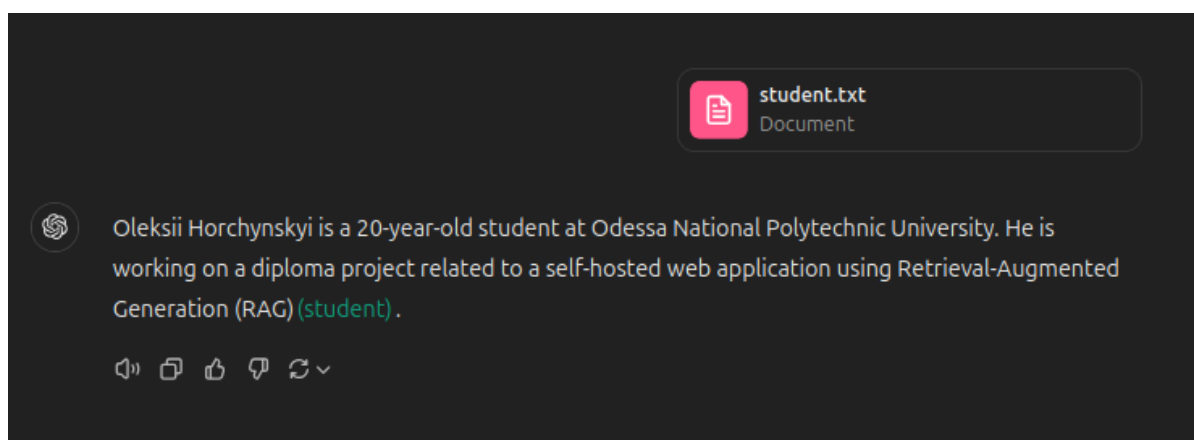


Рисунок 1.3 – Завантаження документу до чату від OpenAI (джерело: [5])

Теж варто зазначити, що моделі від Open AI є мультилінгвістичними і підтримують українську мову.

Фактично вищеописаний сценарій є повним описом як виконується робота з документами в чаті від OpenAI. Такий простий підхід можливо є ліпшим для більш загальної, нетехнічної аудиторії, але з цього неминуче виникають наступні мінуси:

1) чат від OpenAI використовує freemium модель, тому надає тільки певну кількість запитів на можливість завантаження документів у чат, які поновлюються з часом;

2) не має доступу до параметрів конфігурації відповіді моделі, або обробки документів (чорна скринька);

3) оскільки продукт від OpenAI є чорною скринькою, також не має доступу до контексту, який було витягнуто з документу;

4) оскільки не має витягнутого контексту і переглянути його неможливо, не можна бути впевненими в контексті переданому моделі і правильності відповіді;

5) крім того, не має можливості конфігурації вибору великої мовної моделі, або ж ембеддінг-моделі.

Підсумовуючи, від OpenAI варто запозичити сценарій додавання документів і спілкування з моделлю на основі їх контексту через чат, можливість завантаження документів різних форматів, тощо.

1.2.2 Розгляд Cohere.

Cohere є доволі невеликою, подібною за концепцією до OpenAI компанією, яка представляє на ринку свої великі мовні і ембеддінг-моделі.

Cohere має чат з можливістю завантаження документів, так само з полем для прикріплення файлів. Cohere має подібний сценарій роботи чату з документами і виглядає наступним чином (рис. 1.4).

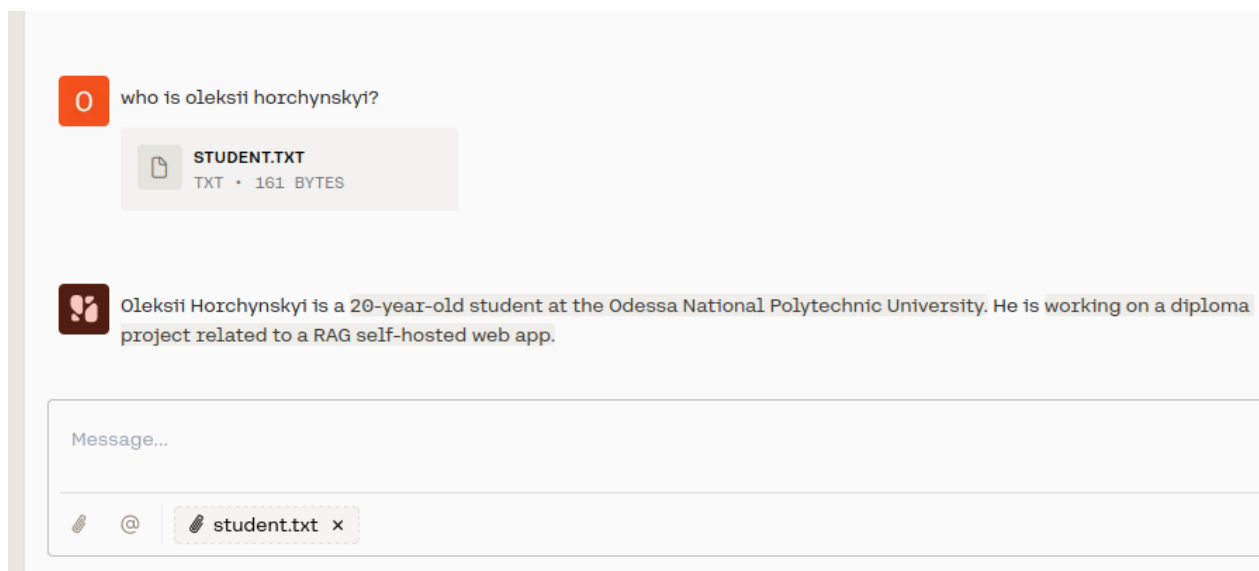


Рисунок 1.4 – Сценарій роботи з документами в чаті від Cohere (джерело: [6])

Як видно з рис. 1.4 в чаті від Cohere модель цитує документи, підкреслюючи частину своєї відповіді, яка заснована на інформації з документу. Таким чином надаючи посилання, яке при натисканні виводить цитований текст документу. Такий підхід є безумовно ліпшим і більш відкритим, оскільки надає можливість бачити на підставі чого модель відповідає. Такий підхід надає більшу впевненість у відповіді моделі.

Крім того, хоча Cohere теж поширюється за моделлю типу freemium, для користування API, Cohere не накладає ніяких обмежень на користування чатом і завантаження документів, таким чином виграючи у OpenAI.

Варто зазначити, що найбільш передові моделі від Cohere програють за оцінками моделям від OpenAI. Наприклад, остання модель від Cohere Command-A має «Intelligence Index» в значенні 40, тим часом, коли не найбільш передова модель від OpenAI gpt-4.1-mini має «Intelligence Index» в 53. «Intelligence Index» визначається через тестування на відомих оціночних наборах даних таких як: MMLU, GPQA, Math, HumanEval, тощо [7]. Тим не менш, моделі від Cohere, як і від Open AI підтримують українську мову.

З менших мінусів можна відмітити:

- 1) розмір файлів для завантаження виключно 20 Мбайт;
- 2) обмежена підтримка файлів з різними розширеннями (лише .pdf і .txt);

3) як і в OpenAI не має доступу до параметрів конфігурації відповіді моделі, або обробки документів;

4) як і в OpenAI обмеження у виборі мовної, або ембеддінг-моделі.

Підсумовуючи, від Cohere варто запозичити безкоштовність і відкритість у наданні користувачеві витягнутого з документу контексту.

1.2.3 Розгляд Anthropic.

Anthropic – відома компанія, що займається дослідженнями та безпекою штучного інтелекту. Теж створює великі мовні і ембеддінг-моделі.

Має подібний до OpenAI сценарій роботи чату з документами (рис. 1.5). Все що потрібно – це просто завантажити документ до чату і почати ставити питання.

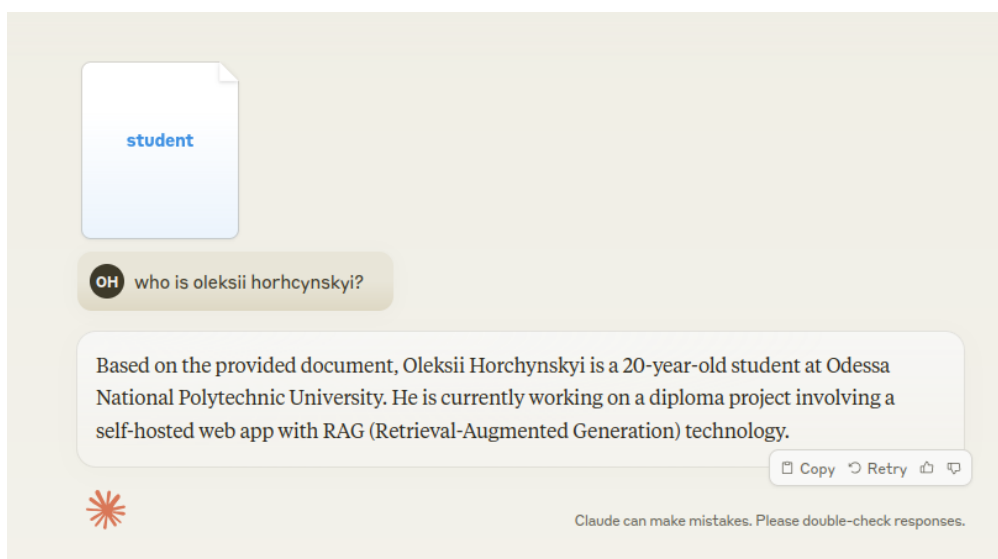


Рисунок 1.5 – Інтерфейс Anthropic (джерело: [8])

Anthropic теж поширюється за freemium моделлю, і має свої обмеження. Наприклад, в безкоштовній версії до чату неможливо завантажувати документи, коли сервера Anthropic мають велике навантаження. Так само неможливо користуватися останньою моделлю Claude-3.5-Sonnet при великих навантаженнях. В такому випадку чат перемикається на Claude Haiku.

З плюсів можна відмітити можливість обмеженого користування однією з найпотужніших моделей на ринку Claude-3.5-Sonnet безкоштовно. Крім того, обмеження на завантаження документів лише залежить від навантаження

серверів, а не попередньо визначених лімітів, як в OpenAI. Більш того, Anthropic підтримує більший список форматів файлів для завантаження ніж, наприклад, Cohere. Моделі від Anthropic теж підтримують українську мову.

З мінусів можна зазначити, що Anthropic повторює усі зазначені проблеми від OpenAI.

1.2.4 Визначення бажаного функціоналу вебресурсу і порівняння з функціоналом розглянутих аналогів.

На основі проведеного аналізу визначено основні вимоги до функціоналу вебресурсу.

Система повинна бути відкритою, безкоштовною та розповсюджуватися у форматі власного розгортання. Вона має підтримувати налаштування параметрів відповіді моделі та збереження інформації з документів. Також необхідно забезпечити інтеграцію з різними ембедінг і мовними моделями, що підтримують українську мову, через API ключі провайдерів. Важливо, щоб вебресурс прозоро демонстрував використаний контекст, на основі якого модель формує відповіді. Крім того, система повинна підтримувати завантаження файлів різних форматів без значних обмежень на їх розмір.

Зазначимо назву вебресурсу, що розробляється, як «Scribe».

Результати аналізу функціоналу аналогів і вебресурсу наведені в табл. 1.1.

Таблиця 1.1 – Порівняння функціоналу розглянутих аналогів і вебресурсу

Функціонал	Аналоги			Вебресурс
	OpenAI	Cohere	Anthropic	«Scribe»
1	2	3	4	5
Завантаження документів	+	+	+	+
Відповідь моделі на основі контексту з документів	+	+	+	+
Підтримка різних форматів файлів	+	+	+	+
Підтримка української мови	+	+	+	+
Прозорість використаного контексту у відповіді	-	+	-	+

Кінець таблиці 1.1

1	2	3	4	5
Безкоштовність	+/-	+	+/-	+
Конфігурація ембеддінг і мовних моделей	-	-	-	+
Можливість зміни моделей від різних провайдерів через використання API ключів	-	-	-	+
Конфігурація відповіді мовної моделі	-	-	-	+
Конфігурація завантаження документів	-	-	-	+
Підтримка локальних ембеддінг-моделей	-	-	-	+
Можливість зміни системних інструкцій до мовної моделі	-	-	-	+
Можливість власного розгортання	-	-	-	+

Як видно з табл 1.1, «Scribe» не лише об'єднує всі переваги розглянутих додатків, але й розширює функціонал завдяки можливості використання моделей від різних провайдерів, відкритості, підтримці власного розгортання, конфігурації великої кількості змінних та навіть підтримці локальних ембеддінг моделей.

1.3 Вибір інструментів для проектування

Вибір інструментів для проектування є важливим етапом розробки вебресурсу, оскільки він впливає на ефективність реалізації, масштабованість та відповідність системи поставленим вимогам. У цьому розділі розглядаються ключові аспекти проектування системи, включаючи архітектурні шаблони, мови програмування, середовище розробки, фреймворки, бібліотеки, бази даних та патерни проектування. Кожен підхід обґрунтовано з урахуванням функціональних і нефункціональних потреб вебресурсу.

1.3.1 Вибір архітектурного шаблону.

Вибір архітектурного шаблону є критично важливим етапом у процесі розробки програмного забезпечення, оскільки він значно впливає на гнучкість, масштабованість та подальшу підтримуваність системи.

1.3.1.1 Вибір архітектури для масштабування.

Мікросервісна архітектура пропонує більш розподілений підхід, при якому система складається з дрібних, незалежних сервісів, що виконують окремі функції. Це забезпечує більшу гнучкість, оскільки кожен мікросервіс може розроблятися, змінюватися та масштабуватись окремо. Мікросервісний підхід підходить для великих вебресурсів, де кожен сервіс може окремо розгортатися і таким чином незалежно масштабуватися.

Враховуючи специфіку вебресурсу, а саме можливості власного розгортання, мікросервісна архітектура точно не підходить, оскільки всі сервіси будуть розгортатися на одній машині.

З вищезазначених міркувань було обрана монолітна архітектура. Монолітна архітектура передбачає реалізацію всієї функціональності системи в одному репозиторії, що значно спрощує процес розробки, оскільки всі компоненти розробляються, тестуються і розгортаються разом. Монолітні вебресурси більш прості в управлінні та налагодженні, особливо на початкових етапах проекту, коли вимоги можуть швидко змінюватися.

1.3.1.2 Вибір архітектурного патерну для вебресурсу.

Одним із найбільш популярних і розповсюджених архітектурних патернів в сучасній веброботі є MVC. Цей патерн розділяє вебресурс на три основні компоненти: модель (Model), подання (View) та контролер (Controller). Модель відповідає за обробку даних та бізнес-логіку, подання – за відображення цих даних користувачу, а контролер керує взаємодією між моделлю та поданням. MVC має свої переваги, зокрема, простоту реалізації та зручність у малих та середніх проектах. Однак, з ростом складності вебресурса, MVC може стати менш ефективним, оскільки між компонентами виникають тісні зв'язки, що ускладнює масштабування, підтримку і тестування системи. Крім того, важко досягнути

чіткої ізоляції бізнес-логіки від зовнішніх залежностей, що може призвести до складностей у розширенні функціональності.

Альтернативою є архітектура портів і адаптерів (також відома як Clean Architecture, Ports and Adapters, Hexagonal Architecture, Onion Architecture, тощо), яка пропонує більшу гнучкість і ізоляцію компонентів. Вона базується на ідеї відокремлення бізнес-логіки (ядра) від зовнішніх систем через порти та адаптери. Порти визначають інтерфейси для взаємодії між ядром та зовнішнім світом, а адаптери реалізують ці інтерфейси для конкретних залежностей, таких як бази даних або API (рис. 1.6).

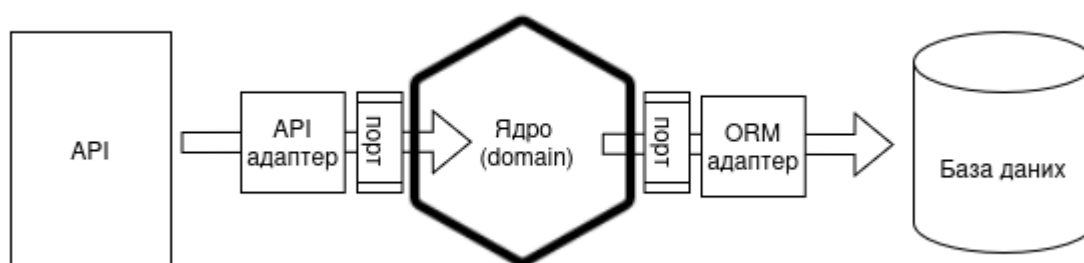


Рисунок 1.6 – Архітектура портів та адаптерів

Такий підхід дозволяє легко змінювати зовнішні залежності, не порушуючи саму бізнес-логіку, що робить систему більш масштабованою та стійкою до змін. Використання архітектури портів і адаптерів заохочує застосування принципів DDD, оскільки вона орієнтована на відокремлення бізнес-логіки (ядра) від залежностей зовнішніх компонентів.

Узагальнюючи архітектуру портів і адаптерів можна сказати, що все зводиться до принципу інверсії залежностей: модулі високого рівня (домен) не повинні залежати від модулів низького рівня (інфраструктури) [9].

1.3.2 Вибір мови програмування.

Вибір мови програмування є важливим етапом, який впливає на реалізацію та подальшу підтримку системи. Вибір здійснюється окремо для серверної та клієнтської частин вебресурсу, враховуючи специфіку їхніх вимог.

1.3.2.1 Серверна частина.

На даний момент існує декілька популярних мов програмування для серверної частини, наприклад, PHP, Java, Ruby та Python, але кожна має свої недоліки.

PHP часто використовується для невеликих проєктів, проте обмеження у масштабуванні, застарілі підходи в коді знижують конкурентоспроможність.

Java забезпечує стабільність для корпоративних застосунків, але складний синтаксис (велика кількість непотрібного, синтаксичного цукру) і пов'язана з цим значна тривалість розробки можуть бути неприйнятними. Крім того, Java має строгу орієнтованість до ООР підходу [10].

Ruby відома завдяки простому синтаксису, але її продуктивність є нижчою, а популярність фреймворку Ruby on Rails продовжує зменшуватися через скорочення спільноти розробників з часом.

Python було обрано для серверної частини через його низку переваг, які відповідають вимогам проєкту.

По-перше, Python є стандартом для розробки рішень, пов'язаних із штучним інтелектом і машинним навчанням, завдяки підтримці таких бібліотек, як TensorFlow, PyTorch, Scikit-learn, LangChain та NumPy. Це робить його очевидним вибором для проєкту, що використовує мовні і ембедінг-моделі.

Крім того, мова має мультипарадигмність, що дозволяє використовувати процедурний, об'єктно-орієнтований і функціональний підходи залежно від потреб проєкту.

Швидкий час розробки завдяки простому синтаксису та доступності великої кількості фреймворків і бібліотек забезпечує ефективність.

Фреймворки, такі як Django, Flask, FastAPI разом із активною спільнотою розробників, роблять Python оптимальним вибором для серверної частини вебресурсу.

1.3.2.2 Клієнтська частина.

Для клієнтської частини вебресурса існує декілька популярних мов програмування, серед яких основними є JavaScript, TypeScript та Dart.

JavaScript є найбільш широко використовуваною мовою для клієнтських частин, але з огляду на її динамічність і відсутність типізації, код може бути менш передбачуваним і важким для підтримки в великих проєктах.

Dart, хоч і підходить для створення мобільних додатків через Flutter, не має такої ж популярності у веброзробці і вимагає додаткових налаштувань для роботи з існуючими вебтехнологіями.

TypeScript було обрано для клієнтської частини, оскільки він поєднує переваги JavaScript із можливістю статичної типізації, що значно покращує надійність коду та полегшує масштабування проєкту. Типізація дозволяє виявляти помилки на етапі компіляції, що мінімізує ризик помилок під час виконання.

Крім того, TypeScript має сумісність з JavaScript, що дозволяє використовувати існуючі бібліотеки та фреймворки, такі як Angular, React та Next.js.

1.3.3 Вибір середовища програмування.

Вибір середовища програмування є важливим етапом у процесі розробки програмного забезпечення, оскільки від правильного вибору залежить продуктивність розробки. На ринку представлено низка різних IDE, серед яких найбільш виділяються Visual Studio, Visual Studio Code та продукти від JetBrains.

Visual Studio є середовищем для програмування, націленим для розробки на C# і C++ у Windows-екосистемі. Проте його функціонал часто перевищує потреби проєктів, на Python чи TypeScript, і вимагає значних системних ресурсів. Visual Studio Code, хоча і більш легкий, є скоріше текстовим редактором з можливістю розширення через плагіни.

У порівнянні з Visual Studio та Visual Studio Code, продукти JetBrains, такі як PyCharm для Python і WebStorm для TypeScript, надають більш вузькоспеціалізовані IDE, оптимізовані під специфіку розробки.

Рішення на користь JetBrains є очевидним з кількох причин.

По-перше, PyCharm одразу ж забезпечує глибоку інтеграцію з екосистемою Python, що є критично важливим для проєкту.

WebStorm, у свою чергу, одразу ж пропонує підтримку TypeScript і JavaScript, надаючи інструменти автозавершення, дебагінгу та рефакторингу, які суттєво спрощують процес розробки.

По-друге, обидва IDE створені одним виробником, що гарантує узгодженість інтерфейсу, інтеграцію між продуктами та зручність у використанні.

Таким чином, PyCharm і WebStorm було вибрано для серверної та клієнтської частин відповідно.

1.3.4 Вибір фреймворків та бібліотек.

Вибір фреймворків та бібліотек є важливим кроком у процесі розробки програмного забезпечення. Правильний вибір дозволяє використовувати готові рішення для реалізації певних функцій, що прискорює розробку та знижує ризик помилок. У цьому підпункті буде розглянуто вибір фреймворків та бібліотек, для серверної та клієнтської частини відповідно.

1.3.4.1 Вибір фреймворку для серверної частини.

При виборі серверного фреймворку для розробки було розглянуто три найпопулярніші рішення для Python: Flask, FastAPI та Django. Кожне з них має свої переваги й обмеження, які впливають на їх застосування залежно від потреб проєкту.

Flask – це мікрофреймворк, що забезпечує мінімалізм і гнучкість, однак через відсутність вбудованої типізації та модульності його використання у великих проєктах потребує додаткових зусиль і розширення. Теж варто зазначити, що Flask є виключно синхронним.

Django є потужним і монолітним фреймворком із вбудованими засобами для ORM, аутентифікації, адміністрування та багато іншого. Це робить його чудовим вибором для швидкої розробки CRUD-додатків. Однак через меншу гнучкість Django і синхронність він не підходить.

FastAPI – це сучасний фреймворк для Python, створений для побудови високопродуктивних API, базуючись на типізації в Python (type hints). Він вирізняється швидкістю роботи через підтримку асинхронності (на рівні з Node.js та Go), завдяки використанню бібліотеки Starlette, і є одним із найшвидших

Python-фреймворків. FastAPI забезпечує високу продуктивність розробки, скорочуючи час написання коду на 200-300% та зменшуючи кількість помилок на 40% [11]. Фреймворк також підтримує зручний механізм впровадження залежностей, що дозволяє ефективно управляти залежностями в коді. Крім того, FastAPI автоматично створює інтерактивну документацію, на підставі стандартів OpenAPI.

Таким чином, FastAPI є оптимальним вибором для розробки завдяки високій продуктивності, підтримці асинхронного програмування, автоматичній генерації документації та інтегрованим валідації даних.

1.3.4.2 Вибір фреймворку для роботи з LLM провайдерами.

Для роботи з великими мовними моделями кожен провайдер, (OpenAI, Cohere, Anthropic, тощо) пропонує власні бібліотеки для взаємодії з їх API. Вони дозволяють виконувати конкретні завдання, наприклад, генерацію тексту чи створення ембедінг-векторів, але є заточеними під роботу виключно зі своїм сервісом і кожен має різні інтерфейси, що ускладнює інтеграцію з іншими провайдерами або масштабування проєкту.

LangChain забезпечує єдиний інтерфейс для роботи з різними провайдерами. Наприклад, клас Runnable дозволяє створювати запити до будь-якого LLM провайдеру однаково і об'єднувати запити в ланцюги [12]. Крім того, LangChain пропонує додаткові можливості, як створення підказок, створення ланцюгів запитів, тощо.

Враховуючи ці переваги, LangChain обрано як основний фреймворк для «Scribe», оскільки він спрощує роботу з різними провайдерами.

1.3.4.3 Вибір фреймворку ORM.

Для Python існує кілька популярних фреймворків ORM, серед яких виділяються Django ORM і SQLAlchemy. Вибір ORM є важливим, оскільки саме цей інструмент забезпечує ефективну взаємодію між об'єктами та базою даних.

Django ORM забезпечує простий і зрозумілий синтаксис для виконання CRUD-операцій, що робить його популярним вибором для швидкого старту проєктів. Він інтегрований у фреймворк Django, що дозволяє розробникам

уникати додаткової конфігурації. Однак його обмеження з роботою тільки з Django, знижує його гнучкість.

SQLAlchemy виділяється підтримкою різноманітних баз даних, включаючи MySQL, PostgreSQL, SQLite, Oracle, Microsoft SQL Server, та інші [13]. Завдяки цьому SQLAlchemy є ідеальним вибором для проєктів, які потребують масштабованості або використання різних баз даних, або ж їх зміни.

SQLAlchemy пропонує дві моделі взаємодії: низькорівневий SQL Core для ручного написання SQL-запитів і високорівневу ORM для автоматизації роботи з базою даних через Python-об'єкти.

Таким чином, було обрано SQLAlchemy, оскільки він поєднує підтримку широкого спектра баз даних, гнучкість, високу адаптивність до потреб проєкту і не є прив'язаним до окремого фреймворку.

1.3.4.4 Вибір бібліотеки для обробки завантажених документів.

Для обробки документів у різних форматах, таких як pdf, docx, txt та html, були розглянуті кілька бібліотек: PyPDF2, яка працює лише з pdf-файлами; python-docx, що спеціалізується на docx-документах. Ці бібліотеки обмежені підтримкою одного формату, що не відповідає вимогам проєкту.

Натомість було обрано Unstructured, універсальну бібліотеку для роботи з документами. Unstructured пропонує відкритий інструментарій для обробки різноманітних форматів, включаючи pdf, html, docx, та навіть зображення. Бібліотека також оптимізована для роботи з RAG [14].

1.3.4.5 Вибір фреймворку для побудови клієнтської частини.

Для побудови клієнтської частини вебресурсу існує кілька популярних фреймворків, серед яких React, Vue.js та Next.js.

React є однією з найбільш популярних бібліотек для створення динамічних, компонентних інтерфейсів користувача, що дозволяє створювати інтерактивні додатки. Однак для реалізації функцій серверного рендерингу або маршрутизації необхідно використовувати додаткові інструменти, що може ускладнити розробку.

Vue.js також є легким і зручним фреймворком, популярним завдяки простоті використання і можливості інтегруватися в існуючі проекти. Однак Vue.js не має вбудованих рішень для деяких аспектів, як, наприклад, маршрутизація на сервері.

Next.js є фреймворком на базі React, який пропонує вбудовані можливості для серверного рендерингу, статичної генерації сторінок, а також маршрутизації. Ці функції дозволяють значно покращити продуктивність та зменшити час завантаження сторінок.

Завдяки зручності в налаштуванні, оптимізації і маршрутизації Next.js є ідеальним вибором для створення клієнтської частини вебресурса.

1.3.4.6 Вибір клієнтської бібліотеки CSS стилів.

Передусім, вибір бібліотеки стилів CSS залежить від естетичних потреб проекту. Є кілька популярних рішень, таких як Bootstrap, Tailwind CSS та Material-UI (MUI).

Bootstrap – це класичний фреймворк, який надає велику кількість готових компонентів і стилів для швидкої розробки, однак він може обмежувати творчість через свою фіксовану структуру.

Tailwind CSS, з іншого боку, є фреймворком, який пропонує більшу гнучкість у стилізації, але вимагає більше часу для налаштування і може бути складним для розробників, які передусім не працюють з розробкою клієнтських частин.

Material-UI (MUI) – це бібліотека компонентів, що базується на принципах Material Design (принципи дизайну створені Google) і надає безліч готових інтерфейсних елементів з можливістю кастомізації. MUI інтегрується з React і дозволяє швидко створювати інтерфейси, не втрачаючи гнучкості.

Вибір на користь MUI був зроблений завдяки зручній документації, широкому набору компонентів і особистих вподобань.

1.3.5 Вибір системи управління базою даних.

Вибір системи управління базою даних (СУБД) є важливою частиною розробки вебресурсу, враховуючи специфіку розробки RAG систем, де кількість окремих СУБД може сягати двох.

В таких системах одна з СУБД відповідає за управління реляційною базою даних, яка зберігає, наприклад, інформацію о користувачеві. Тим часом інша СУБД відповідає за управління векторною базою даних, що зберігає ембеддінг-вектори. Ембеддінг-вектори представляють з себе великі за розмірністю числові вектори і потребують зберігання у відповідно оптимізованих векторних базах даних.

1.3.5.1 Розгляд системи управління реляційною базою даних.

Розглянемо три популярні системи управління реляційними базами даних: MySQL, PostgreSQL та SQLite.

MySQL добре підходить для масштабованих додатків із високими вимогами до швидкості та підтримки великої кількості користувачів, але серверна архітектура та складність налаштування роблять цю базу даних надлишковою для невеликих проєктів передусім орієнтованих на власне розгортання.

PostgreSQL пропонує розширені можливості для складних транзакцій і високих навантажень, але ресурсомісткість і необхідність серверного процесу також не виправдовують себе для власного розгортання.

Натомість SQLite є оптимальним вибором для вебресурсу. Це легка й швидка реляційна база даних яка не потребує серверного процесу, а всі дані зберігаються в одному файлі. SQLite забезпечує високу надійність і повну підтримку SQL, залишаючись кросплатформним рішенням із стабільним форматом файлу, який розробники зобов'язуються підтримувати до 2050 року [15].

Саме ці вищезазначені переваги роблять SQLite ідеальною базою даних для вебресурсів з можливістю власного розгортання, яким і є вебресурс «Scribe».

1.3.5.2 Розгляд системи управління векторною базою даних.

Для вебресурса, що тісно працює з ембеддінг-векторами, розглянемо три популярні системи управління векторними базами даних (СУБД): Weaviate, Pinecone та ChromaDB.

Weaviate пропонує інтеграцію з різними векторними моделями та можливість створення гібридного пошуку, поєднуючи семантичний і традиційний

пошук. Однак Weaviate потребує складнішої серверної архітектури та може бути надлишковим для власного розгортання.

Pinescone фокусується на високій продуктивності та низькій затримці для пошуку у великих наборах векторів. Але Pinescone є платформою як послуга (PaaS), що робить її залежною від інтернет-з'єднання та сторонньої інфраструктури. Таким чином, вона є не придатною для роботи з вебресурсом з можливістю власного розгортання.

ChromaDB є оптимальним вибором для вебресурсу. Це відкрита СУБД, яка забезпечує просте зберігання та пошук ембеддінг-векторів із метаданими. ChromaDB підтримує високу продуктивність, гнучкість у використанні (працює як окремий процес, так і в серверному режимі), ідеально інтегрується через Python і JavaScript/TypeScript, а також має відкритий код, ліцензований під Apache 2.0. ChromaDB підтримує запуск у Jupyter Notebook, що спрощує її використання для тестування та локальних проєктів, не залежних від інтернет-з'єднання [16]. Ці переваги роблять ChromaDB найкращим вибором для вебресурсу з можливістю власного розгортання, яким є «Scribe».

1.3.6 Вибір патернів проектування.

Проектування програмного забезпечення потребує використання перевірених практик, які сприяють розв'язуванню типових проблем і підвищують гнучкість та масштабованість системи.

1.3.6.1 Медіатор.

Важливим патерном при розробці є медіатор, який суттєво впливає на структуру проєкта.

Призначення медіатора є наступним: «Визначити об'єкт, який інкапсулює спосіб взаємодії набору об'єктів. Медіатор сприяє слабкому зв'язуванню, запобігаючи прямому посиланню об'єктів один на одного, і дозволяє змінювати їх взаємодію незалежно» [17, с. 273].

Цей патерн є важливим при розробці, оскільки в FastAPI не має чіткого розподілу на представлення та контролер, тому роль і представлення і контролера

можуть зберігатися в одній функції, яку можна назвати представленням-контролером.

Такий підхід є не найкращим, оскільки розмір функції представлення-контролера стає великим, крім того ролі представлення і контролера змішуються в одне ціле. Це є очевидним антипатерном, адже змішує ролі представлення і контролера в одному об'єкті.

По-друге, без медіатора, контролер неминуче тісно прив'язується до представлення-контролера, що призводить до неможливості зміни представлення. Наприклад, якщо ми хочемо надати користувачам консольний інтерфейс або базовий GUI, нам доведеться повністю дублювати код.

Для вирішення вищезазначених проблем була обрана готова асинхронна імплементація медіатора під назвою `mediatr`. Цей медіатор працює через створення команд, що містять певну інформацію, та окремих обробників, які обробляють ці команди. Завдяки цьому логіка контролерів для ендпоїнтів (представлень) була винесена до обробників, а ендпоїнти лише збирають та валідують інформацію, формують команду та надсилають її медіатору для подальшої обробки.

Варто зазначити, що з винесенням контролерів теж виноситься і всі залежності представлення-контролера до обробників медіатора. Після цього представлення залежать виключно від медіатору. Тіла функцій представлень стають мінімальними, часто з лише 2-3 рядками коду (див. лістинг 1.1).

Лістинг 1.1 – Приклад тіла функції ендпоїнта-представлення

```
@router.post(
    <</>,
    response_model=ApiKeyResponseModel,
    status_code=status.HTTP_201_CREATED
)
@Inject
def add_api_key(
    item: ApiKeyPostModel,
    mediatr: Mediator = Depends(Provide[Container.mediatr])
):
    command = ApiKeyCredentialCommand(**item.model_dump())
    return mediatr.send(command)
```

Таким чином, представлення-контролери FastAPI були чітко розділені на контролери (обробники медіатора) та представлення (функції, що визначають ендпоїнти).

Оскільки контролери і їх залежності винесені окремо до медіатора, можна у разі чого легко відкинути API і змінити інтерфейс, наприклад, на консольний, GUI чи будь-який інший, оскільки роль інтерфейсу обмежується лише валідацією даних та створенням команд.

1.3.6.2 Впровадження залежностей.

Впровадження залежностей є ключовим і необхідним патерном у вебресурсу. Для цього була обрана готова імплементація з рір під назвою `dependency-injector`.

За допомогою впровадження залежностей всі залежності у вебресурсу стають явними, а не прихованими, оскільки вони визначаються в одному класі-контейнері, до якого звертаються класи та функції.

Імпліцитні залежності мають серйозний недолік: вони тихо ініціалізуються і визначаються в самих класах або функціях.

Припустимо, що в функції, на якій залежить 5 функцій і 10 класів було змінено, або додано один параметр. При використанні імпліцитних залежностей буде потрібно вручну змінити новий параметр в кожному з 5 функцій і 10 класів. Більш того, оскільки імпліцитні залежності визначаються тихо, можна забути про їх існування в одній з функцій, або класі, про що стане відомо виключно під час виконання.

Використовуючи впровадження залежностей, а саме через клас-контейнер залежностей, достатньо змінити лише одну строку коду, і ці зміни автоматично відобразяться у всіх залежних функціях та класах. Це забезпечує набагато більшу прозорість і керованість кодом.

Варто теж зазначити про взаємодію впровадження залежностей з медіатором, а саме про те, що залежності впроваджуються до конструкторів класів-обробників.

1.3.6.3 Репозиторій.

Репозиторій – це патерн, який є абстракцією над зберіганням даних. Він приховує всі рутинні деталі доступу до даних, надаючи інтерфейс, який виглядає так, ніби всі дані зберігаються в оперативній пам'яті [9]. Цей підхід дозволяє зосередитися на бізнес-логіці, а не на складнощах зберігання та доступу до даних. Патерн репозиторію спрощує тестування та роботу з різними джерелами даних, адже зміни в зберіганні даних не впливають на основний код програми. Це також знижує дублювання коду, оскільки забезпечує централізований доступ до даних у вигляді одного інтерфейсу.

Варто зазначити, що у вебресурсі було імплементовано єдиний універсальний (generic) репозиторій для всіх об'єктів SQLAlchemy.

1.3.6.4 Unit of Work.

Unit of Work – це патерн, компаньйон патерну репозиторію, який обгортає його та групує кілька операцій в одну транзакцію. Це забезпечує цілісність та узгодженість змін у базі даних, дозволяючи скасувати всі операції в разі помилки. Патерн дозволяє обробляти взаємодії з базою даних в рамках одного об'єкта, що сприяє зручності роботи з транзакціями.

Плюси включають зниження кількості запитів до бази та підвищення продуктивності за рахунок групування операцій, а також забезпечення атомарності і консистентності змін.

Варто зазначити, що в проєкті було реалізовано обгортку UoW над об'єктом Session з SQLAlchemy (який за своєю природою також є UoW) із використанням контекстного менеджера Python.

Висновки до розділу 1

На основі проведеного аналізу та вибору технологій, вебресурс має всі необхідні компоненти для створення ефективного та функціонального вебресурсу, який відповідає вимогам RAG-системи.

Як архітектурний шаблон була обрана монолітна архітектура, що забезпечує простоту розробки та розгортання, особливо враховуючи можливість власного розгортання. Архітектура портів і адаптерів додає гнучкість та ізоляцію компонентів, полегшуючи масштабування та підтримку системи.

Python був обраний для серверної частини завдяки його популярності в сфері штучного інтелекту та наявності бібліотек для машинного навчання. TypeScript для клієнтської частини забезпечує надійність коду та сумісність з існуючими веб-технологіями.

Вибір фреймворків та бібліотек також був ретельно обґрунтований. FastAPI забезпечує високу продуктивність та підтримку асинхронності, що є критично важливим для обробки запитів до LLM. LangChain спрощує взаємодію з різними провайдерами LLM, а SQLAlchemy та ChromaDB забезпечують ефективне зберігання даних у реляційній та векторній базах відповідно.

Патерни проектування, такі як медіатор, впровадження залежностей, UoW та репозиторій, додають гнучкість та керованість кодом. Медіатор дозволяє чітко розділити контролери та представлення, покращуючи структуру коду. Впровадження залежностей забезпечує прозорість та керованість залежностями в проєкті, а патерн репозиторію та UoW спрощують роботу з даними.

Отже, «Scribe» має основу для розробки RAG-системи, яка поєднує в собі ефективність, гнучкість та прозорість. Вибір технологій та архітектурних рішень забезпечує надійну та масштабовану систему, здатну обробляти запити до LLM та надавати точні та актуальні відповіді, спираючись на зовнішні джерела знань.

2 ПРОЕКТУВАННЯ ВЕБРЕСУРСУ З ВИКОРИСТАННЯМ RETRIEVAL-AUGMENTED GENERATION З ПІДТРИМКОЮ УКРАЇНСЬКОЇ МОВИ

2.1 Мета та завдання вебресурсу

Мета: Розробити систему, яка використовує RAG для надання точних та актуальних відповідей на запити користувачів, інтегруючи зовнішні джерела даних та підтримуючи різні великі мовні та ембеддінг-моделі з підтримкою української мови.

Кінцева аудиторія системи: індивідуальні користувачі, які шукають зручний та гнучкий спосіб розширення можливостей великих мовних моделей від різних провайдерів через інтеграцію з зовнішніми джерелами даних.

Головними завданням вебресурсу є:

- 1) інтеграція зовнішніх джерел даних для актуалізації відповідей (статті, документи);
- 2) підтримка різних провайдерів мовних та ембеддінг-моделей для гнучкості та адаптації під потреби користувачів;
- 3) персоналізація відповідей, налаштування системи під індивідуальні вимоги;
- 4) забезпечення точності та надійності відповідей через пошук та генерацію тексту;
- 5) забезпечення відкритого доступу до витягнутого з джерел контексту при генерації відповідей.

В системі можна виділити основний інформаційний потік «Створення чату з документами» (рис. 2.1).

На рис. 2.1 показано, як вебресурс обробляє різні вхідні дані, зокрема API ключі провайдерів, налаштування мовної моделі, параметри обробки документів, самі документи, системну підказку та параметри векторного пошуку. Ці вхідні потоки дозволяють інтегрувати зовнішні сервіси та налаштувати процеси обробки тексту. Як результат, система створює інтерактивний чат з документами, що дає

можливість користувачам отримувати швидкі та релевантні відповіді в реальному часі на основі заданих конфігурацій.

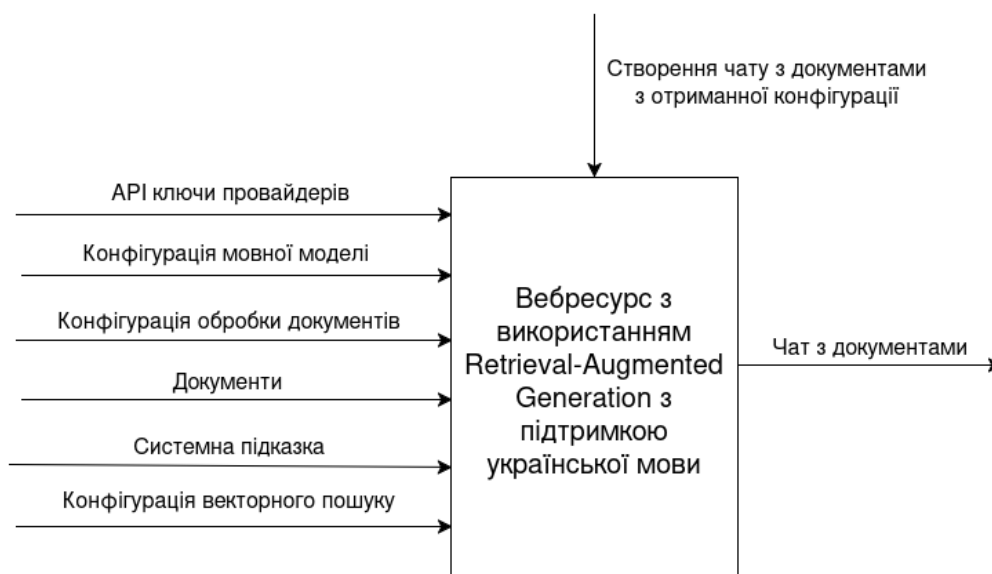


Рисунок 2.1 – Інформаційний потік «Створення чату з документами»

2.2 Типи користувачів

Система має можливість власного розгортання, тобто розгортається та обслуговується безпосередньо на інфраструктурі користувача. Такий підхід забезпечує незалежність від сторонніх сервісів, дає змогу контролювати всі аспекти експлуатації та конфіденційності даних, а також адаптувати середовище до конкретних вимог.

У зв'язку з цим можна виділити лише один основний тип користувача.

Основний користувач (хостинг-система) – це особа, що відповідає за розгортання, налаштування та експлуатацію системи. Основний користувач має повний доступ до всіх параметрів конфігурації, включаючи інтеграцію з зовнішніми API, вибір мовної моделі, способів обробки документів, конфігурацію векторного пошуку та налаштування інтерфейсу користувача. Крім цього, він здійснює адміністрування локального сховища.

2.3 Функціональні та нефункціональні вимоги

2.3.1 Функціональні вимоги.

Нижче визначено функції (англ. Function, F) та функціональні вимоги (англ. Functional Requirement, FR) для вебресурсу з використанням Retrieval-Augmented Generation з підтримкою української мови «Scribe».

F1. Функція запуску вебресурсу.

FR1.1. Після запуску вебресурсу на системі користувача в домашній директорії повинна з'явитися директорія `.scribe`.

FR1.1.1. Директорія `.scribe` повинна містити файл `scribe.key`, який містить згенерований ключ для шифрування і дешифрування даних.

FR1.1.2. Директорія `.scribe` повинна містити директорію `logs` з логами роботи вебресурсу.

FR1.1.3. Директорія `.scribe` при запуску вебресурсу повинна самоочищуватися і видаляти всі файли окрім `scribe.key` та логів.

FR1.1.4. Директорія `.scribe` при запуску вебресурсу потрібна відтворювати `scribe.key` та директорію `logs` при їх видаленні.

FR1.2. Після запуску на системі користувача працюють окремо клієнтська і серверна частини, до яких користувач має доступ через веббраузер.

F2. Функція додавання API ключа провайдерів.

FR2.1. Користувач повинен ввести назву та API ключ у відповідні поля.

FR2.2. Введений API ключ повинен бути зашифрованим на підставі створеного файлу ключа (див. FR1.1.1).

FR2.3. Після успішного додання, користувач перенаправляється на сторінку з усіма доданими ключами.

FR2.4. Користувач може лише змінити ім'я створеного API ключа.

FR2.5. Користувач може видалити створений API ключ.

F3. Функція додавання системної підказки.

FR3.1. Користувач повинен ввести назву і зміст системної підказки.

FR3.2. Після успішного додавання, користувач перенаправляється на сторінку з усіма доданими системними підказками.

FR3.3. Користувач може змінити всі параметри створеної системної підказки.

FR3.4. Користувач може видалити створену системну підказку.

F4. Функція додавання конфігурації обробки документів.

FR4.1. Користувач повинен обов'язково ввести назву конфігурації обробки документів.

FR4.1.1. Якщо користувач ввів виключно назву, буде використано сценарій обробки документів за замовчуванням.

FR4.2. Користувач може необов'язково ввести (додатково до імені конфігурації): стратегію розбиття на частин, обрати декілька постобробників, максимальний розмір частини документа в символах, кількість символів бажаного розміру частини, кількість символів прилягання, увімкнення прилягання для всіх частин.

FR4.3. Після успішного додавання, користувач перенаправляється на сторінку з усіма доданими конфігураціями.

FR4.4. Користувач може змінити всі параметри створеної конфігурації.

FR4.5. Користувач може видалити створену конфігурацію.

F5. Функція додавання конфігурації семантичної обробки документів.

FR5.1. Користувач повинен обов'язково ввести назву конфігурації, максимальну кількість речень в документі, значення порогу косинусної відстані.

FR5.2. Після успішного додавання, користувач перенаправляється на сторінку з усіма доданими семантичними конфігураціями обробки тексту.

FR5.3. Користувач може змінити всі параметри створеної конфігурації.

FR5.4. Користувач може видалити створену конфігурацію.

F6. Функція додавання ембеддінг-моделі.

FR6.1. Користувач може додати ембеддінг-модель, виключно після створення відповідного API ключа (див. F2).

FR6.2. Користувач обов'язково повинен обрати одну з перелічених ембеддінг-моделей.

FR6.3. Користувач повинен обрати на якому девайсі буде розгорнута модель, якщо вона локальна (GPU або CPU).

FR6.4. Після успішного додання, користувач перенаправляється на сторінку з усіма доданими ембеддінг-моделями.

FR6.5. Користувач може змінити всі параметри створеної ембеддінг-моделі.

FR6.6. Користувач може видалити створену ембеддінг-модель.

F7. Функція додавання мовної моделі.

FR7.1. Користувач може додати мовну модель, виключно після створення відповідного API ключа (див. F2).

FR7.2. Користувач обов'язково повинен обрати одну з перелічених мовних моделей.

FR7.3. Користувач може необов'язково ввести (додатково до мовної моделі) наступні параметри: базове посилання, top-p, максимальну кількість токенів, максимальну кількість повторів, температуру і стоп послідовності.

FR7.4. Після успішного додання, користувач перенаправляється на сторінку з усіма доданими мовними моделями.

FR7.5. Користувач може змінити всі параметри створеної мовної моделі.

FR7.6. Користувач може видалити створену мовну модель.

F8. Функція додавання векторної колекції.

FR8.1. Користувач може додати векторну колекцію, виключно після створення відповідної ембеддінг-моделі.

FR8.2. Користувач обов'язково повинен ввести функцію виміру відстані і назву колекції.

FR8.3. Після успішного додання, користувач перенаправляється на сторінку з усім доданими векторними колекціями.

FR8.4. Користувач може видалити створену векторну колекцію.

F9. Функція завантаження документів до векторної колекції.

FR9.1. Користувач може завантажувати документи, виключно після створення конфігурації обробки (див. F4 та F5).

FR9.2. Користувач може завантажувати документи у форматі .pdf, .txt, .odt, .md при звичайній обробці.

FR9.3. Користувач може завантажувати виключно .pdf при семантичній обробці.

FR9.4. Користувач може завантажувати веб-сторінки за посиланням при звичайній обробці.

FR9.5. Користувач може одночасно завантажувати декілька сторінок і файлів при звичайній обробці.

FR9.6. Користувач може одночасно завантажувати декілька файлів при семантичній обробці.

FR9.7. Після завантаження документів користувач може «заглянути» до 3 перших завантажених частин.

FR9.7.1. Завантажені частини повинні нести наступну інформацію: id, числовий вектор, текст частини, метадані.

FR9.8. Користувач може видаляти завантажені документи.

F10. Функція дитячого майданчика векторної колекції.

FR10.1. Користувач може обрати кількість результатів.

FR10.2. Користувач може ввести свій запит і отримати найближчі до цього запиту частини документів.

FR10.2.1. Користувач отримає стільки частин, скільки зазначено в кількості результатів.

FR10.2.2. Отримані частини містять всю інформацію з (див. FR9.7.1), але додається ще відстань.

F11. Функція створення базового чату.

FR11.1. Користувач може створити базовий чат виключно після створення мовної моделі (див. F7).

FR11.2. Користувач повинен обов'язково ввести назву і опис базового чату.

FR11.3. Користувач може необов'язково обрати (додатково до назви і опису): системну підказку і векторну колекцію.

FR11.4. Після успішного додання, користувач перенаправляється на сторінку з усім доданими базовими чатами.

FR11.5. Користувач може оновити створений базовий чат і змінити системну підказку, мовну модель і векторну колекцію.

FR11.6. Користувач може видалити створений базовий чат.

F12. Функція спілкування в базовому чаті.

FR12.1. Користувач може користуватися базовим чатом і спілкуватися з моделлю без контексту, якщо не надав векторну колекцію.

FR12.2. Користувач повинен отримувати відповідь від моделі у вигляді потоку.

FR12.3. Користувач повинен отримати контекстуалізовані відповіді, якщо надав векторну колекцію.

FR12.3.1. Користувач може обрати кількість частин документів, що будуть шукатися для контексту.

FR12.4. Разом з відповіддю моделі, користувач повинен отримувати частини документів, що були використані в контексті.

2.3.2 Нефункціональні вимоги.

Після визначення функції та функціональних вимог, перейдемо до визначення нефункціональних вимог (англ. Non-Functional Requirement, NFR).

NFR1. Вебресурс повинен бути розгорнутим на Unix системах.

NFR2. Підтримувані веббраузер: Chrome, Firefox.

NFR3. Вебресурс повинен перестворювати залежні файли при видаленні (ключ шифрування, директорія).

NFR4. Вебресурс повинен використовувати симетричний шифр.

NFR5. Вебресурс повинен використовувати base64-кодований 32-байтовий ключ.

NFR6. Вебресурс повинен вести логування дій вебсерверу.

NFR7. Вебресурс повинен мати підключення до інтернету.

NFR8. Вебресурс повинен бути зручним в користуванні.

2.4 Загальне моделювання вебресурсу

2.4.1 Прецеденти вебресурсу. Для ліпшого розуміння та демонстрації функціональних вимог була побудована наступна діаграма прецедентів вебресурсу (рис. 2.2).

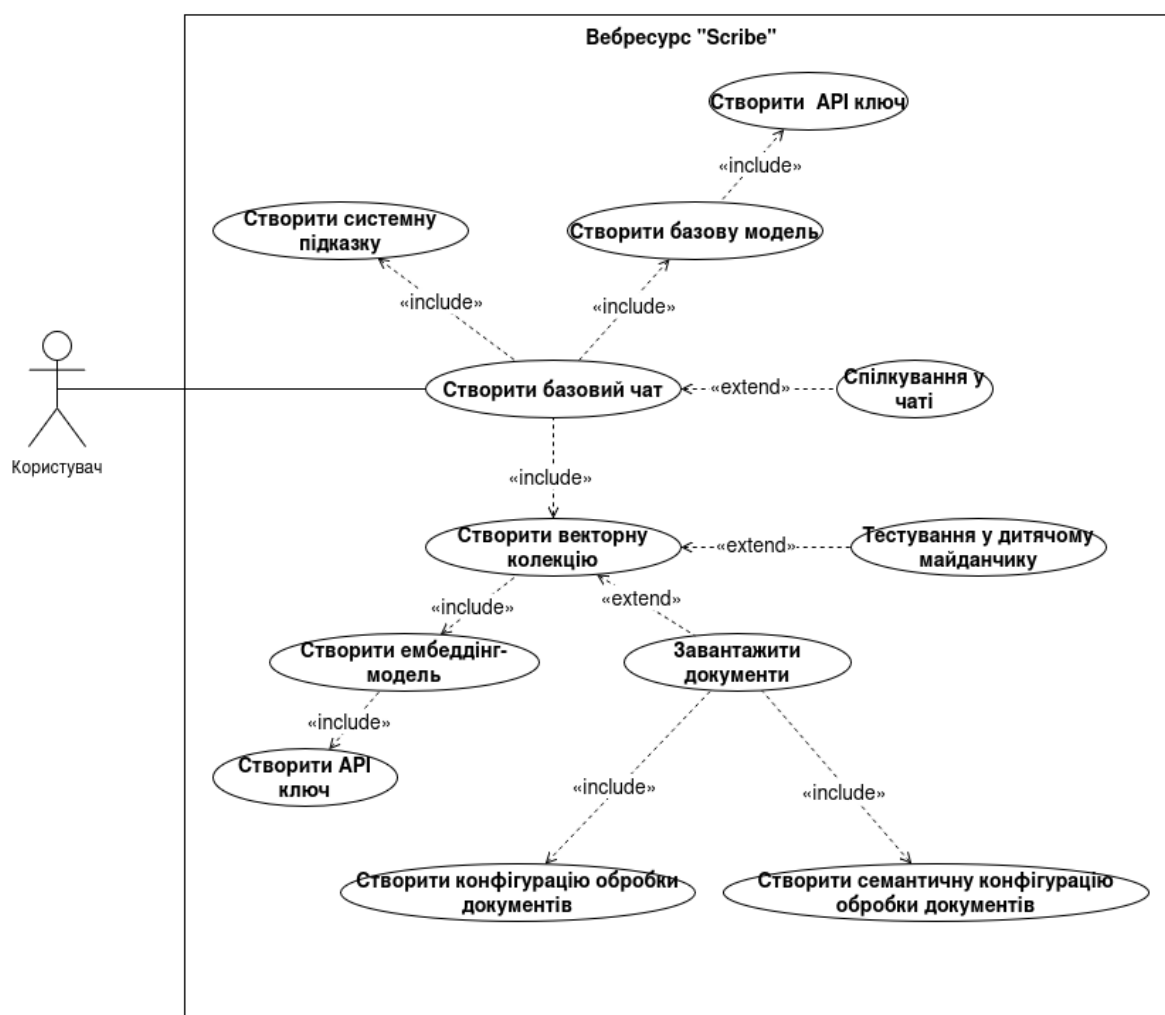


Рисунок 2.2 – Діаграма прецедентів вебресурсу «Scribe»

2.4.2 Ідентифікація архетипу вебресурсу. Проєкт є змішаним додатком (Mixed Application), оскільки поєднує в собі елементи як локального інсталюваного додатка, так і вебресурсу. Вебресурс виконується на локальному

пристрої користувача, але також надає вебінтерфейс і використовує вебсервер для взаємодії з користувачем через браузер.

2.4.3 Обмеження архітектури. Оскільки система є вебресурсом з можливістю власного розгортання, то основним протоколом комунікації є HTTP.

Через HTTP можливо спілкування на пряму з серверною частиною, або ж клієнтською. Через HTTP також спілкується між собою серверна частина з клієнтською і серверна частина з ChromaDB.

2.5 Уявлення інтерфейсу користувача

Для розробки вебресурсу і подальшої побудови мокапів, було визначено назви сторінок вебінтерфейсу. Крім того, до кожної сторінки і під-сторінки був присвоєний відповідний номер для легшої навігації. Результати представлені у табл. 2.1.

Таблиця 2.1 – Визначені сторінки вебресурсу і присвоєні номери

Назва сторінки	Присвоєний номер, <i>n</i>
Головна сторінка	0
Сторінка API ключів (створення: 0, перегляд: 1, зміна: 2)	1
Сторінка системних підказок (створення: 0, перегляд: 1, зміна: 2)	2
Сторінка конфігурацій обробки документів (створення: 0, перегляд: 1, зміна: 2)	3
Сторінка конфігурацій семантичної обробки документів (створення: 0, перегляд: 1, зміна: 2)	4
Сторінка ембеддінг-моделей (створення: 0, перегляд: 1, зміна: 2)	5
Сторінка мовних моделей (створення: 0, перегляд: 1, зміна: 2)	6
Сторінка векторних колекцій (створення: 0, перегляд: 1)	7
Сторінка дитячого майданчику векторної колекції	8
Сторінка базового чату (створення: 0, перегляд: 1, зміна: 2)	9
Сторінка розмови в базовому чаті	10

Почнемо з вітальної головної сторінки (рис. 2.3). Ця сторінка є навігаційним хабом. Ліва частина інтерфейсу містить вертикальну навігаційну панель з кнопками, що відповідають розділам налаштувань системи, зокрема: api-key, sys-prompt, doc-proc-cnf, sem-doc-proc-cnf, embed-model, chat-model, vec-col, base-chat, тощо.

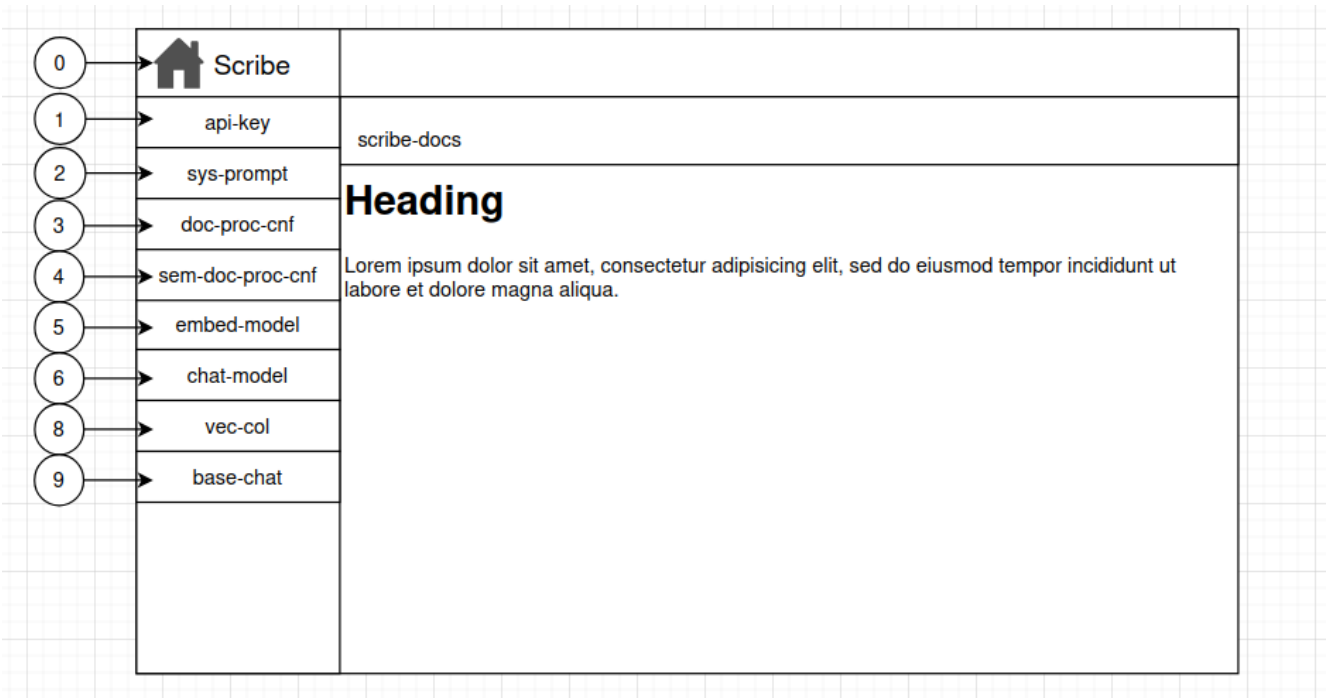


Рисунок 2.3 – Мокап головної сторінки вебресурсу

Варто зазначити, що деякі сторінки з зазначених в табл. 2.1 об’єднують в собі декілька під-сторінок, а саме під-сторінки: створення – 0, перегляду – 1 і зміни – 2.

Оскільки такі під-сторінки повторюється і відрізняється виключно значеннями які є у формі або перегляді, для зменшення надмірності буде представлено загальні мокап версії цих під-сторінок (рис. 2.4–2.6).

На *n.0* сторінці створення користувач записує дані до форми. Зі сторінки створення можна перейти до сторінки перегляду *n.1*. При успішному створенні користувач автоматично перенаправляється до сторінки перегляду *n.1* (рис 2.4).

0	Scribe	n.1
api-key	lorem-ipsum / add	
sys-prompt		
doc-proc-cnf	Lorem Ipsum Ipsum ▼	
sem-doc-proc-cnf	Lorem Ipsum provide lorem Ipsum	
embed-model		
chat-model		
vec-col		
base-chat		
	Submit	

Рисунок 2.4 – Мокап загальної підсторінки створення *n.0*

На *n.1* сторінці перегляду користувач може переглядати всі створенні записи, додати новий запис *n.0*, або ж змінити запис *n.2* (рис. 2.5).

0	Scribe	
api-key	lorem-ipsum	+ ← n.0
sys-prompt	27.11.2024 lorem-ipsum	🗑️ ↗️ ← n.2
doc-proc-cnf	27.11.2024 lorem-ipsum	🗑️ ↗️
sem-doc-proc-cnf		
embed-model		
chat-model		
vec-col		
base-chat		
	Previous 1 2 3 Next	

Рисунок 2.5 – Мокап загальної підсторінки перегляду *n.1*

На $n.2$ сторінці зміни, користувач може змінити кожне поле форми, перейти до сторінки перегляду $n.1$. При успішній зміні користувач автоматично перенаправляється до сторінки перегляду $n.1$ (рис 2.6).

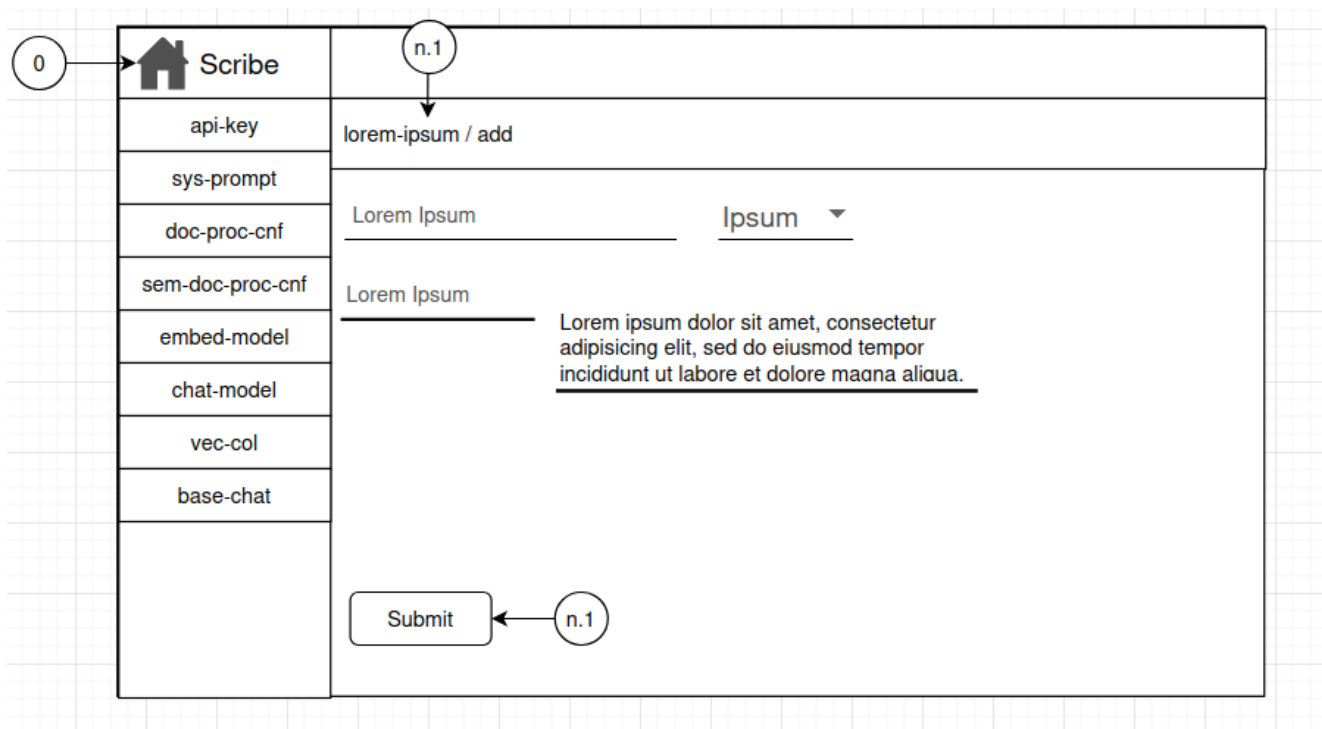


Рисунок 2.6 – Мокап загальної підсторінки зміни $n.2$

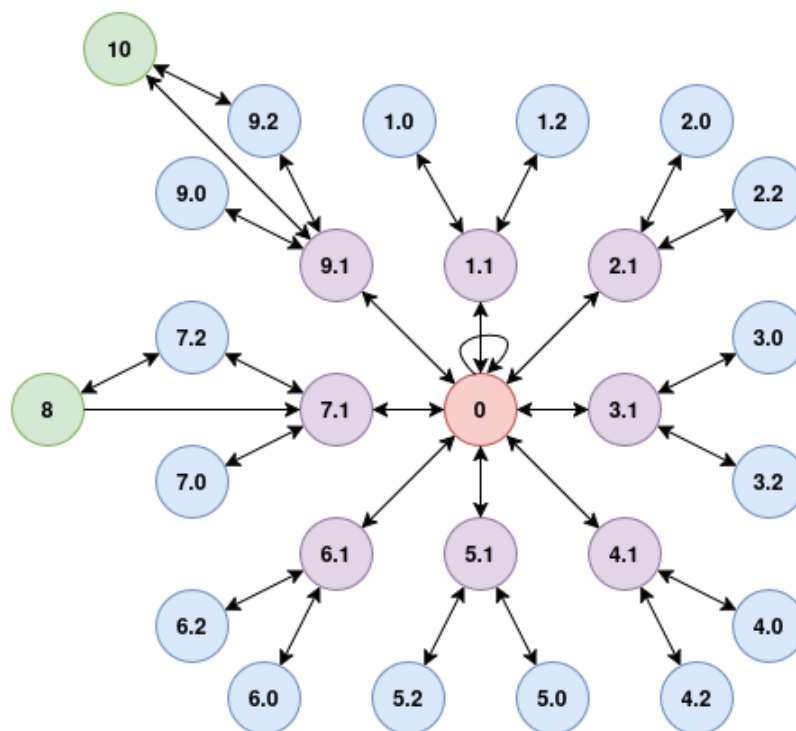
Перейдемо до розгляду сторінок векторної колекції і базового чату (додаток Б, рис. Б.1–рис. Б.3).

На сторінці зміни векторної колекції відбувається завантаження документів. Крім завантаження документів, на цій сторінці можна зазирнути на початкові документи колекції і перейти до сторінки дитячого майданчика векторної колекції.

На сторінці дитячого майданчика векторної колекції користувач може протестувати витягнення частин документів з векторної колекції на підставі запитів, обрати кількість результатів, видалити документи колекції, тощо.

На сторінці розмови в базовому чаті можна написати запит до мовної моделі, вибрати кількість частин документів, що будуть додані як контекст, отримати відповідь від мовної моделі і частини документів, що були додані як контекст, тощо.

Для ліпшого розуміння навігації, на рис. 2.7 представлена діаграма основних переходів між сторінками з табл. 2.1. Варто зазначити, що кольори на діаграмі позначають рівні вкаденості для кращого сприйняття.



Умовні позначки:

n – номер сторінки (1-10);
 $n.0$ – створення;
 $n.1$ – перегляд;
 $n.2$ – зміна.

Рисунок 2.7 – Діаграма основних переходів між сторінками вебресурсу

2.6 Логічне уявлення про вебресурс

На рис. 2.8 представлено логічне уявлення вебресурсу, яке відображає загальну архітектуру системи та взаємодію її основних компонентів. Діаграма ілюструє клієнтську частину, що відповідає за обробку дій користувача та формування запитів, які у форматі JSON надсилаються до серверної частини через API. Серверна частина, у свою чергу, виконує необхідну обробку запитів, взаємодіє з базою даних і повертає відповідь клієнту.

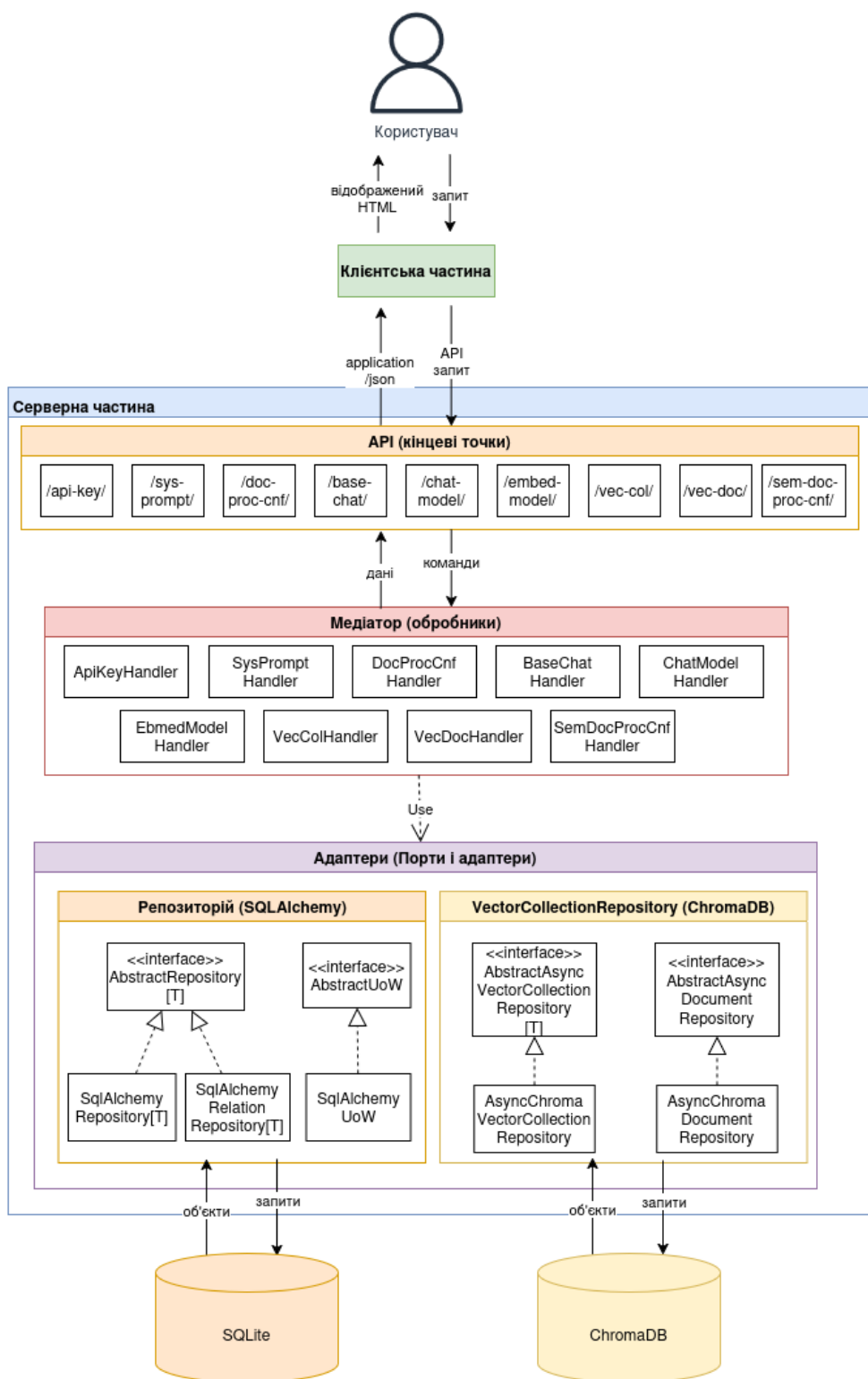


Рисунок 2.8 – Діаграма логічного представлення вебресурсу

Серверна частина складається з пакетів: `api`, що містить різні ендпоїнти для обробки запитів (`/api-key/`, `/sys-prompt/` та інші); `handlers`, що містить обробники медіатора (`ApiKeyHandler`, `SysPromptHandler` та інші), які залежать від портів в пакеті `adapters`. Пакет `adapters` відповідає за роботу з базами даних і містить

інтерфейси-порти репозиторієв, UoW та їх імплементацій-адаптерів для SQLAlchemy та ChromaDB відповідно. Для збереження та запитів до баз даних використовуються SQLite і ChromaDB. Структура вебресурсу з використанням Retrieval-Augmented Generation з підтримкою української мови реалізує патерн портів та адаптерів, що забезпечує модульність та гнучкість системи.

2.7 Уявлення розгортання вебресурсу

Діаграма розгортання вебресурсу показує архітектуру системи, розгорнутої на одному пристрої користувача. Клієнтський додаток працює на сервері next.js і спілкується з серверною частиною, реалізованим на сервері FastAPI, за допомогою REST (HTTP). Серверна частина взаємодіє з реляційною базою даних SQLite, використовуючи RDBMS, і з векторною базою даних ChromaDB через REST (HTTP). Усі компоненти функціонують у межах однієї системи користувача (рис. 2.9).

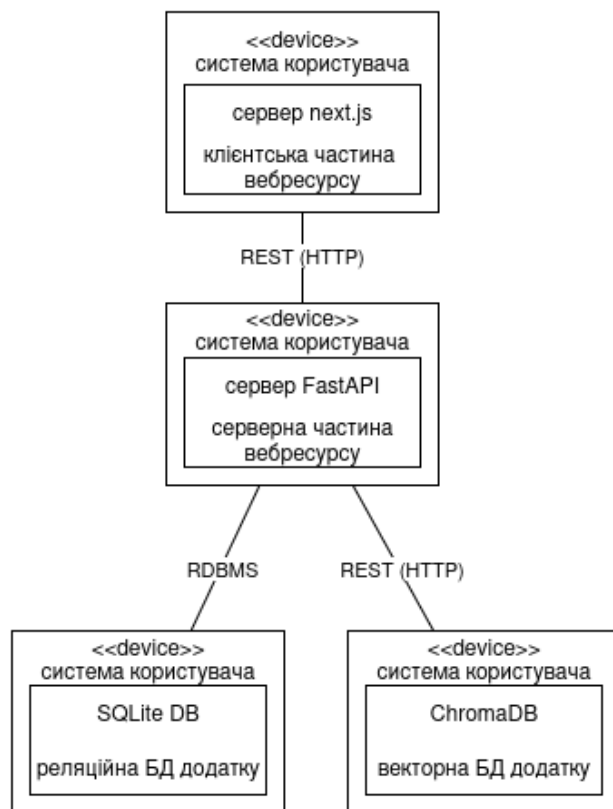


Рисунок 2.9 – Діаграма розгортання вебресурсу

2.8 Уявлення шарів вебресурсу

Діаграма шарів інформаційної системи демонструє багаторівневу архітектуру вебресурсу, що складається з презентаційного шару, бізнес-шару, шару збереження даних і шару бази даних (рис. 2.10).

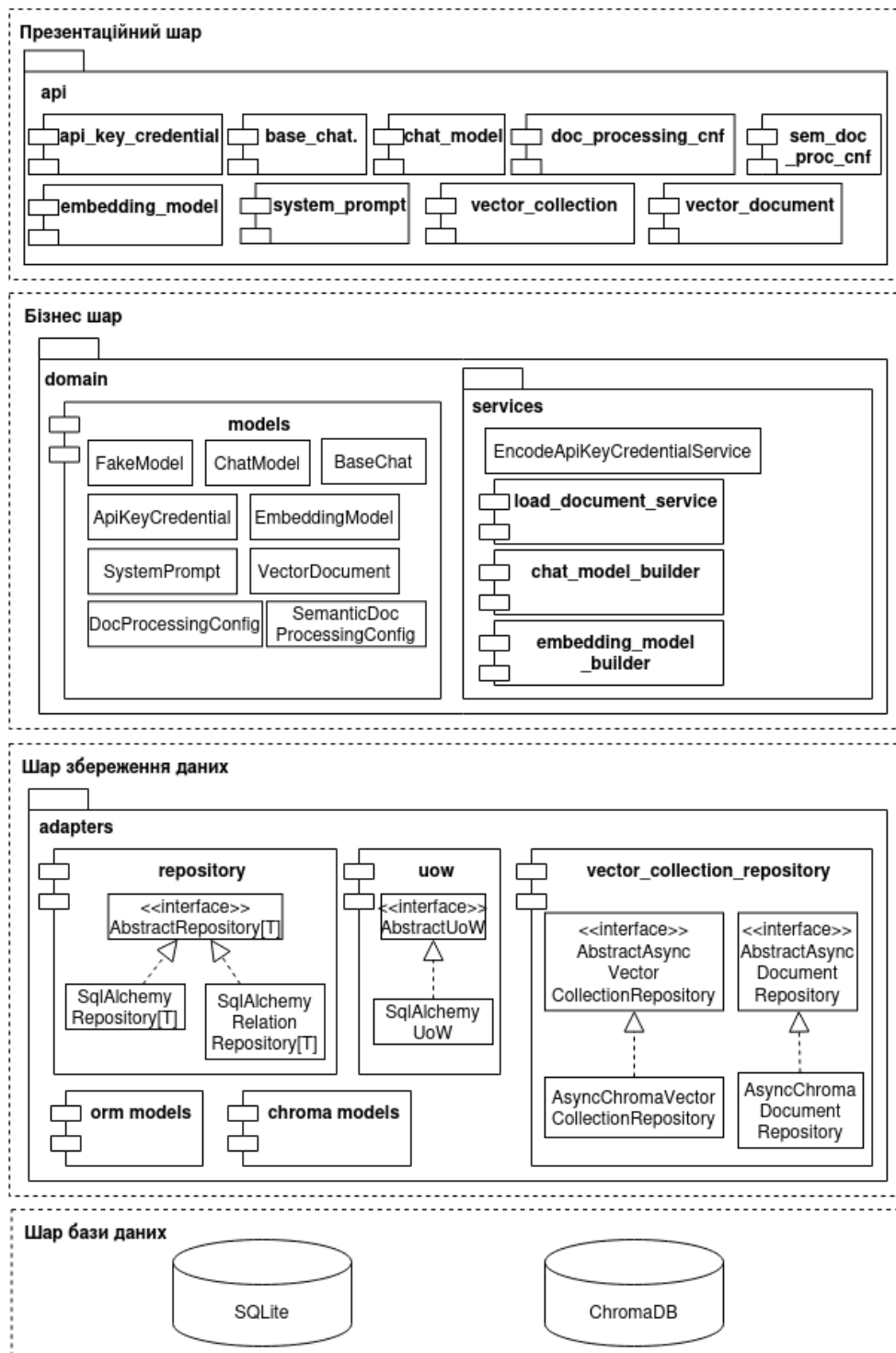


Рисунок 2.10 – Діаграма слоїв вебресурсу

Презентаційний шар відповідає за взаємодію з користувачем через API-ендпоїнти, такі як `api_key_credential`, `chat_model`, `embedding_model` тощо. Він ізольований від обробки даних і виконує роль точки входу в систему.

Бізнес-шар реалізує основні доменні моделі (`ChatModel`, `SystemPrompt`, `VectorDocument`) і сервіси для бізнес-логіки, зокрема ініціалізацію моделей та обробку облікових даних.

Шар збереження даних забезпечує абстракцію над сховищами, використовуючи `SQLAlchemy` для реляційної моделі та `ChromaDB` для векторної. Доступ реалізовано через відповідні інтерфейси та репозиторії.

Шар бази даних включає `SQLite` і `ChromaDB`, забезпечуючи фізичне зберігання даних. Він ізольований від логіки додатка за допомогою адаптерів.

2.9 Уявлення процесів вебресурсу

Основним процесом у вебресурсу з використанням `Retrieval-Augmented Generation` з підтримкою української мови є асинхронний запит користувача до базового чату. Цей процес має два можливих результата (якщо не включати виключення), які залежить від того, чи була під'єднана векторна колекція до чату.

Якщо ні – модель відповідає виключно спираючись на базу знань вкладену до моделі під час тренування провайдером.

Якщо ж векторна колекція надана, при запитах користувача відбувається векторний пошук по документах колекції, і відповідь моделі формується вже на підставі витягнутих даних.

Теж варто зазначити, що успішним результатом цього процесу в обох випадках є надання текстового потоку відповіді моделі.

Діаграма активності описує процес обробки запиту користувача до базового чату з більш бізнесової точки зору (рис. 2.11), а діаграма послідовності (рис. 2.12) описує той самий процес, але з більш детальної і технічної.

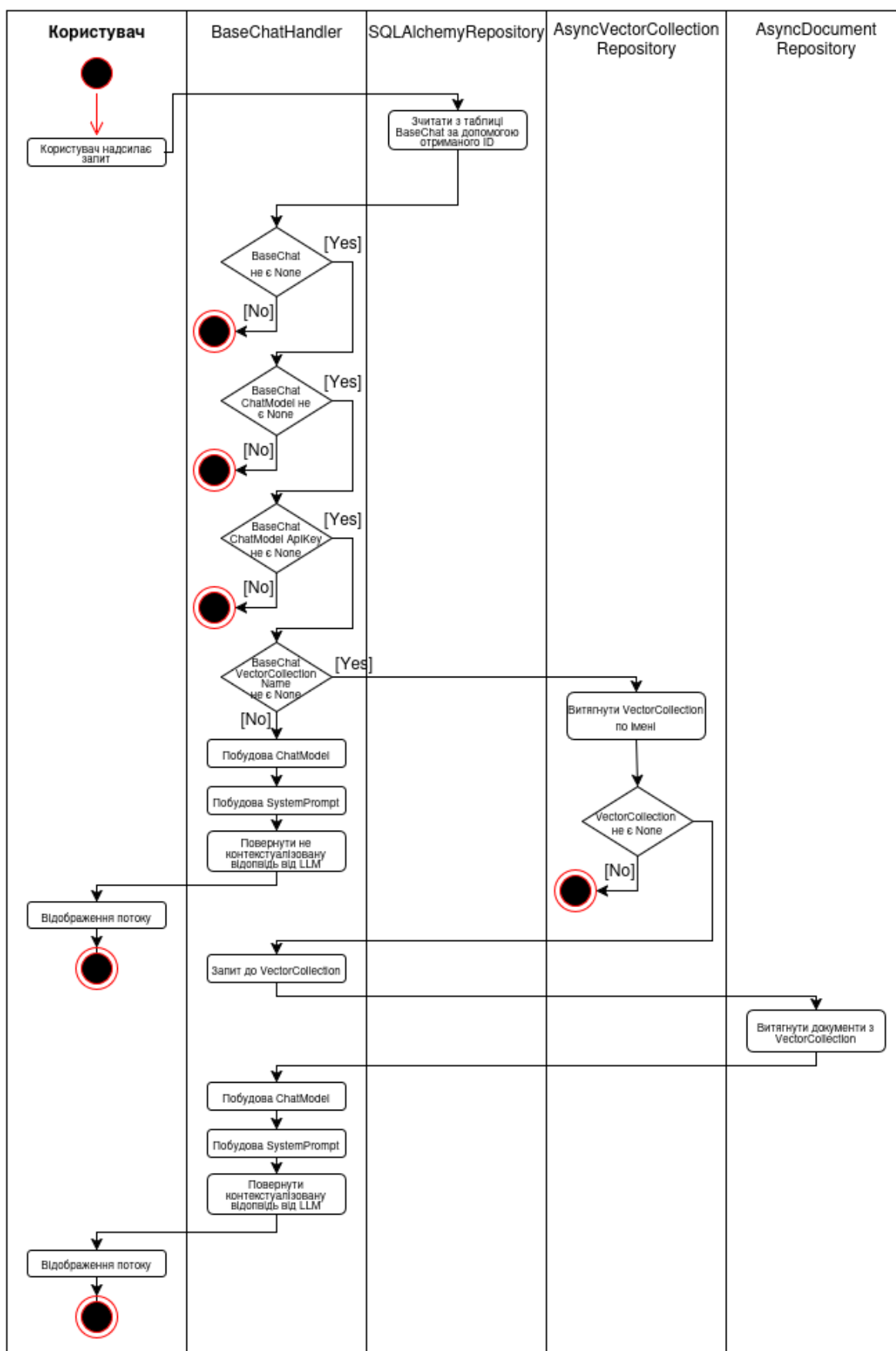


Рисунок 2.11 – Діаграма активності процесу обробки запита користувача до базового чату

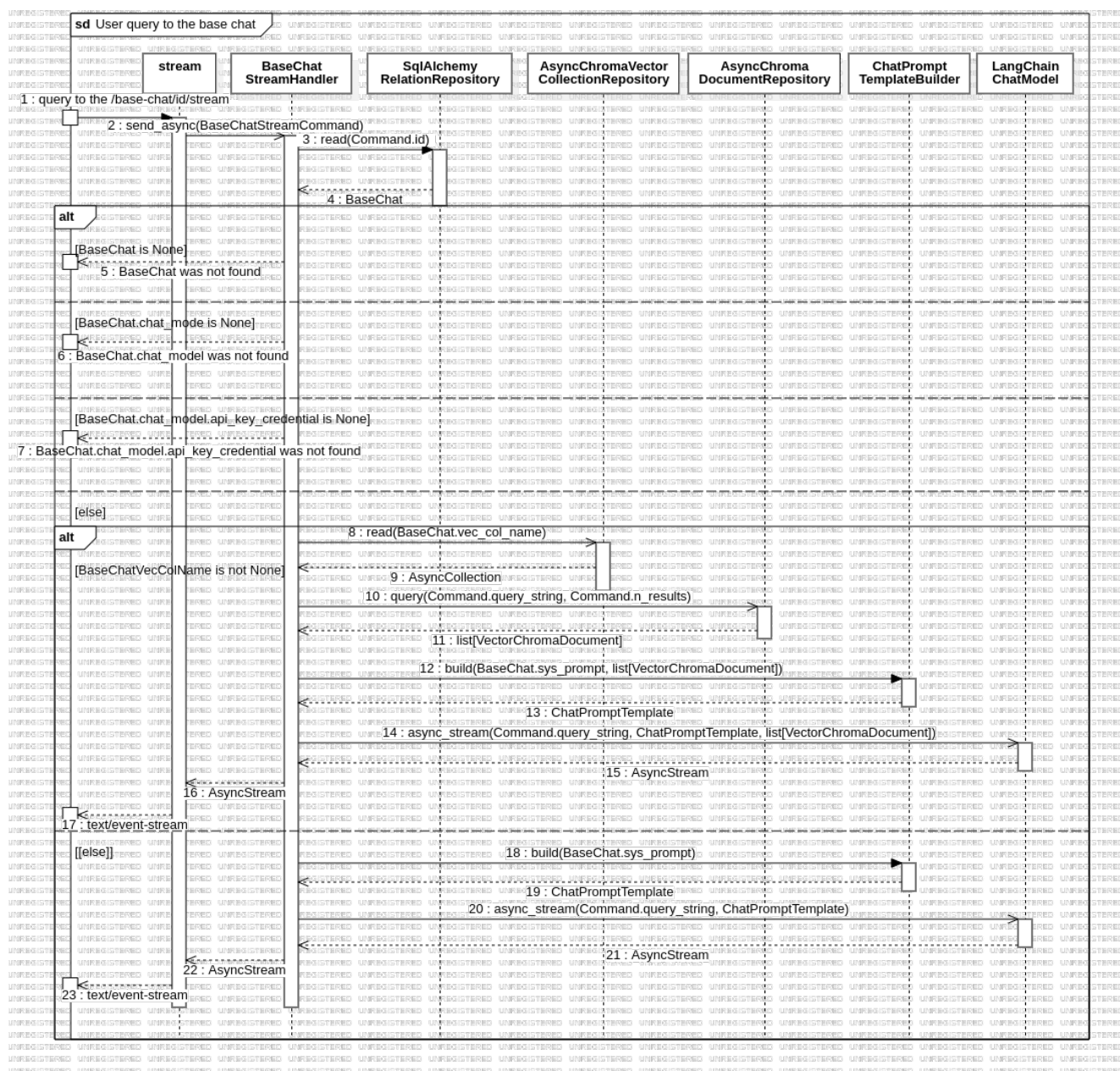


Рисунок 2.12 – Діаграма послідовності процесу обробки запита користувача до базового чату

2.10 Уявлення даних вебресурсу

На рис. 2.13 зображена діаграма класів доменних об'єктів вебресурсу. Діаграма доменних об'єктів містить кілька класів із різними взаємозв'язками, а саме: BaseChat, ChatModel, SystemPrompt, VectorCollection, EmbeddingModel, ApiKeyCredential, SemanticDocProcessingConfig, DocProcessingConfig, тощо.

BaseChat має агрегативний зв'язок з SystemPrompt, ChatModel, VectorCollection. VectorCollection має агрегативний зв'язок з EmbeddingModel, а EmbeddingModel і ChatModel мають агрегативний зв'язок з ApiKeyCredential.

SemanticDocProcessingConfig і DocProcessingConfig є окремими від інших класів.

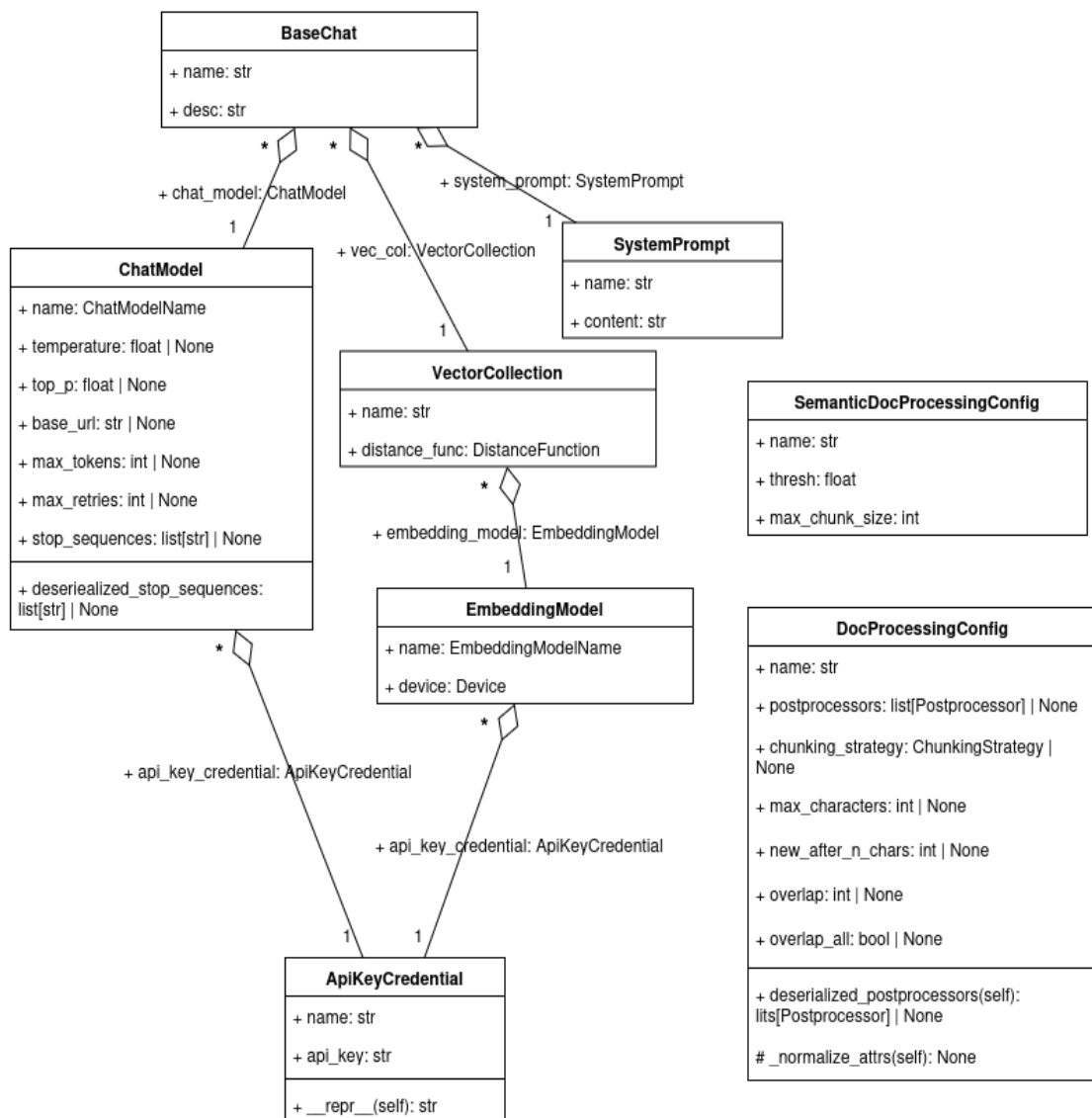


Рисунок 2.13 – Діаграма класів доменних об'єктів вебресурсу

На рис. 2.14 представлена схема реляційної бази даних. Як можна побачити, в таблицях додатково існують поля id і datetime. Це пов'язано з тим, що ці поля додаються через ORM.

Крім того, на діаграмі видно, первинні та зовнішні ключі, зв'язки між таблицями, тощо. Як видно зв'язки між таблицями зберігають структуру зв'язків між доменними об'єктами з рис. 2.13.

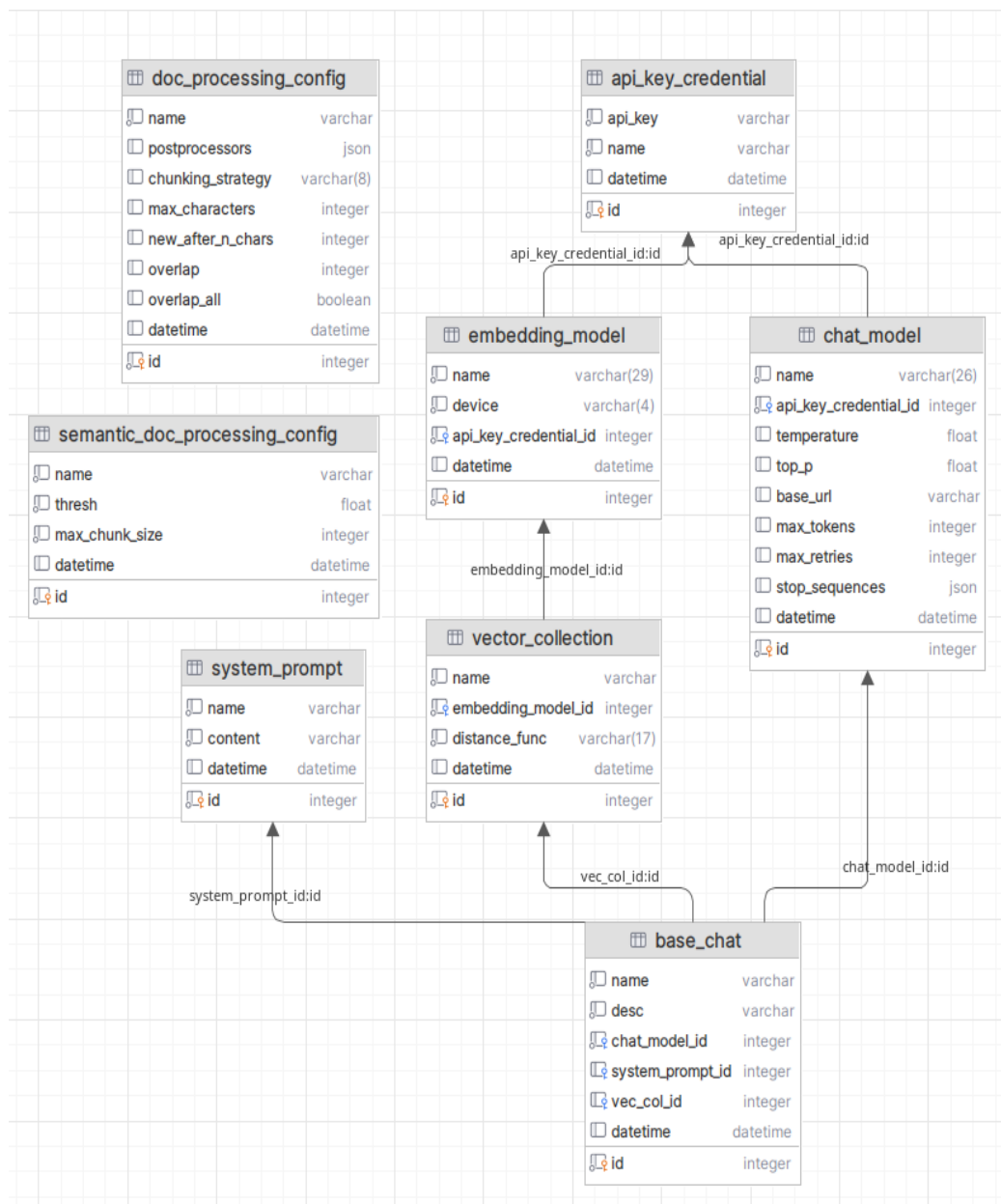


Рисунок 2.14 – Схема реляційної бази даних

2.11 Уявлення комунікацій вебресурсу

Діаграма комунікацій відображає тришарову архітектуру системи і взаємодію між шарами (рис. 2.15). Користувач взаємодіє з презентаційним шаром (Next.js на порту 3000), який рендерить сторінки через модулі типу api-key, base-

chat, chat-model тощо. Цей шар надсилає HTTP-запити до серверної частини (FastAPI на порту 8000), яка реалізує API та формує запити до обробників медіатора за допомогою CQRS. Серверна частина з'єднується з шаром збереження даних: SQLite через RDBMS та ChromaDB через HTTP (порт 8001). CQRS забезпечує розділення читання і змін – запити обробляються окремо від команд, що змінюють стан системи.

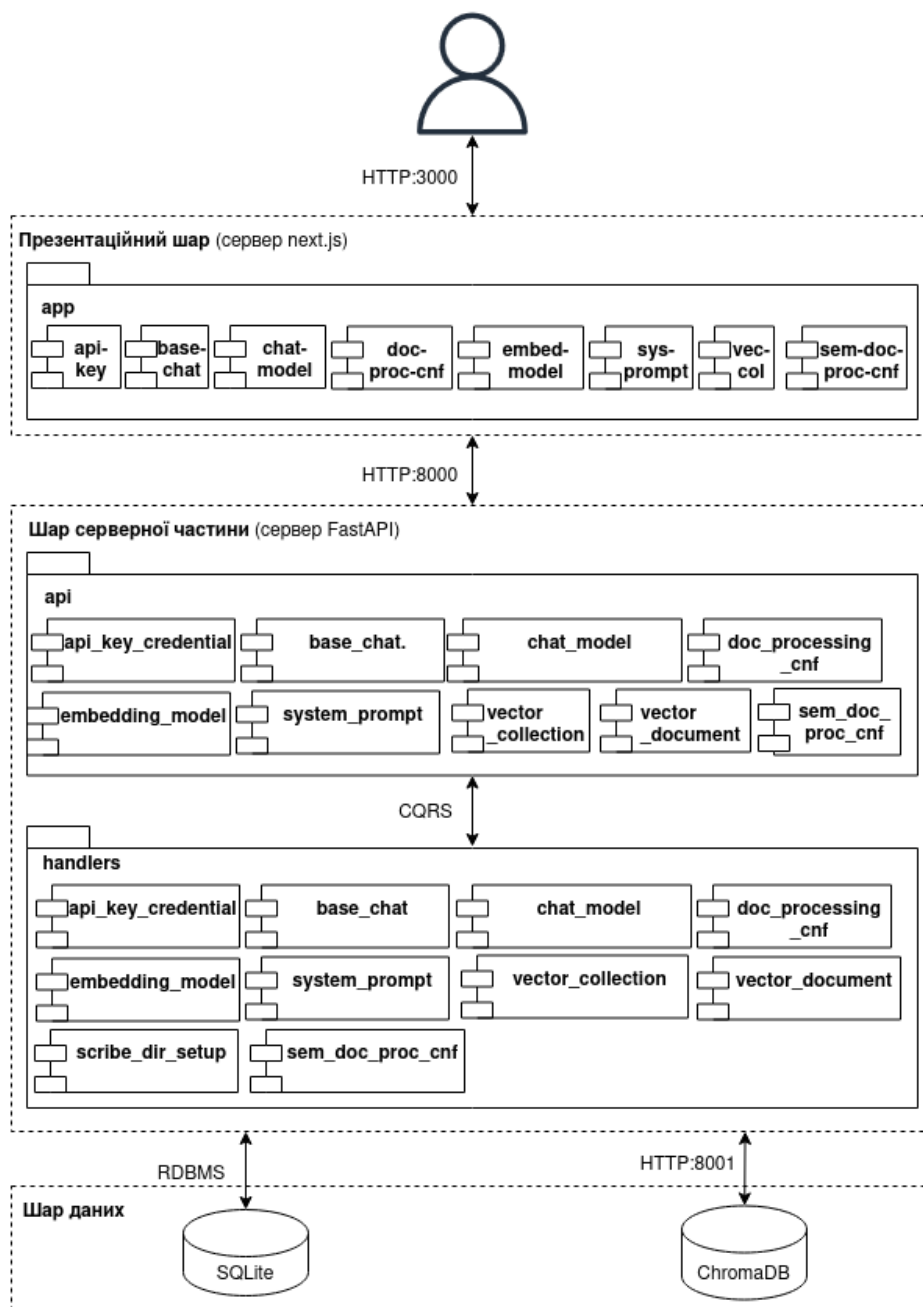


Рисунок 2.15 – Діаграма комунікацій вебресурсу

2.12 Алгоритмічне уявлення

2.12.1 Україномовна ембедінг-модель.

Для підтримки української мови у вебресурсу була натренована україномовна ембедінг-модель xlm-roberta-ua-distilled [18].

Для навчання моделі було використано підхід запропонований у наступній науковій роботі [19].

Ідея підходу полягає у дистанції знань з моделі вчителя до моделі учня, де функцією втрат виступає середньоквадратична помилка (MSE).

MSE обчислюється між вектором речення, створеним вчительською моделлю (наприклад, англійською мовою), і вектором того ж речення, згенерованим моделлю студентом, як для англійської версії, так і для перекладу цього ж речення іншими мовами. У процесі тренування речення було перекладено на українську мову (рис. 2.16).

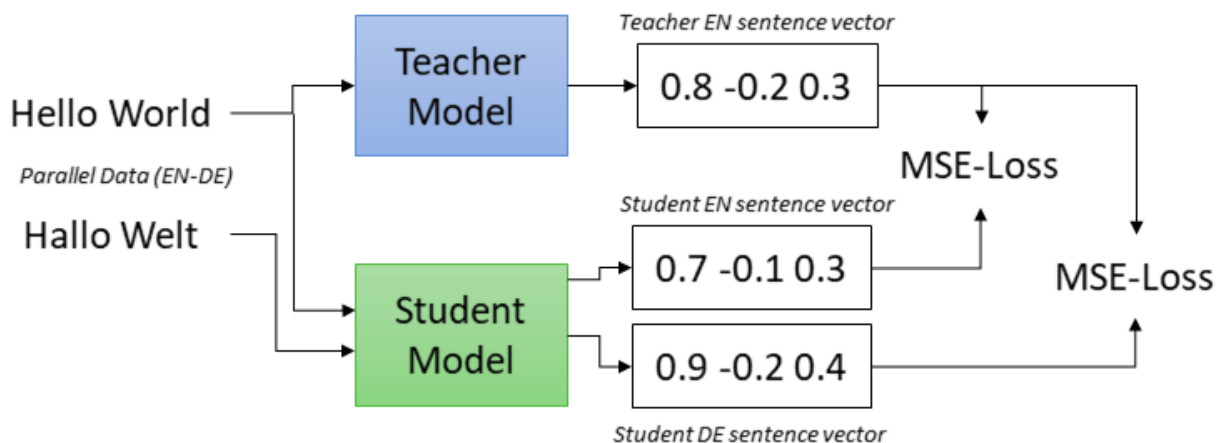


Рисунок 2.16 – Процес мультлінгвістичного тренування через дистиляцію знань моделі вчителя (джерело: [19])

Таким чином, запропонований підхід не лише дистилює знання моделі вчителя до учня, але й «стискає» вектори тих самих речень на різних мовах, роблячи їх ближчими одне до одного, що є логічним, адже семантично однакові речення повинні мати схожі векторні представлення незалежно від мови.

У результаті модель демонструє покращену продуктивність у кількох мовах навчання, а також кращий ефект крос-лінгвістичного перенесення.

В якості моделі вчителя була обрана модель multi-qa-mpnet-base-dot-v1 [20]. Це монолінгвістична, англійська модель, яка демонструє відмінну продуктивність у задачах семантичного пошуку, що добре підходить для систем типу RAG, згідно з бенчмарками.

В якості моделі учня була обрана модель XLM-RoBERTa. Це багатомовна версія моделі RoBERTa, навчена на даних CommonCrawl, що охоплюють 100 мов.

Навчання проводилось на наступних наборах даних паралельних речень (англійська і українська):

- 1) parallel-sentences-talks;
- 2) parallel-sentences-wikimatrix;
- 3) parallel-sentences-tatoeba.

Об'єднаний навчальний набір даних містив понад 500 000 пар речень.

Навчання тривало 4 епохи з розміром батчу 48. Для тренування використовувалася відеокарта GPU P100 із 16 Гбайт пам'яті, надана платформою Kaggle. На цій GPU тренування тривало понад 8 годин.

Для оцінки якості натренованої моделі використовувався набір даних sts17-crosslingual-sts. Він складається з багатомовних пар речень та оцінок схожості від 0 до 5, які були виставлені людьми. Однак у цьому наборі даних відсутні пари речень для української мови, тому вони були машинно-перекладені за допомогою моделі від OpenAI GPT-4o. У результаті було сформовано три підмножини для оцінки:

- 1) en-en (англійська-англійська);
- 2) en-ua (англійська-українська);
- 3) ua-ua (українська-українська).

В табл. 2.2 наведено результати оцінки моделей за бенчмарком sts-17. Числові значення, що було отримано є кореляцію Спірмена між оцінкою людини і

оцінкою моделі. Розроблена модель досягає найліпших результатів серед інших в наборах даних en-ua і ua-ua.

Таблиця 2.2 – Оцінка якості моделей

Модель	Набір даних		
	en-en	en-ua	ua-ua
multi-qa-mpnet-base-dot-v1 (вчитель)	75,8	12,9	62,3
XLM-RoBERTa (базова)	52,2	13,5	41,5
xlm-roberta-ua-distilled (студент)	73,1	62,0	64,5

2.12.2 Метод семантичного поділу тексту.

Розроблений вебресурс реалізує декілька способів поділу тексту на частини. Крім поділу за допомогою бібліотеки Unstructured, вебресурс також використовує розроблений семантичний метод поділу тексту на фрагменти на основі динамічного ковзного вікна [21].

Метод складається з алгоритму поділу і алгоритму підбору порогового значення за допомогою алгоритму на основі бінарного пошуку.

Перед початком роботи алгоритму поділу, текст поділяється на речення, після цього встановлюються два покажчики l – на перше речення і r – на друге. Далі, вікно $W[l; r]$ поступово розширюється за рахунок переміщення покажчика r . Речення S_l і вікно $W[l; r]$ перетворюються на вектори за допомогою ембедінг-моделі E . Між утвореними векторами обчислюється косинусна подібність. Якщо значення подібності менше за заданий поріг t , або розмір вікна $W[l; r]$ перебільшує значення максимального розміру документа – створюється документ $W[l, r)$. В іншому ж випадку покажчик r переміщується вперед.

Таким чином, часова складність семантичного алгоритму поділу тексту є:

$$O(n), \quad (2.1)$$

де n – кількість речень для обробки [1].

Алгоритм підбору порогового значення на основі бінарного пошуку теж використовує текст поділений на речення як і алгоритм семантичного ділення.

При його роботі речення об'єднується у фрагменти фіксованого розміру m . Кожен отриманий фрагмент C_i і його перше речення C_1 на вектори за допомогою ембеддінг-моделі E . Далі обчислюється косинусна подібність між цими двома векторами. Після цього формується словник D , де ключ – це косинусна подібність між векторами, а значення – відповідний фрагмент. Отриманий словник сортується за зростанням за значеннями ключа.

Після сортування запускається процес бінарного пошуку, з оцінкою від людини. Користувач оцінює сформовані фрагменти через командний рядок – вказуючи, чи є фрагмент семантично повним. Якщо фрагмент видається логічним і цілісним – користувач зменшує поріг; якщо ні – збільшує.

Такий процес продовжується, поки не буде знайдено оптимальне порогове значення, яке повертається з точністю до двох знаків після коми. Це значення вважається фінальним порогом t .

Таким чином, часова складність алгоритму підбору порогового значення на основі бінарного пошуку є:

$$O(\log(k)), \quad (2.2)$$

де k – кількість елементів в отриманому словнику [1].

Запропонований метод у порівнянні з іншими передовими методами семантичного поділу тексту забезпечує такі прирости по метриках оціночного фреймворку залежно від параметра максимального розміру документа: Intersect over Union від 0,2% до 2,8%, точність від 0,4% до 3,1%, омега точність від 1,4% до 14,8% [1].

2.13 Забезпечення безпеки вебресурсу

Вебресурс є системою з можливістю власного розгортання, що виключає більшість ризиків, характерних для вебдодатків, таких як атаки на сервер чи витоки через мережу. Для забезпечення безпеки чутливих даних, зокрема API-ключів провайдерів, використовується криптографічний алгоритм Fernet, реалізований у модулі cryptography для Python [22].

API-ключі шифруються перед збереженням у базі даних, що унеможливорює їх використання у разі компрометації бази.

Fernet – це симетричний алгоритм шифрування, що базується на AES-128 у режимі CBC з випадковим ініціалізаційним вектором (IV) та забезпечує цілісність даних за допомогою HMAC-SHA256. Ключ алгоритму має розмір 256 біт (32 байти). Формат зашифрованих даних включає версію протоколу, часову мітку, IV, зашифроване повідомлення та HMAC. Для розшифрування використовується той самий ключ, який застосовувався під час шифрування, що робить його критично важливим для безпеки системи [23].

У вебресурсі ключ Fernet зберігається у файлі `scribe.key`, розташованому в директорії `.scribe` у домашньому каталозі користувача: `~/.scribe/scribe.key`.

2.14 Опис стеку технологій

2.14.1 Серверна частина.

Обрана мова програмування для серверної частини: Python – мова програмування загального призначення, популярна у сфері аналізу даних, веброзробки та автоматизації.

Фреймворки:

- 1) FastAPI – фреймворк для створення API на Python із підтримкою асинхронності та OpenAPI;
- 2) LangChain – фреймворк для роботи з LLM;
- 3) SQLAlchemy – ORM-бібліотека для роботи з базами даних, що дозволяє взаємодіяти з SQL за допомогою Python.

Бібліотеки:

- 1) ChromaDB – бібліотека для управління векторними базами даних у пам'яті;
- 2) Unstructured – бібліотека для обробки неструктурованих даних;
- 3) dependency-injector – бібліотека для впровадження залежностей у Python-додатках;
- 1) mediatr – бібліотека для реалізації патерну медіатор у Python;
- 2) faker – бібліотека для генерації фейкових даних для тестування;
- 3) pytest – бібліотека для модульного тестування коду;
- 4) cryptography – бібліотека для роботи з криптографічними операціями.

Обраний пакетний менеджер для серверної частини: poetry – пакетний менеджер для Python, що полегшує управління залежностями та створення проєктів.

2.14.2 Клієнтська частина.

Обрана мова програмування для клієнтської частини: TypeScript – статично типізована мова програмування, яка компілюється в JavaScript.

Фреймворки:

- 1) Next.js – React-фреймворк для створення серверних рендерів та статичних вебзастосунків;
- 2) React – бібліотека JavaScript для створення користувацьких інтерфейсів.

Бібліотеки:

- 1) Zod – TypeScript-бібліотека для декларативної перевірки та парсингу даних;
- 2) MUI – набір компонентів React для створення сучасних користувацьких інтерфейсів.

Обраний пакетний менеджер для клієнтської частини: npm – менеджер пакетів для JavaScript, який є частиною Node.js.

Висновки до розділу 2

У другому розділі дипломної роботи детально спроектовано вебресурс з використанням Retrieval-Augmented Generation з підтримкою української мови, призначену для інтеграції мовних моделей із зовнішніми джерелами даних. Основна функціональність системи полягає у формуванні запитів, витягуванні релевантного контексту з документів, та генерації відповідей за допомогою мовної моделі.

Архітектура вебресурсу з використанням Retrieval-Augmented Generation з підтримкою української мови реалізує сучасні патерни такі як: CQRS, архітектура портів та адаптерів, і має розділення на клієнтську та серверні частини і рівень збереження даних.

Функціональні вимоги системи охоплюють повний сценарій користування вебресурсом: від додавання API ключів та мовних моделей до побудови чату та взаємодії з ним. Користувач має змогу налаштовувати семантичну обробку документів, параметри моделей і векторної колекції. Нефункціональні вимоги передбачають підтримку різних браузерів, Unix-систем, захист конфіденційних даних через симетричне шифрування Fernet, та ведення логування.

Окрема увага приділена алгоритмам семантичного поділу тексту та тренуванню україномовної ембедінг-моделі xlm-roberta-ua-distilled. Підхід до мультилінгвістичної дистиляції знань дозволив створити україномовну ембедінг-модель. Це підтверджено результатами оцінки за бенчмарком STS-17, де розроблена модель демонструє значне покращення якості для української мови.

Таким чином, запропоноване рішення є обґрунтованим, модульним, масштабованим і захищеним.

3 РЕАЛІЗАЦІЯ ВЕБРЕСУРСУ З ВИКОРИСТАННЯМ RETRIEVAL-AUGMENTED GENERATION З ПІДТРИМКОЮ УКРАЇНСЬКОЇ МОВИ

3.1 Уявлення про структуру проекту вебресурсу

3.1.1 Серверна частина.

Серверна частина має два кореневих пакета для зберігання коду: `src` і `tests`. Як зрозуміло з назви `tests` зберігає тести проекту, коли в `src` його основний код.

`src` містить в собі пакети: `adapters`, `api`, `domain`, `handlers`, `system`. Крім того, містить в собі і окремі модулі: `bootstrap.py`, `di_container.py`, `enums.py`. Розглянемо кожний з пакетів і модулів детально.

Почнемо з модулів. `di_container.py` містить в собі контейнер зі всіма визначеним залежностями вебресурсу інкапсульовані у класі `Container`, який наслідується від класу `DeclarativeContainer` з бібліотеки `dependency-injector`.

Модуль `bootstrap.py` містить в собі логіку для початку роботи вебресурсу. Тут надсилається команда про початок роботи до медіатора, конфігурується логування, відбувається мапінг `SQLAlchemy` моделей.

Модуль `enums.py` містить в собі глобальні на проєкт визначенні перелічення.

Тепер перейдемо до пакетів, почнемо з `adapters`. Саме пакет `adapters` є частиною архітектури портів і адаптерів. Для того, щоб не бути надмірними пакет `adapters` визначає порти-інтерфейси і одразу ж імплементації-адаптери в єдиних модулях. Крім портів, в цьому модулі також визначенні і різноманітні обгортки. Пакет `adapters` містить наступні модулі:

- 1) `async_vector_client.py` – порт і адаптер асинхронного векторного клієнта;
- 2) `chat_model` – порт і адаптер мовної моделі;
- 3) `chroma_models` – обгортки над об'єктам з `ChromaDB`;
- 4) `codecs` – порти і адаптери кодеків для шифрування і дешифрування;
- 5) `orm_models` – мапінг моделей `SQLAlchemy` до доменних моделей;
- 6) `repository` – порти і адаптери репозиторієв;
- 7) `uow` – порти і адаптери `unit of work`;

8) `vector_collection_repositroy` – порти і адаптери векторної колекції.

Далі перейдемо до пакету `api`. Він складається з модуля `app.py`, що містить в собі конфігурації вебресурса і пакету `routers`, що містить в собі визначенні ендпоінти і має наступний зміст: `api_key_credential.py`, `base_chat.py`, `chat_model.py`, `doc_processing_cnf.py`, `sem_doc_proc_cnf.py`, `embedding_model.py`, `system_prompt.py`, `vector_collection.py`, `vector_documnet.py`.

Тепер перейдемо до пакету `domain`, що містить бізнес-логіку і моделі вебресурса. Він складається з модуля `models.py`, що містить моделі, і пакета `services`, що відповідає за бізнес-логіку і має наступний зміст:

1) `chat_model_builder.py` – відповідає за створення об'єкту мовної моделі і підказки;

2) `embedding_model_builder.py` – відповідає за створення ембеддінг-моделі;

3) `load_document_service.py` – відповідає за логіку завантаження і обробки документів;

4) `__init__.py` – містить в собі клас `EncodeApiKeyCredentialService`, що відповідає за шифрування `api` ключів користувача.

Далі перейдемо до `handlers`. Цей пакет в більшості містить в собі класи-обробники медіатора, що відповідають за обробку команд надісланих з ендпоінтів. Через це назви файлів ідентичні з назвами ендпоінтів. Виключенням є модуль `scribe_dir_setup.py`, що містить в собі обробник, який відповідає за роботу з директорією `.scribe` на початку роботи вебресурса.

Пакет `handlers` містить в собі наступні модулі: `api_key_credential.py`, `base_chat.py`, `chat_model.py`, `doc_processing_cnf.py`, `sem_doc_proc_cnf`, `embedding_model.py`, `system_prompt.py`, `vector_collection.py`, `vector_documnet.py`, `scribe_dir_setup.py`.

Пакет `system` містить в собі функції, що відповідають за роботу з файлами на ОС користувача і має наступний зміст:

1) `dir.py` – відповідає за створення директорії `.scribe` на системі користувача, директорії логування, ключа шифрування, тощо;

2) `logging.py` – відповідає за читання файлу конфігурацій логування;

3) `utils.py` – додаткові функції.

На рис. 3.1 наведена діаграма структури серверної частини вебресурсу.

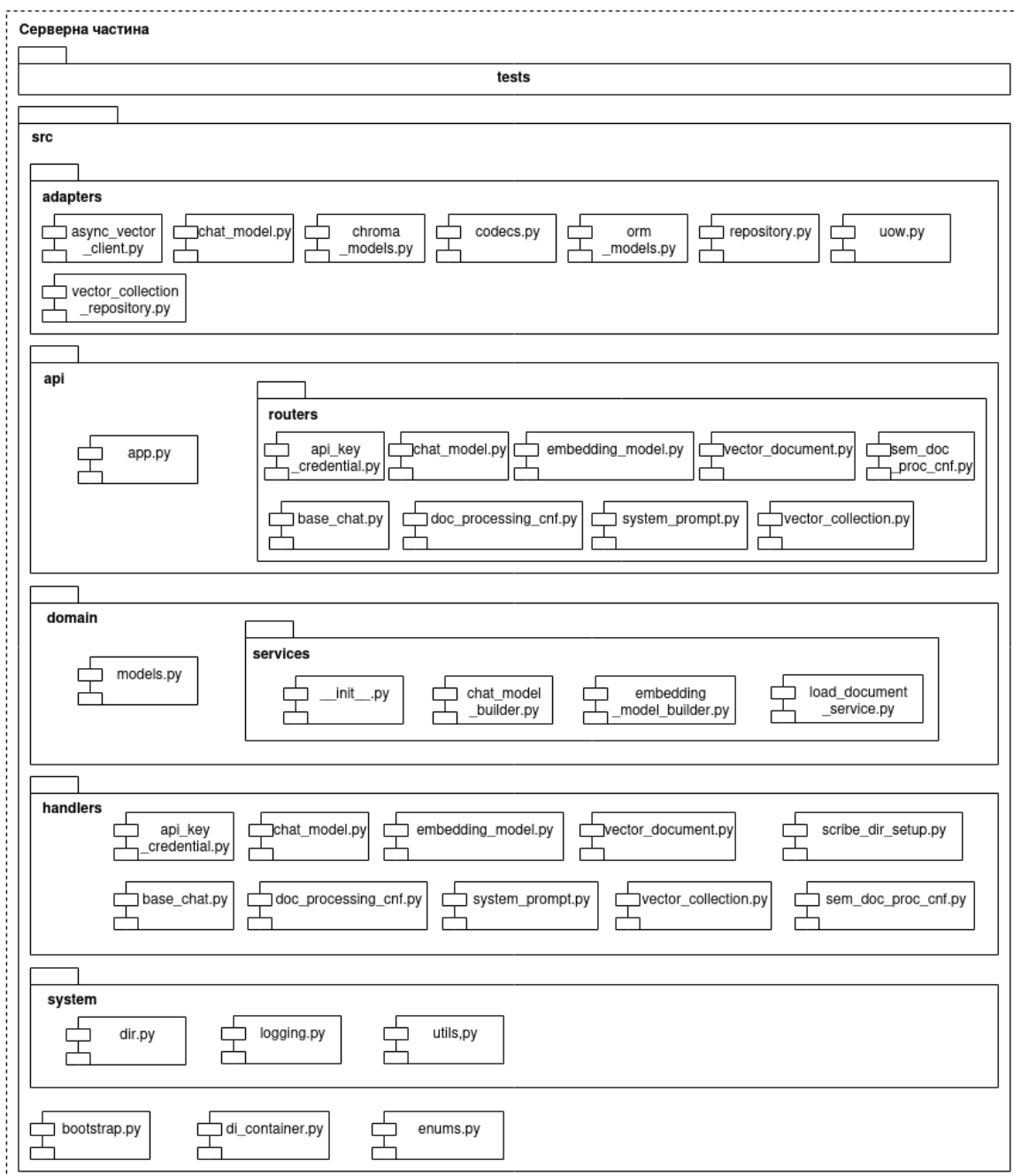


Рисунок 3.1 – Діаграма структури серверної частини вебресурсу

3.1.2 Клієнтська частина.

Передусім структура клієнтської частини вебресурсу визначена обмеженням фреймворку Next.js, а саме концепцією маршрутизації на основі директорії `app`, де

кожна директорія відповідає маршруту, а файл `page.js` у директорії відповідає рендерингу цього маршруту, що забезпечує модульність та простоту в організації.

Кореневий пакет має назву `src` і містить в собі пакет `app` і модулі:

- 1) `constants.ts` – для збереження константних значень;
- 2) `utils.ts` – допоміжні функції.

Далі розглянемо більш детально пакет `app`. Він містить в собі:

- 1) `layout.tsx` – визначає кореневий шаблон UI для всього вебресурса;
- 2) `page.tsx` – визначає вигляд кореневої сторінки;
- 3) `loading.tsx` – визначає вигляд сторінки при завантаженні;
- 4) `api-key` – пакет, що визначає вигляд сторінки API ключів;
- 5) `base-chat` – пакет, що визначає вигляд сторінки мовної моделі;
- 6) `doc-proc-cnf` – пакет, що визначає вигляд сторінки конфігурації обробки документів;
- 7) `sem-doc-proc-cnf` – пакет, що визначає вигляд сторінки конфігурації семантичної обробки документів;
- 8) `embed-model` – пакет, що визначає вигляд сторінки ембедінг-моделей;
- 9) `sys-prompt` – пакет, що визначає вигляд сторінки системних підказок;
- 10) `vec-col` – пакет, що визначає вигляд сторінки векторних колекцій.

На рис. 3.2 представлена діаграма структури клієнтської частини вебресурсу.

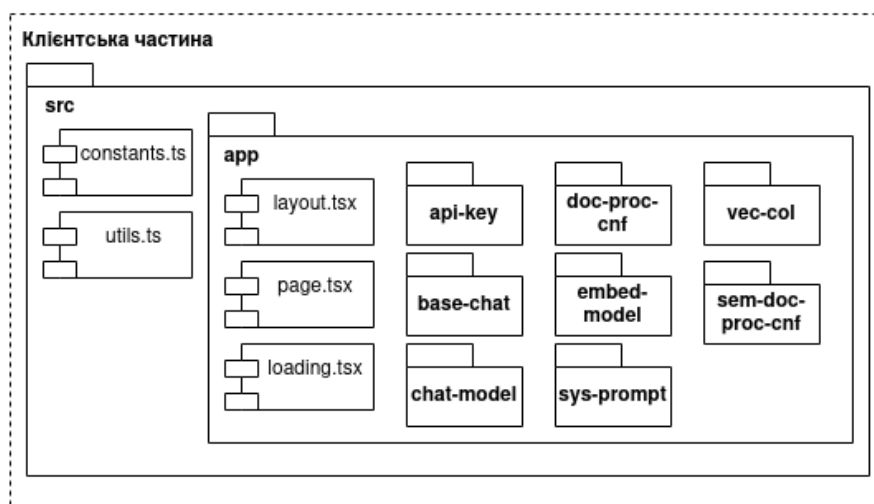


Рисунок 3.2 – Діаграма структури клієнтської частини вебресурсу

3.2 Уявлення про класи вебресурсу

В цьому розділі буде розглянуто основний модуль вебресурсу з пакету `adapters`, що відповідає за роботу з векторною колекцією, а саме `vector_collection_repository.py`. На рис. 3.3 представлена діаграма класів цього модулю і відношень між ними.

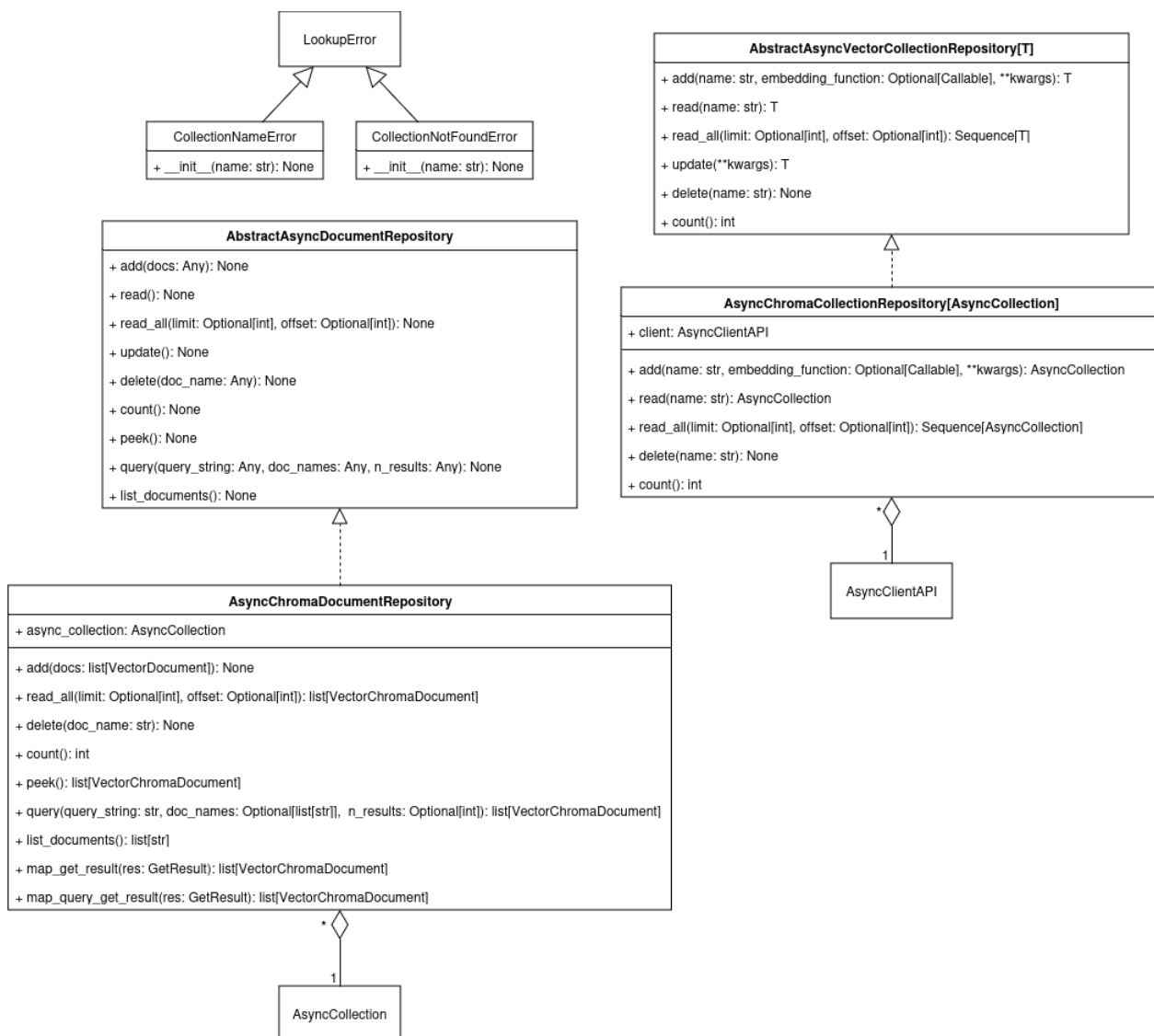


Рисунок 3.3 – Діаграма класів модулю `vector_collection_repository.py`

Розглянемо діаграму з рис. 3.3 більш детально. Діаграма класів представляє кілька асинхронних репозиторіїв для роботи з колекціями документів та векторів.

Класи помилок, такі як `CollectionNameError` та `CollectionNotFoundError`, наслідуються від `LookupError` і визначають помилки, пов'язані з іменем колекції: коли колекція вже існує або ж не була знайдена.

Абстрактні класи, такі як `AbstractAsyncVectorCollectionRepository[T]` та `AbstractAsyncDocumentRepository`, визначають інтерфейс для роботи з векторними колекціями та документами відповідно. `AbstractAsyncVectorCollectionRepository` є generic класом, що дозволяє використовувати різні типи даних під час роботи з колекціями. Ці класи містять оголошення асинхронних методів для додавання, читання, видалення та підрахунку елементів.

Репозиторій `AsyncChromaVectorCollectionRepository` реалізує `AbstractAsyncVectorCollectionRepository[AsyncCollection]` і використовує `AsyncClientAPI` для управління асинхронними колекціями, включаючи створення, читання та видалення колекцій.

`AsyncChromaDocumentRepository` реалізує `AbstractAsyncDocumentRepository` і використовує `AsyncCollection` для асинхронного управління документами, включаючи додавання, читання та видалення документів. Обидва репозиторії мають методи для роботи з базою даних: `add`, `read`, `read_all`, `update`, `delete`, `count`, які мають асинхронне виконання і використовують різні параметри, такі як `name`, `embedding_function`, `limit`, `offset` тощо. `AsyncChromaDocumentRepository` додатково має методи `query`, `list_documents`, `peek`, які виконують пошук документів за запитом та перелік існуючих документів.

`AsyncChromaVectorCollectionRepository` має агрегативний зв'язок з класом `AsyncClientAPI`. `AsyncClientAPI` є класом, який використовується для асинхронної взаємодії з `ChromaDB`.

`AsyncChromaDocumentRepository` має агрегативний зв'язок з `AsyncCollection`. Взаємодія і робота з документами відбувається для отриманою через цей зв'язок асинхронною колекцією.

Перейдемо до більш детального розгляду класів.

Клас `AsyncChromaVectorCollection repository` відповідає за створення, видалення і читання векторної колекції.

`AsyncChromaVectorCollection` містить наступні методи:

- 1) метод `__init__` (публічний): конструктор класу, приймає клієнт `client` типу `AsyncClientAPI` і зберігає його як внутрішній атрибут, нічого не повертає;
- 2) метод `add` (публічний): асинхронно створює нову колекцію за назвою `name` (тип `str`) та опціональною функцією ембеддингу `embedding_function` (тип `Optional[Callable]`), приймає додаткові аргументи через `kwargs`, повертає створену колекцію типу `AsyncCollection`;
- 3) метод `read` (публічний): асинхронно отримує колекцію за назвою `name` (тип `str`), повертає об'єкт типу `AsyncCollection`;
- 4) метод `read_all` (публічний): асинхронно отримує всі доступні колекції з обмеженням `limit` та зміщенням `offset` (типи `Optional[int]`), повертає список колекцій типу `Sequence[AsyncCollection]`;
- 5) метод `delete` (публічний): асинхронно видаляє колекцію за назвою `name` (тип `str`); нічого не повертає;
- 6) метод `count` (публічний): асинхронно рахує кількість колекцій, повертає кількість типу `int`;

Клас `AsyncChromaDocumentRepository`, відповідає за роботу з документами векторної колекції.

`AsyncChromaDocumentRepository` містить наступні методи:

- 1) метод `__init__` (публічний): конструктор класу, приймає `async_collection` типу `AsyncCollection` і зберігає його як внутрішній атрибут, нічого не повертає;
- 2) метод `add` (публічний): асинхронно додає список документів `docs` (типу `list[VectorDocument]`) до колекції `async_collection`, нічого не повертає;
- 3) метод `read_all` (публічний): асинхронно отримує всі документи з колекції з обмеженням `limit` та зміщенням `offset` (типи `int` або `None`), повертає список документів типу `list[VectorChromaDocument]`;
- 4) метод `delete` (публічний): асинхронно видаляє документи з колекції за ім'ям `doc_name` (тип `str`), нічого не повертає;
- 5) метод `count` (публічний): асинхронно рахує кількість документів у колекції, повертає кількість документів типу `int`;

6) метод `peek` (публічний): асинхронно отримує кілька, а саме 3 документи, з колекції для перегляду, повертає список документів типу `list[VectorChromaDocument]`;

7) метод `query` (публічний): асинхронно виконує пошук у колекції за рядком `query_string` (тип `str`), списком імен документів `doc_names` (тип `Optional[list[str]]`) і кількістю результатів `n_results` (тип `Optional[int]`), повертає список документів типу `list[VectorChromaDocument]`;

8) метод `list_documents` (публічний): асинхронно отримує всі унікальні імена документів з колекції, повертає список імен документів типу `list[str]`;

9) метод `map_get_result` (публічний, статичний): перетворює результат типу `GetResult` у список документів типу `list[VectorChromaDocument]`, приймає об'єкт `res` і повертає список;

10) метод `map_query_get_result` (публічний, статичний): перетворює результат пошуку типу `GetResult` у список документів типу `list[VectorChromaDocument]`, приймає об'єкт `res` і повертає список;

3.3 Операційне уявлення вебресурсу

Для моніторингу серверної частини вебресурсу був використаний наступний стек:

1) `prometheus-fastapi-instrumentator` – інструмент для інтеграції моніторингу та збору метрик з FastAPI за допомогою Prometheus;

2) Prometheus – система моніторингу і збору метрик, що дозволяє відслідковувати та аналізувати продуктивність додатків;

3) Grafana – платформа для візуалізації даних, що дозволяє створювати інтерактивні панелі моніторингу для метрик.

Всі компоненти стеку взаємодіють один з одним. Prometheus відповідає за збір і зберігання метрик, таких як навантаження на сервер, час виконання запитів та інші показники продуктивності додатку. Для інтеграції з FastAPI використовується `prometheus-fastapi-instrumentator`, який автоматично збирає

метрики з API, відстежуючи різні аспекти роботи вебресурсу, включаючи запити і відповіді. Grafana дозволяє візуалізувати ці метрики на інтерактивних панелях, що дає можливість зручно моніторити стан системи, аналізувати дані в реальному часі та створювати кастомізовані дашборди для різних користувачів і сценаріїв.

Як панель керування для відображення зібраних даних, було використано готовий `fastapi-dashboard.json` з репозиторію `fastapi-prometheus-grafana`. Загалом він має наступний вигляд (рис. 3.4).

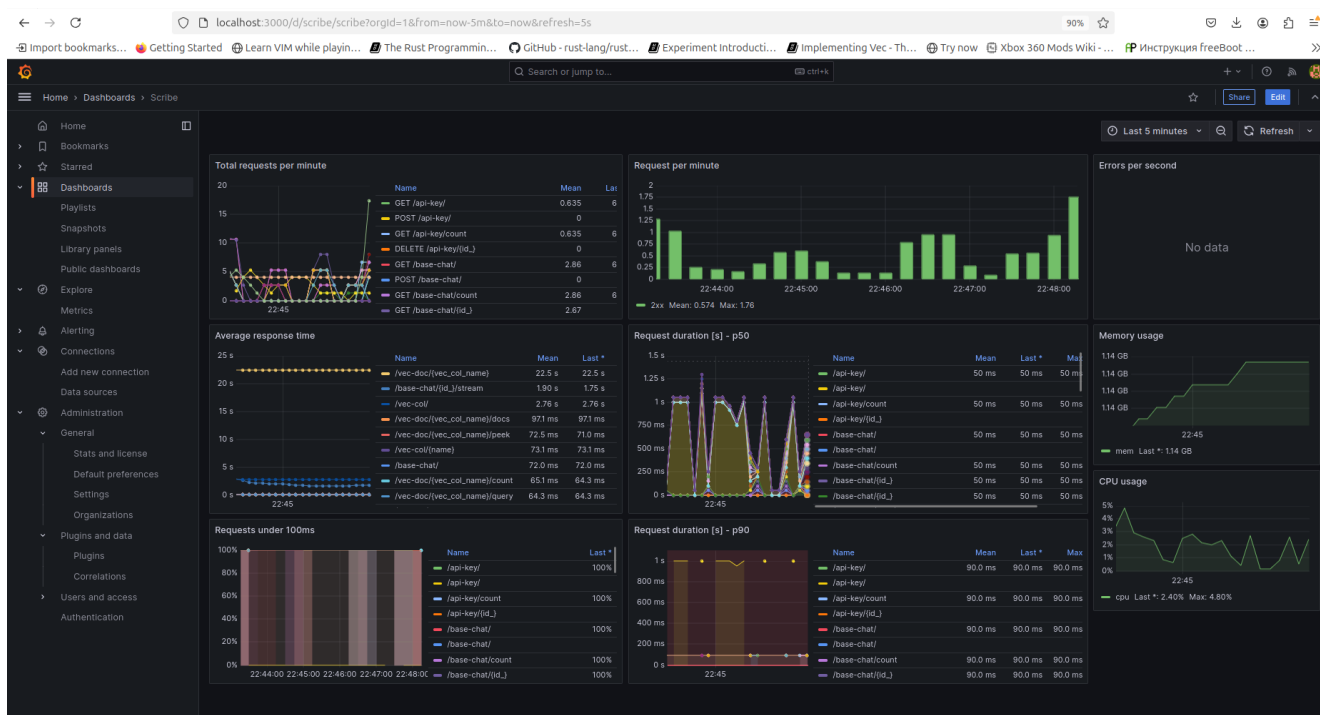


Рисунок 3.4 – Вигляд панелі керування в Grafana

Перейдемо до розгляду телеметричних параметрів сервера після типового сценарію використання. Всі параметри було отримано після активної роботи з системою протягом 5 хвилин. Як мовна модель була обрана `gpt-4o-mini`, як модель ембеддінгів `all-MiniLM-L6-v2` (локальна). Для тесту була проведена робота з доволі об'ємними статтями на `wikipedia`, а саме: `Assembly language`, `RollerCoaster Tycoon`, `C (programming language)`, тощо.

Почнемо з параметру «Average response time». Найбільший середній показник у значенні 22,5 секнуд має ендпоїнт `/vec-doc/`, що не є неочікуваним,

оскільки саме цей ендпоїнт відповідає за обробку документів. Найменше ж має ендпоїнт `/api-key/count` у значенні 4,11 мілісекунд, що просто повертає кількість записів у таблиці (рис. 3.5).

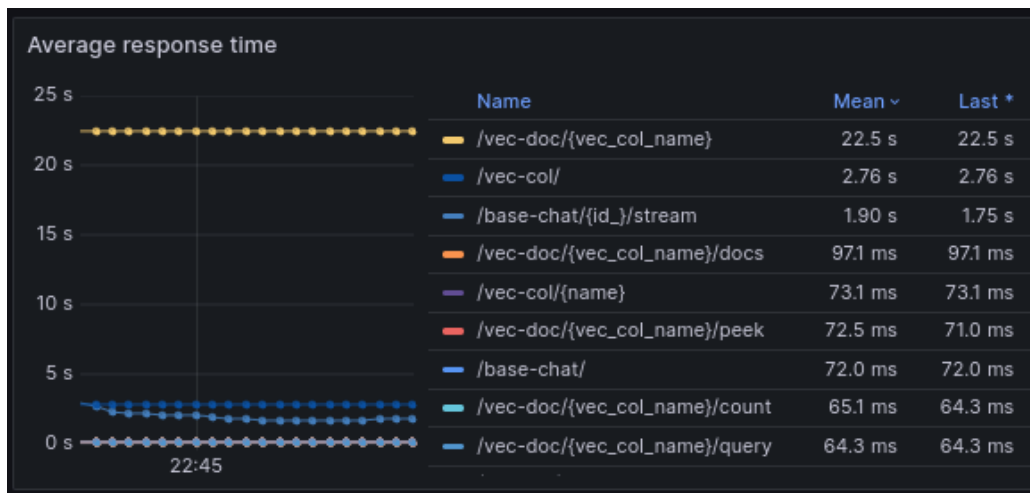


Рисунок 3.5 – Значення параметру «Average response time»

Далі перейдемо до «Requests under 100 ms». Цей параметр дозволяє виявити відсоток реквестів до ендпоїнтів, що були виконанні менше ніж за 100 мілісекунд (рис. 3.6).

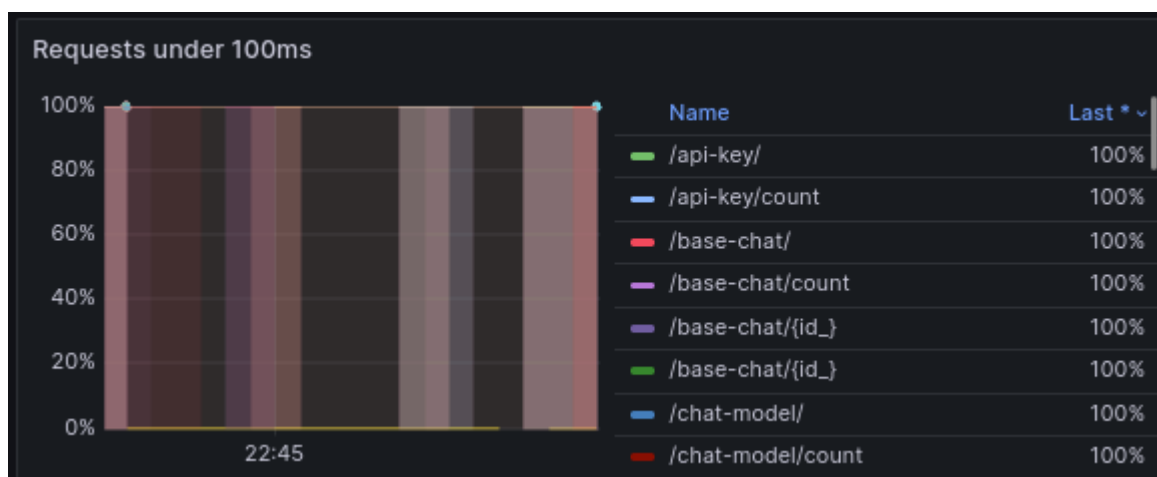


Рисунок 3.6 – Значення параметру «Requests under 100 ms»

Не менш важливим параметрами є використання оперативної пам'яті і ресурсів процесора. При використанні вебресурсу пам'ять змінюється в межах від

1,14 до 1,16 Гбайт в піку. Процесор ж має доволі великий розкид. Це передусім пов'язано з використанням локальної ембеддінг-моделі. Зазвичай сервер займає 0,133% процесорних ресурсів, але в піку обробки документа набуває значення до 243% (тобто задіяно більше двох ядер) (рис. 3.7).

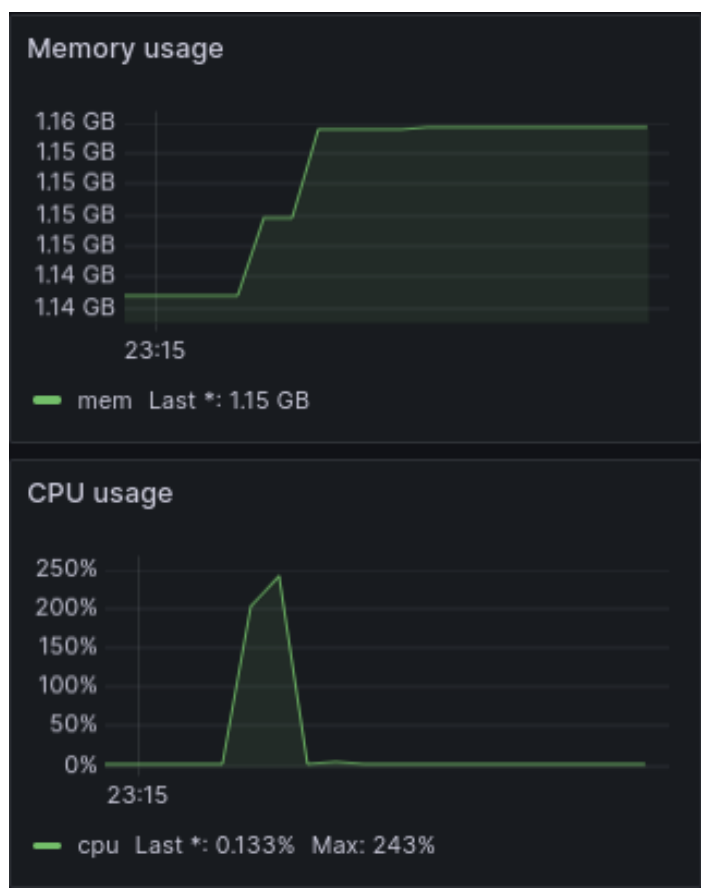


Рисунок 3.7 – Параметри використання оперативної пам'яті і процесорних ресурсів

3.4 Стратегія доставки продукту

Для продукту обрана Git-based distribution, характерна для open-source додатків. Цей підхід спрощує управління версіями, забезпечуючи інструменти для відстеження змін і повернення до попередніх версій, полегшує співпрацю між розробниками, дозволяє швидко інтегрувати й поширювати оновлення, а також забезпечує відкритий доступ до репозиторіїв для встановлення продукту. На рис. 3.8 зображено вигляд сторінки публічного репозиторію проєкту.

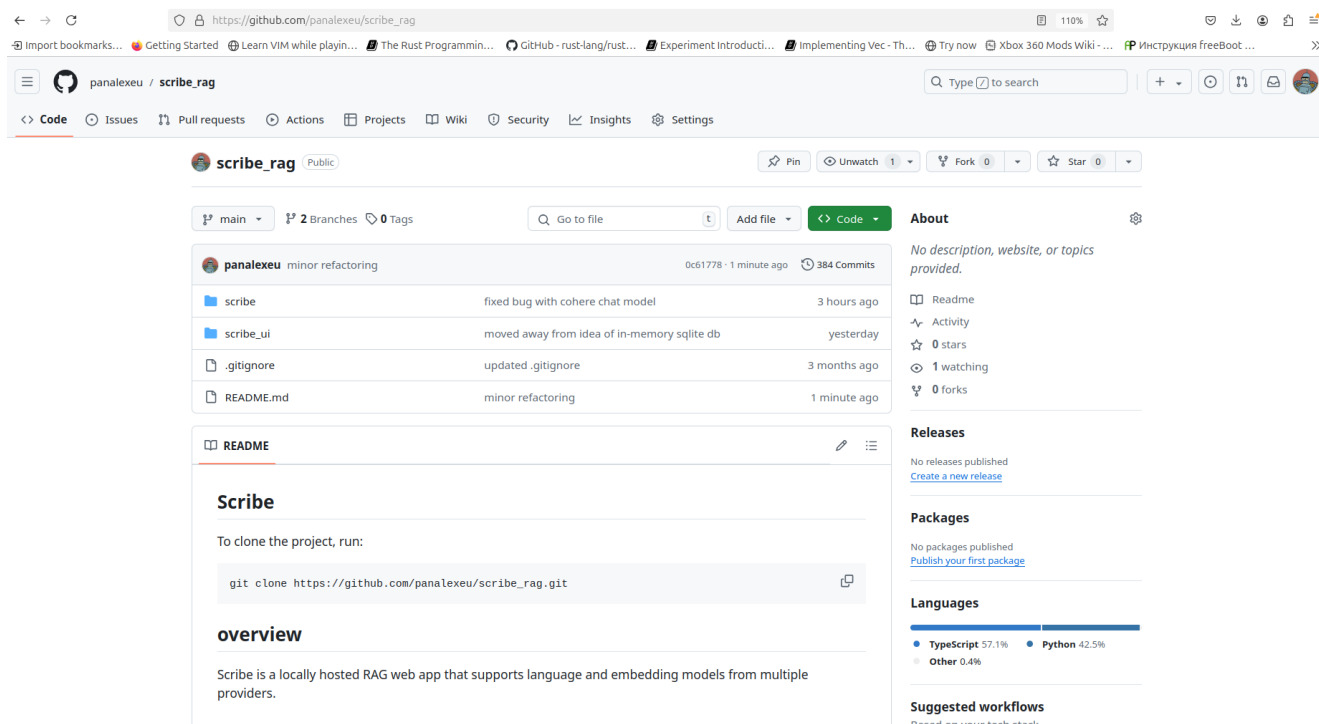


Рисунок 3.8 – Вигляд сторінки публічного репозиторія проєкта

3.5 Управління якістю програмного коду вебресурсу

В табл. 3.1 представлено параметри репозиторію проєкту на момент останнього коміту.

Як система контролю версій було обрано Git, а для хостування віддаленого репозиторію – GitHub. Була використана техніка мікрокомітів, що зрозуміло з їх кількості. Одна з бранчів (dev) була виключно для розробки, працюючі результати з якої зливались до гілки main.

Таблиця 3.1 – Метрики репозиторію проєкта

Метрика	Значення
Кількість комітів у гілці main	464
Загальна кількість гілок	2
Кількість зачинених PR	28

3.6 Розрахунок метрик програмного коду вебресурсу

В табл. 3.2 представлені метрики коду для серверної частини вебресурсу. Метрики було отримано за допомогою рір пакету radon і JetBrains плагіну Statistic.

Таблиця 3.2 – Метрики програмного коду серверної частини вебресурсу

Метрика	Значення
Загальна кількість рядків коду у проєкті вебресурсу	6785
Середня кількість рядків коду в одному файлі	123
Максимальна кількість рядків коду в одному файлі	319
Середня кількість рядків коду в одному методі	9
Максимальна кількість рядків коду в одному методі	37
Максимальна глибина дерева успадкування	2
Середня цикломатична складність методу	1,74
Максимальна цикломатична складність методу	15
Коментованість коду	4%
Покриття коду модульними тестами	93%

Проект містить загальну кількість рядків коду 6785, середня кількість рядків у файлі складає 123, а максимальна – 319. Середня кількість рядків у методі становить 9, з максимальним значенням у 37 рядків. Глибина дерева успадкування не перевищує 2.

Середня цикломатична складність методів – 1,74, що є хорошим показником. Максимальна цикломатична складність одного методу становить 15, що характерно для методів-білдерів, що збирають різні об'єкти на підставі отриманих аргументів.

Коментованість коду складає лише 4%, що є позитивним, оскільки велика кількість коментарів може бути антипатерном – добре написаний код має бути достатньо зрозумілим без необхідності коментарів.

Проект має високе покриття модульними тестами – 93%, що свідчить про належну якість тестування.

3.7 Контрольний список за якістю реалізації вебресурсу

В табл. 3.3 представлено критерії якості, для оцінки вебресурса. Проєкт покриває майже кожний критерій, що є доцільним для специфіки вебресурса.

Таблиця 3.3 – Контрольний список якості вебресурсу

Твердження	Відповідь	Пояснення у разі негативної відповіді
Використання Dependency Injection	Так	-
Використання логування (logging)	Так	-
Використання модульного тестування	Так	-
Захист від ін'єкцій SQL	Ні	Не є актуальним для вебресурсу з можливістю власного розгортання
Захист від Javascript/XSS ін'єкцій	Ні	Не є актуальним для вебресурсу з можливістю власного розгортання
Використання валідації даних у всіх полях введення для користувача інтерфейсів	Так	-
Використання інсталяторів/інтернет магазинів	Ні	Проєкт поширюється через Git
Використання засобів синхронізації даних у разі багатопоточної програми	Так	-
Відсутність зашитих в програмний код конфігураційних параметрів програми.	Так	-
Чи застосовані UI паттерни під час розробки UI.	Так	-
Враховано coding style guide lines для вибраної мови програмування.	Так	-
Чи враховані рекомендовані guide lines при розробці користувача інтерфейсу для тієї чи іншої ОС.	Так	-
Чи підтримується локалізація та глобалізація вебресурсу	Так	-

3.8 Генеральний тест-план

При розробці генерального плану тестування було прийнято рішення дотримуватися автоматизованого тестування, де це тільки можливо. Фактично створені тести є цементом, який фіксує і пов'язує систему в певному стані, забезпечуючи стабільність і надійність протягом усього життєвого циклу програми.

Саму структуру пакетів автоматизованих тестів було організовано наступним чином:

1) unit – відповідає за перевірку окремих функцій або класів, включаючи тестування доменних моделей, сервісів і обробку директорій, таких як .scribe;

2) document – фактично є частиною unit тестів, але було винесено в окрему категорію через довгий час виконання, необхідний для обробки документів;

3) integr (інтеграційні) – відповідають за перевірку інтеграції вебресурсу з базою даних, включаючи тестування репозиторіїв, ORM-моделей, UoW і їх взаємодії з базою даних;

4) e2e (end-to-end) – забезпечують повне тестування інформаційної системи від моменту надходження запиту до ендпоїнту й взаємодії з базою даних, проходячи повністю по всій системі.

Другою частиною тестування інформаційної системи є функціональне тестування, яке виконується вручну через UI на підставі розроблених тест-кейсів. Це тестування фактично є розширенням e2e тестування серверної частини, забезпечуючи повне тестування коду від UI до бази даних, і допомагає знайти та усунути потенційні проблеми в інтерактивному інтерфейсі користувача.

3.9 Розробка тест-кейсів для функціонального тестування

Нижче наведено розроблені тест-кейси для функціонального тестування системи. Тест-кейси посилаються на визначенні функціональні вимоги у розділі 2.3.

Тест-кейс № 1. FR1.1.2. Перевірка наявності директорії logs – Директорія .scribe створена після запуску – Перевірити наявність директорії logs в .scribe – Директорія logs наявна в .scribe.

Тест-кейс № 2. FR1.1.3. Перевірка самоочищення директорії .scribe – Директорія .scribe містить додаткові файли – Запустити вебресурс – Всі файли окрім scribe.key та logs видалено.

Тест-кейс № 3. FR1.1.4. Перевірка відтворення scribe.key після видалення – Файл scribe.key видалено – Запустити вебресурс – Файл scribe.key відтворено в .scribe.

Тест-кейс № 4. FR1.2. Перевірка доступності клієнтської частини через веб-браузер – Вебресурс запущено – Відкрити веб-браузер і перейти за вказаним URL – Клієнтська частина доступна через веб-браузер.

Тест-кейс № 5. FR2.1. Перевірка валідації імені API ключа – Ім'я API ключ не введено – Підтвердити введення – З'являється повідомлення про некоректність імені API ключа.

Тест-кейс № 6. FR2.2. Перевірка шифрування API ключа – Введено API ключ – Додати ключ – Повернення зашифрованого API ключа у UI.

Тест-кейс № 7. FR2.4. Перевірка зміни імені API ключа – Існує доданий API ключ – Змінити ім'я API ключа – Ім'я API ключа успішно змінено.

Тест-кейс № 8. FR2.5. Перевірка видалення API ключа – Існує доданий API ключ – Видалити API ключ – API ключ успішно видалено.

Тест-кейс № 9. FR3.1. Перевірка введення системної підказки – Запущена клієнтська частина вебресурсу – Ввести назву і зміст системної підказки – Системну підказку успішно введено.

Тест-кейс № 10. FR3.2. Перевірка перенаправлення після додання системної підказки – Введено і підтверджено системну підказку – Натиснути "Зберегти" – Користувач перенаправлений на сторінку з усіма підказками.

Тест-кейс № 11. FR3.3. Перевірка зміни параметрів підказки – Створено системну підказку – Змінити параметри підказки – Параметри підказки успішно змінено.

Тест-кейс № 12. FR3.4. Перевірка видалення підказки – Створено підказку – Видалити підказку – Підказку успішно видалено.

Тест-кейс № 13. FR4.1.1. Перевірка застосування сценарію за замовчуванням для конфігурації обробки документів – Введено тільки назву конфігурації – Додати конфігурацію – Застосовано сценарій за замовчуванням.

Тест-кейс № 14. FR4.2. Перевірка введення додаткових параметрів конфігурації – Ввести стратегію розбиття та інших параметрів – Додати конфігурацію – Параметри успішно збережено.

Тест-кейс № 15. FR4.3. Перевірка перенаправлення після додання конфігурації – Введено і підтверджено конфігурування – Натиснути "Зберегти" – Користувач перенаправлений на сторінку з усіма конфігураціями.

Тест-кейс № 16. FR4.4. Перевірка зміни параметрів конфігурації – Створено конфігурацію – Змінити параметри конфігурації – Параметри успішно змінено.

Тест-кейс № 17. FR4.5. Перевірка видалення конфігурації – Створено конфігурацію – Видалити конфігурацію – Конфігурація успішно видалено.

Тест-кейс № 18. FR5.2. Перевірка перенаправлення після додання семантичної конфігурації – Введено конфігурацію – Натиснути «Зберегти» – Користувач перенаправлений на сторінку з усіма семантичними конфігураціями.

Тест-кейс № 19. FR6.2. Перевірка вибору ембеддінг-моделі – Наявний API ключ – Вибрати ембеддінг-модель зі списку – Модель обрана.

Тест-кейс № 20. FR6.4. Перевірка перенаправлення після додання ембеддінг-моделі – Вибрано ембеддінг-модель – Натиснути "Зберегти" – Користувач перенаправлений на сторінку з усіма ембеддінг-моделями.

Тест-кейс № 21. FR6.5. Перевірка зміни параметрів ембеддінг-моделі – Створено ембеддінг-модель – Змінити параметри моделі – Параметри успішно змінено.

Тест-кейс № 22. FR6.6. Перевірка видалення ембеддінг-моделі – Створено ембеддінг-модель – Видалити модель – Модель успішно видалено.

Тест-кейс № 23. FR7.2. Перевірка вибору мовної моделі – Наявний API ключ – Вибрати мовну модель зі списку – Модель обрана.

Тест-кейс № 24. FR7.3. Перевірка введення додаткових параметрів мовної моделі – Ввести базове посилання і інші параметри – Зберегти модель – Параметри успішно збережено.

Тест-кейс № 25. FR7.4. Перевірка перенаправлення після додання мовної моделі – Вибрано мовну модель – Натиснути "Зберегти" – Користувач перенаправлений на сторінку з усіма мовними моделями.

Тест-кейс № 26. FR7.5. Перевірка зміни параметрів мовної моделі – Створено мовну модель – Змінити параметри моделі – Параметри успішно змінено.

Тест-кейс № 27. FR7.6. Перевірка видалення мовної моделі – Створено мовну модель – Видалити модель – Модель успішно видалено.

Тест-кейс № 28. FR8.2. Перевірка введення функції виміру відстані для векторної колекції – Створена ембедінг-модель – Ввести функцію виміру відстані – Функцію виміру відстані успішно збережено.

Тест-кейс № 29. FR8.3. Перевірка перенаправлення після додання векторної колекції – Введено всі необхідні дані для колекції – Натиснути "Зберегти" – Користувач перенаправлений на сторінку з усіма векторними колекціями.

Тест-кейс № 30. FR8.4. Перевірка видалення векторної колекції – Створено векторну колекцію – Видалити колекцію – Колекція успішно видалено.

Тест-кейс № 31. FR9.4. Перевірка завантаження вебсторінки за посиланням – Створена конфігурація обробки – Завантажити вебсторінку за URL – Вебсторінка успішно завантажено.

Тест-кейс № 32. FR9.5. Перевірка одночасного завантаження декількох файлів – Створена конфігурація обробки – Завантажити декілька файлів – Файли успішно завантажено.

Тест-кейс № 33. FR9.7. Перевірка можливості огляду перших частин документів – Завантажити документи – Оглянути перші три частини – Інформація успішно відображено.

Тест-кейс № 34. FR9.8. Перевірка видалення завантажених документів – Завантажені документи – Видалити документи – Документи успішно видалено.

Тест-кейс № 35. FR10.1. Перевірка вибору кількості результатів пошуку – Відкрити дитячий майданчик колекції – Вибрати кількість результатів – Кількість результатів успішно вибрано.

Тест-кейс № 36. FR10.2. Перевірка отримання результатів за запитом – Ввести запит – Здійснити пошук – Отримані найближчі частини документів.

Тест-кейс № 37. FR10.2.2. Перевірка відображення відстані – Здійснено пошук частин документів – Перевірити наявність поля відстані – Відстань між частинами і запитом відображено.

Тест-кейс № 38. FR11.2. Перевірка обов'язковості введення назви та опису чату – Не введено назву або опис – Спробувати створити чат – З'являється повідомлення про необхідність введення даних.

Тест-кейс № 39. FR11.3. Перевірка вибору векторної колекції для чату – Створено векторну колекцію – Вибрати векторну колекцію для чату – Векторна колекція додана до чату.

Тест-кейс № 40. FR12.1. Перевірка спілкування без контексту – Створено базовий чат без векторної колекції – Надіслати повідомлення в чат – Модель відповідає без включення контекстуальних даних.

Тест-кейс № 41. FR12.2. Перевірка отримання відповіді у вигляді потоку – Створено базовий чат – Надіслати запит до чату – Відповідь моделі надходить у вигляді безперервного текстового потоку.

Тест-кейс № 42. FR12.3. Перевірка контекстуалізованих відповідей при наданні векторної колекції – Створено чат з доданою векторною колекцією – Надіслати повідомлення в чат – Модель надає контекстуалізовану відповідь на основі документів з колекції.

Тест-кейс № 43. FR12.3.1. Перевірка вибору кількості частин документів для контексту – Створено чат з векторною колекцією – Вибрати кількість частин документів для контексту – Кількість частин, що будуть використані, успішно вибрано.

Тест-кейс № 44. FR12.4. Перевірка отримання частин документів разом з відповіддю – Створено чат з використанням векторної колекції – Надіслати запит

в чат – Разом з відповіддю моделі отримано частини документів, що використовувалися для контексту.

На основі сформованих тест-кейсів було проведено функціональне тестування вебресурсу з використанням Retrieval-Augmented Generation з підтримкою української мови. Результати відображено у додатку В (табл. В.1).

3.10 Результати проведення автоматизованого тестування

Для автоматизованого тестування використовується фреймворк pytest. Для e2e тестів застосовується TestClient – фейковий вебсервер, який надається FastAPI. Інтеграційні та e2e тести виконуються за допомогою SQLite в пам'яті та фейкової версії ChromaDB. Під час тренування було мінімізовано використання моків, віддаючи перевагу фейковим імплементаціям інтерфейсів. Для генерації фейкових даних був використаний пакет faker.

Усі тести було написано у AAA (Arrange – Упорядкувати, Act – Діяти, Assert – Стверджувати) стилі. Як приклад у лістингу 3.1 наведено вигляд типової функції в такому стилі.

Лістинг 3.1 – Вигляд тестової функції вебресурсу у AAA-стилі

```
def test_encode_api_key_credential_service():
    # Arrange
    api_key_credential = ApiKeyCredential(«fake-api», «fake-key»)
    api_key_credential_copy = copy(api_key_credential)

    # Act
    # encoding the api key and setting encoding service
    codec = FakeCodec(«fake-key»)
    encode_service = EncodeApiKeyCredentialService(codec)
    encode_service.encode(api_key_credential)

    # Assert
    # only api key is modified by codec encoding
    assert api_key_credential.api_key != api_key_credential_copy.api_key
    assert api_key_credential.name == api_key_credential_copy.name
```

Перейдемо до результатів тестування по кожній групі тестів окремо. Результати unit, documents, інтеграційного і end-to-end тестування наведено на рис. 3.9–3.12 відповідно. Як видно з рисунків всі тести успішно пройдено.

```
tests/unit/test_chroma_models.py ... [ 8%]
tests/unit/test_codecs.py .. [ 11%]
tests/unit/test_dir_and_key_setup.py ..... [ 40%]
tests/unit/test_domain.py ..... [ 91%]
tests/unit/test_logging.py .. [ 97%]
tests/unit/test_utils.py . [100%]

===== 35 passed in 9.58s =====
(scribe-py3.12) oleksii@oleksii-ASUS-TUF-Dash-F15-FX516PC-FX516PC:~/big_projects
/scribe_rag/scribe$
```

Рисунок 3.9 – Результат unit тестування

```
tests/documents/test_load_document.py ... [100%]

===== 3 passed in 6.40s =====
(scribe-py3.12) oleksii@oleksii-ASUS-TUF-Dash-F15-FX516PC-FX516PC:~/big_projects
/scribe_rag/scribe$
```

Рисунок 3.10 – Результат documents тестування

```
tests/integrn/test_sqlalchemy.py ..... [100%]

===== 15 passed in 6.70s =====
(scribe-py3.12) oleksii@oleksii-ASUS-TUF-Dash-F15-FX516PC-FX516PC:~/big_projects
/scribe_rag/scribe$
```

Рисунок 3.11 – Результат інтеграційного тестування

```
tests/e2e/test_api_key_endpoints.py ..... [ 10%]
tests/e2e/test_api_root_endpoints.py . [ 12%]
tests/e2e/test_base_chat_endpoints.py ..... [ 23%]
tests/e2e/test_chat_model_endpoints.py ..... [ 34%]
tests/e2e/test_doc_processing_cnf_endpoints.py ..... [ 45%]
tests/e2e/test_embedding_model_endpoints.py ..... [ 56%]
tests/e2e/test_sem_doc_processing_cnf_endpoints.py ..... [ 67%]
tests/e2e/test_system_prompt_endpoints.py ..... [ 78%]
tests/e2e/test_vector_collection.py ..... [ 87%]
tests/e2e/test_vector_document.py ..... [100%]

===== 55 passed in 17.16s =====
(scribe-py3.12) oleksii@oleksii-ASUS-TUF-Dash-F15-FX516PC-FX516PC:~/big_projects
/scribe_rag/scribe$
```

Рисунок 3.12 – Результат end-to-end тестування

На рис. 3.13 наведено результати всіх типів тестування з репортом покриття. Як видно з наведених даних, покриття тестами становить 93%, що свідчить про високу якість проведеного тестування та ретельну перевірку функціональності системи.

```
----- coverage: platform linux, python 3.12.3-final-0 -----
```

Name	Stmts	Miss	Cover
src/__init__.py	0	0	100%
src/adapters/__init__.py	0	0	100%
src/adapters/async_vector_client.py	12	1	92%
src/adapters/chat_model.py	39	16	59%
src/adapters/chroma_models.py	20	0	100%
src/adapters/codecs.py	35	4	89%
src/adapters/orm_models.py	24	0	100%
src/adapters/repository.py	95	8	92%
src/adapters/uow.py	35	9	74%
src/adapters/vector_collection_repository.py	122	31	75%
src/api/__init__.py	0	0	100%
src/api/app.py	60	10	83%
src/api/routers/__init__.py	0	0	100%
src/api/routers/api_key_credential.py	49	0	100%
src/api/routers/base_chat.py	74	3	96%
src/api/routers/chat_model.py	71	0	100%
src/api/routers/doc_processing_cnf.py	66	0	100%
src/api/routers/embedding_model.py	57	0	100%
src/api/routers/sem_doc_proc_cnf.py	53	0	100%
src/api/routers/system_prompt.py	50	0	100%
src/api/routers/vector_collection.py	46	0	100%
src/api/routers/vector_document.py	60	1	98%
src/bootstrap.py	19	1	95%
src/di_container.py	62	4	94%
src/domain/__init__.py	0	0	100%
src/domain/models.py	100	7	93%
src/domain/services/__init__.py	8	0	100%
src/domain/services/chat_model_builder.py	31	6	81%
src/domain/services/chat_prompt_template_builder.py	14	0	100%
src/domain/services/embedding_model_builder.py	27	6	78%
src/domain/services/load_document_service.py	116	37	68%
src/enums.py	46	0	100%
src/handlers/__init__.py	0	0	100%
src/handlers/api_key_credential.py	84	0	100%
src/handlers/base_chat.py	125	25	80%
src/handlers/chat_model.py	86	0	100%
src/handlers/doc_processing_cnf.py	86	0	100%
src/handlers/embedding_model.py	82	0	100%
src/handlers/scribe_dir_setup.py	16	0	100%
src/handlers/sem_doc_proc_cnf.py	82	0	100%
src/handlers/system_prompt.py	81	0	100%
src/handlers/vector_collection.py	78	0	100%
src/handlers/vector_document.py	169	6	96%
src/system/__init__.py	0	0	100%
src/system/dir.py	51	0	100%
src/system/logging.py	18	0	100%
src/system/utils.py	7	0	100%
TOTAL	2356	175	93%

```
(scribe-py3.12) oleksii@oleksii-ASUS-TUF-Dash-F15-FX516PC-FX516PC:~/big_projects/scribe_rag/scribe$
```

Рисунок 3.13 – Покриття вебресурсу автоматизованими тестами

Висновки до розділу 3

У третьому розділі наведено детальний технічний опис реалізації інформаційної системи на основі архітектурних рішень, визначених у

попередньому етапі проектування. Реалізація охоплює як серверну, так і клієнтську частини, що побудовані відповідно до принципів модульності, масштабованості та підтримки принципів чистої архітектури (порти і адаптери, CQRS, dependency injection).

Серверна частина реалізована на мові Python і структурована у вигляді пакетів: `adapters`, `api`, `domain`, `handlers`, `system`. Усі компоненти розділені за відповідальністю, з ізоляцією бізнес-логіки, інтерфейсів доступу до даних та механізмів взаємодії із зовнішніми сервісами.

Окрему увагу приділено роботі з векторними колекціями: реалізовано відповідні репозиторії з підтримкою асинхронного доступу, типовими CRUD-операціями і пошуком документів за ембеддінг-запитами.

Операційна частина системи включає налаштований стек моніторингу (Prometheus, Grafana), що забезпечує збір та візуалізацію ключових метрик роботи API. Було перевірено роботу системи і проаналізовано результати за допомогою моніторингового стеку.

Система протестована як автоматизованими, так і ручними засобами. Було покрито тестування функціональних вимог через набір тест-кейсів, а також забезпечено unit, integration та end-to-end тестування. Автоматизоване тестування має покриття 93%.

Поширення вебресурсу реалізовано через Git-based розповсюдження, що забезпечує зручність встановлення, відкритість коду та підтримку спільноти. Супровідна документація, включаючи інструкції користувача, дозволяє самостійно розгорнути і налаштувати систему кінцевим користувачам.

ЗАГАЛЬНІ ВИСНОВКИ

У рамках роботи було розроблено вебресурс, що інтегрує технологію Retrieval-Augmented Generation для обробки україномовних документів. Досліджено сучасні підходи до реалізації RAG-систем та визначено особливості адаптації таких технологій до умов функціонування українською мовою. Обґрунтовано доцільність впровадження адаптованих мовних моделей з метою забезпечення точності пошуку та генерації релевантних відповідей українською мовою.

На основі проведеного аналізу спроектовано архітектуру додатку, яка забезпечує інтеграцію зовнішніх джерел даних, підтримку локального зберігання ембеддінг-векторів, а також організацію гнучкого вебінтерфейсу для зручної взаємодії з користувачем.

Запропоновано метод семантичного поділу текстових документів, що дозволяє підвищити якість роботи системи завдяки покращенню точності та релевантності результатів пошуку інформації. Результати апробації показали, що розроблений метод забезпечує точність поділу текстів на семантично цілі фрагменти [1].

Було створено україномовну ембеддінг-модель для формування векторних представлень тексту українською мовою. Оцінка моделі на перекладених наборах даних засвідчили її здатність представляти семантичний сенс текстів українською мовою. Про це свідчить приріст на оціночному наборі даних

Було реалізовано вебресурс, що інтегрує розроблену архітектуру, україномовну ембеддінг-модель та запропонований метод семантичного поділу текстових документів у цілісну систему Retrieval-Augmented Generation. Вебінтерфейс ресурсу забезпечує зручну взаємодію користувача з системою, підтримує налаштування параметрів роботи моделей та дозволяє вирішувати практичні задачі швидкого пошуку і генерації релевантних відповідей українською мовою.

Результати виконаної роботи можуть бути впроваджені в автоматизовані системи обробки інформації українською мовою для генерації точних і контекстуалізованих відповідей на основі зовнішніх джерел з допомогою великих мовних моделей.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Galchonkov O., Horchynskyi O., Antoshchuk S., Narezhnoy V. Semantic text splitting method development for rag systems with controlled threshold and sliding window size. Eastern-European Journal of Enterprise Technologies. 2025. Vol. 2, No. 2(134). P. 47-57. DOI: <https://doi.org/10.15587/1729-4061.2025.326177>.
2. Lewis P., Perez E., Piktus A., Petroni F., Karpukhin V., Goyal N., ... Riedel S. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. ArXiv, 2020. [Preprint]. URL: <https://arxiv.org/pdf/2005.11401> (дата звернення: 17.05.2025).
3. Jina AI. Jina-embeddings-v3. Hugging Face. URL: <https://huggingface.co/jinaai/jina-embeddings-v3> (дата звернення: 17.05.2025).
4. OpenAI. Embeddings guide. URL: <https://platform.openai.com/docs/guides/embeddings> (дата звернення: 17.05.2025).
5. OpenAI. URL: <https://openai.com/> (дата звернення: 17.05.2025).
6. Cohere. URL: <https://cohere.com/> (дата звернення: 17.05.2025).
7. Artificial Analysis. Leaderboards: Models. URL: <https://artificialanalysis.ai/leaderboards/models> (дата звернення: 17.05.2025).
8. Anthropic. URL: <https://www.anthropic.com/> (дата звернення: 17.05.2025).
9. Cosmic Python. Repository pattern. URL: https://www.cosmicpython.com/book/chapter_02_repository.html (дата звернення: 17.05.2025).
10. Wikipedia. Criticism of Java. URL: https://en.wikipedia.org/wiki/Criticism_of_Java (дата звернення: 17.05.2025).
11. Tiangolo S. FastAPI. URL: <https://fastapi.tiangolo.com> (дата звернення: 17.05.2025).
12. Langchain. Runnable base. URL: <https://api.python.langchain.com/en/v0.0.339/schema/langchain.schema.runnable.base.Runnable.html> (дата звернення: 17.05.2025).
13. SQLAlchemy. Engines. URL: <https://docs.sqlalchemy.org/en/20/core/engines.html> (дата звернення: 17.05.2025).
14. Unstructured. Overview. URL: <https://docs.unstructured.io/open-source/introd>

uction/overview (дата звернення: 17.05.2025).

15. SQLite. SQLite Home Page. URL: <https://www.sqlite.org/> (дата звернення: 17.05.2025).

16. Chroma. Documentation. URL: <https://docs.trychroma.com/> (дата звернення: 17.05.2025).

17. Gamma E., Helm R., Johnson R., Vlissides J. Design Patterns: Elements of Reusable Object-Oriented Software. Boston : Addison-Wesley, 1994. 395 p.

18. panallexeu. xlm-roberta-ua-distilled. URL: <https://github.com/panallexeu/xlm-roberta-ua-distilled> (дата звернення: 17.05.2025).

19. Reimers N., Gurevych I. Making Monolingual Sentence Embeddings Multilingual using Knowledge Distillation. ArXiv, 2020. [Preprint]. URL: <https://arxiv.org/pdf/2004.09813> (дата звернення: 17.05.2025).

20. sentence-transformers. multi-qa-mpnet-base-dot-v1. URL: <https://huggingface.co/sentence-transformers/multi-qa-mpnet-base-dot-v1> (дата звернення: 17.05.2025).

21. Berrier J. Move along. Dynamic-Size Sliding Window Pattern/Technique. URL: <https://jamie-berrier.medium.com/move-along-c09d59bea473> (дата звернення: 17.05.2025).

22. Cryptography. Fernet (symmetric encryption). URL: <https://cryptography.io/en/latest/fernet/> (дата звернення: 17.05.2025).

23. Fernet. Spec. URL: <https://github.com/fernet/spec/blob/master/Spec.md> (дата звернення: 17.05.2025).

ДОДАТОК А

Програмний код вебресурсу

На лістингах А.1–А.3 наведено основні фрагменти програмного коду серверної частини вебресурсу з використанням Retrieval-Augmented Generation з підтримкою української мови.

Лістинг А.1 – Фрагмент програми доменних об'єктів вебресурсу

```

from typing import Sequence, Type

from dependency_injector.wiring import inject, Provide
from mediatr import Mediator, GenericQuery
from pydantic import BaseModel

from src.adapters.async_vector_client import AbstractAsyncClient
from src.adapters.chat_model import AsyncStream
from src.adapters.uow import AbstractUoW
from src.adapters.vector_collection_repository import (
    AbstractAsyncVectorCollectionRepository,
    AbstractAsyncDocumentRepository
)
from src.di_container import Container
from src.domain.models import BaseChat
from src.domain.services.embedding_model_builder import EmbeddingModelBuilder
from src.domain.services.chat_model_builder import ChatModelBuilder
from src.domain.services.chat_prompt_template_builder import
ChatPromptTemplateBuilder

class FakeModel:

    def __init__(self, portal_gun: bool, spaceship: str):
        self.portal_gun = portal_gun
        self.spaceship = spaceship

    def __repr__(self):
        return f'FakeModel<portal_gun={self.portal_gun},
spaceship={self.spaceship}>'

    def meow(self) -> str:
        return f'Meow! My ship is {self.spaceship}.'

class ApiKeyCredential:

    def __init__(self, name: str, api_key: str):
        self.name = name
        self.api_key = api_key

    def __repr__(self) -> str:

```

```

        return f'ApiKeyCredential<name={self.name}, api_key={self.api_key}>'

class SystemPrompt:
    def __init__(self, name: str, content: str):
        self.name = name
        self.content = content

class DocProcessingConfig:
    """
    Configuration options for processing and chunking Unstructured documents.
    """

    def __init__(
        self,
        name: str,
        postprocessors: list[Postprocessor] | None,
        chunking_strategy: ChunkingStrategy | None,
        max_characters: int | None,
        new_after_n_chars: int | None,
        overlap: int | None,
        overlap_all: bool | None
    ):
        self.name = name
        self.postprocessors = json.dumps(
            list(map(lambda x: x.value, postprocessors))
        ) if postprocessors is not None else None
        self.chunking_strategy = chunking_strategy
        self.max_characters = max_characters
        self.new_after_n_chars = new_after_n_chars
        self.overlap = overlap
        self.overlap_all = overlap_all

        self._normalize_attrs() # <-- preserving configuration logic

    @property
    def deserialized_postprocessors(self) -> list[Postprocessor] | None:
        return list(map(lambda p: Postprocessor(p),
            json.loads(self.postprocessors))) \
            if self.postprocessors is not None else None

    def _normalize_attrs(self):
        if self.chunking_strategy is None:
            self.max_characters = None
            self.new_after_n_chars = None
            self.overlap = None
            self.overlap_all = None
        else:
            self.max_characters = 500 if self.max_characters is None else
self.max_characters
            self.new_after_n_chars = self.max_characters if self.new_after_n_chars
is None else self.new_after_n_chars
            self.overlap = 0 if self.overlap is None else self.overlap
            self.overlap_all = False if self.overlap_all is None else
self.overlap_all

```



```

class SemanticDocProcessingConfig:
    def __init__(
        self,
        name: str,
        thresh: float,
        max_chunk_size: int
    ):
        self.name = name
        self.thresh = thresh
        self.max_chunk_size = max_chunk_size

class ChatModel:
    api_key_credential: ApiKeyCredential

    def __init__(
        self,
        name: ChatModelName,
        api_key_credential_id: int,
        temperature: float | None,
        top_p: float | None,
        base_url: str | None,
        max_tokens: int | None,
        max_retries: int | None,
        stop_sequences: list[str] | None,
    ):
        self.name = name
        self.api_key_credential_id = api_key_credential_id
        self.temperature = temperature
        self.top_p = top_p
        self.base_url = base_url
        self.max_tokens = max_tokens
        self.max_retries = max_retries
        self.stop_sequences = json.dumps(stop_sequences) if stop_sequences is not
None else None

    @property
    def serialized_stop_sequence(self) -> list[str] | None:
        """
        I need this method because of the db mapping 🙄. Probably I did some bad
        designing.
        """
        return json.loads(self.stop_sequences) if self.stop_sequences is not None
else None

class EmbeddingModel:
    api_key_credential: ApiKeyCredential

    def __init__(
        self,
        name: EmbeddingModelName,
        device: Device,
        api_key_credential_id: int
    ):
        self.name = name
        self.device = device
        self.api_key_credential_id = api_key_credential_id

```

```

class VectorCollection:
    embedding_model: EmbeddingModel
    # ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ - populated by db

    def __init__(self,
                  self,
                  name: str,
                  distance_func: DistanceFunction,
                  embedding_model_id: int
    ):
        self.name = name
        self.distance_func = distance_func
        self.embedding_model_id = embedding_model_id

class VectorDocument:
    def __init__(
        self,
        page_content: str,
        metadata: dict[str, list | str | int | float]
    ):
        self.page_content = page_content
        self.metadata = self.normalize_metadata(metadata)
        self.id_ = sha224(self.page_content.encode()).hexdigest() # 28 byte hash
id (sha224)

    @staticmethod
    def normalize_metadata(dict_: dict):
        metadata = {}
        for key, val in dict_.items():
            if isinstance(val, list):
                metadata[key] = json.dumps(val)
            else:
                metadata[key] = val

        return metadata

    def __repr__(self) -> str:
        return f"VectorDocument<id_={self.id_} page_content='{self.page_content}' metadata={self.metadata}>"

class BaseChat:
    chat_model: ChatModel
    system_prompt: SystemPrompt | None
    vec_col: VectorCollection | None
    # ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ - populated by db

    def __init__(
        self,
        name: str,
        desc: str,
        chat_model_id: int,
        system_prompt_id: int | None,
        vec_col_id: int | None
    ):
        self.name = name
        self.desc = desc
        self.system_prompt_id = system_prompt_id
        self.chat_model_id = chat_model_id

```

```
self.vec_col_id = vec_col_id
```

Лістинг А.2 – Фрагмент програми обробників мовної моделі

```
from typing import Sequence

from pydantic import BaseModel
from mediatr import Mediator, GenericQuery
from dependency_injector.wiring import inject, Provide

from src.di_container import Container
from src.enums import ChatModelName
from src.domain.models import ChatModel
from src.adapters.uow import AbstractUoW

class ChatModelAddCommand(BaseModel, GenericQuery[ChatModel]):
    name: ChatModelName
    api_key_credential_id: int
    temperature: float | None
    top_p: float | None
    base_url: str | None
    max_tokens: int | None
    max_retries: int | None
    stop_sequences: list[str] | None

@Mediator.handler
class ChatModelAddHandler:
    @inject
    def __init__(
        self,
        chat_model_uow: AbstractUoW = Provide[Container.chat_model_uow]
    ):
        self.chat_model_uow = chat_model_uow

    def handle(self, request: ChatModelAddCommand) -> ChatModel:
        with self.chat_model_uow as uow:
            chat_model_obj = ChatModel(**request.model_dump())
            uow.repository.add(chat_model_obj)
            uow.commit()

        return chat_model_obj

class ChatModelReadQuery(BaseModel, GenericQuery[ChatModel]):
    id_: int

@Mediator.handler
class ChatModelReadHandler:
    @inject
    def __init__(
        self,
        chat_model_uow: AbstractUoW = Provide[Container.chat_model_uow]
    ):
```

```

        self.chat_model_uow = chat_model_uow

    def handle(self, request: ChatModelReadQuery) -> ChatModel:
        with self.chat_model_uow as uow:
            return uow.repository.read(request.id_)

class ChatModelReadAllQuery(GenericQuery[Sequence[ChatModel]]):
    def __init__(
        self,
        limit: int | None,
        offset: int | None,
        **kwargs
    ):
        self.limit = limit
        self.offset = offset
        self.kwargs = kwargs

@Mediator.handler
class ChatModelReadAllHandler:
    @inject
    def __init__(
        self,
        chat_model_uow: AbstractUoW = Provide[Container.chat_model_uow]
    ):
        self.chat_model_uow = chat_model_uow

    def handle(self, request: ChatModelReadAllQuery) -> Sequence[ChatModel]:
        with self.chat_model_uow as uow:
            return uow.repository.read_all(
                limit=request.limit,
                offset=request.offset,
                **request.kwargs
            )

class ChatModelUpdateCommand(GenericQuery[ChatModel]):
    def __init__(self, id_: int, **kwargs):
        self.id_ = id_
        self.kwargs = kwargs

@Mediator.handler
class ChatModelUpdateHandler:
    @inject
    def __init__(
        self,
        chat_model_uow: AbstractUoW = Provide[Container.chat_model_uow]
    ):
        self.chat_model_uow = chat_model_uow

    def handle(self, request: ChatModelUpdateCommand) -> ChatModel:
        with self.chat_model_uow as uow:
            upd_chat_model = uow.repository.update(request.id_, **request.kwargs)
            uow.commit()

        return upd_chat_model

```

```

class ChatModelDeleteCommand(BaseModel, GenericQuery[None]):
    id_: int

@Mediator.handler
class ChatModelDeleteHandler:
    @inject
    def __init__(
        self,
        chat_model_uow: AbstractUoW = Provide[Container.chat_model_uow]
    ):
        self.chat_model_uow = chat_model_uow

    def handle(self, request: ChatModelDeleteCommand) -> None:
        with self.chat_model_uow as uow:
            uow.repository.delete(request.id_)
            uow.commit()

class ChatModelCountQuery(GenericQuery[int | None]):
    pass

@Mediator.handler
class ChatModelCountHandler:
    @inject
    def __init__(
        self,
        chat_model_uow: AbstractUoW = Provide[Container.chat_model_uow]
    ):
        self.chat_model_uow = chat_model_uow

    def handle(self, request: ChatModelCountQuery) -> int | None:
        with self.chat_model_uow as uow:
            return uow.repository.count()

```

Лістинг А.3 – Фрагмент програми репозиторієв SQLAlchemy

```

from abc import ABC
from typing import get_args, Sequence, Optional, Type

import overrides
from sqlalchemy import func
from sqlalchemy import select
from sqlalchemy.orm import Session, joinedload

class AbstractRepository[T](ABC):

    def add(self, item: T) -> T:
        pass

    def read(self, id_: int) -> T:

```

```

        pass

    def read_all(
        self,
        offset: Optional[int] = None,
        limit: Optional[int] = None,
        **kwargs
    ) -> Sequence[T]:
        """
        Reads all values from the database for the provided type T.

        :param offset: Optional offset value for pagination implementation.
        :param limit: Optional limit value for pagination implementation.
        :param kwargs: Optional filtering criteria, e.g., field_name=value pairs.
        """
        pass

    def update(
        self,
        id_: int,
        **kwargs
    ) -> T:
        """
        Updates an existing record in the database identified by the given id.

        :param id_: The unique identifier of the record to be updated.
        :param kwargs: Optional keyword arguments representing the fields to be
updated and their new values.
        """
        pass

    def delete(self, id_: int) -> None:
        pass

    def count(self) -> int | None:
        pass

class ItemNotFoundError(LookupError):
    def __init__(self, id_: int, item: str):
        super().__init__(f"Item: {item} with the id '{id_}' was not found.")

class SqlAlchemyRepository[T](AbstractRepository):

    def __init__(
        self,
        session: Session
    ):
        self.session = session

    def add(self, item: T) -> T:
        self.session.add(item)
        self.session.flush()

        return item

    def read(self, id_: int) -> T:

```

```

"""
:raises ItemNotFoundError: if a nonexistent **id** is provided.
"""
    type_T = get_args(self.__orig_class__)[0] # this is the only way to
access original type with typing
    statement = select(type_T).where(type_T.id == id_)

    res = self.session.execute(statement).scalar()
    if res is None:
        raise ItemNotFoundError(id_, type_T)

    return res

def read_all(
    self,
    offset: int | None = None,
    limit: int | None = None,
    **kwargs
) -> Sequence[T]:
    type_T = get_args(self.__orig_class__)[0]
    statement = select(type_T).offset(offset).limit(limit).filter_by(**kwargs)
    return self.session.execute(statement).scalars().all()

def update(self, id_: int, **kwargs) -> T:
    """
    :raises ItemNotFoundError: if a nonexistent **id** is provided.
    """
    type_T = get_args(self.__orig_class__)[0]

    item = self.session.get(type_T, id_)
    if item is None:
        raise ItemNotFoundError(id_, type_T)

    item_dict = item.__dict__

    # resolving attributes to be updated in obj_ based on the provided
**kwargs
    for key, value in kwargs.items():
        if key in item_dict:
            setattr(item, key, value)

    self.session.flush()

    return item

def delete(self, id_: int) -> None:
    """
    :raises ItemNotFoundError: if a nonexistent **id** is provided.
    """
    type_T = get_args(self.__orig_class__)[0]

    item = self.session.get(type_T, id_)
    if item is None:
        raise ItemNotFoundError(id_, type_T)

    self.session.delete(item)

def count(self) -> int | None:

```

```

    """
    Counts rows in a type_T table.
    """
    type_T = get_args(self.__orig_class__)[0]
    statement = select(func.count()).select_from(type_T)

    return self.session.execute(statement).scalar()

class SQLAlchemyRelationRepository[T](SQLAlchemyRepository):

    @overrides.override
    def add(self, item: T) -> T:
        type_T = get_args(self.__orig_class__)[0] # this is the only way to
        access original type with typing
        self.session.add(item)
        self.session.flush()

        statement = select(type_T).where().options(joinedload('*'))

        return self.session.execute(statement).scalar()

    @overrides.override
    def read(self, id_: int) -> T:
        """
        :raises ItemNotFoundError: if a nonexistent **id_** is provided.
        """
        type_T = get_args(self.__orig_class__)[0]
        statement = select(type_T).where(type_T.id ==
id_).options(joinedload('*'))

        res = self.session.execute(statement).scalar()
        if res is None:
            raise ItemNotFoundError(id_, type_T)

        return res

    @overrides.override
    def read_all(
        self,
        offset: int | None = None,
        limit: int | None = None,
        **kwargs
    ) -> Sequence[T]:
        type_T = get_args(self.__orig_class__)[0]
        statement =
select(type_T).offset(offset).limit(limit).filter_by(**kwargs).options(joinedload(
'*'))
        return self.session.execute(statement).scalars().all()

    @overrides.override
    def update(self, id_: int, **kwargs) -> T:
        """
        :raises ItemNotFoundError: if a nonexistent **id_** is provided.
        """
        type_T = get_args(self.__orig_class__)[0]

        item = self.session.get(type_T, id_)
        if item is None:

```



```

        raise ItemNotFoundError(id_, type_T)

    item_dict = item.__dict__

    # resolving attributes to be updated in obj_ based on the provided
    **kwargs
    for key, value in kwargs.items():
        if key in item_dict:
            setattr(item, key, value)

    self.session.flush()

    statement = select(type_T).where(type_T.id ==
id_).options(joinedload('*'))

    return self.session.execute(statement).scalar()

```

ДОДАТОК Б

Мокапи сторінок вебресурсу

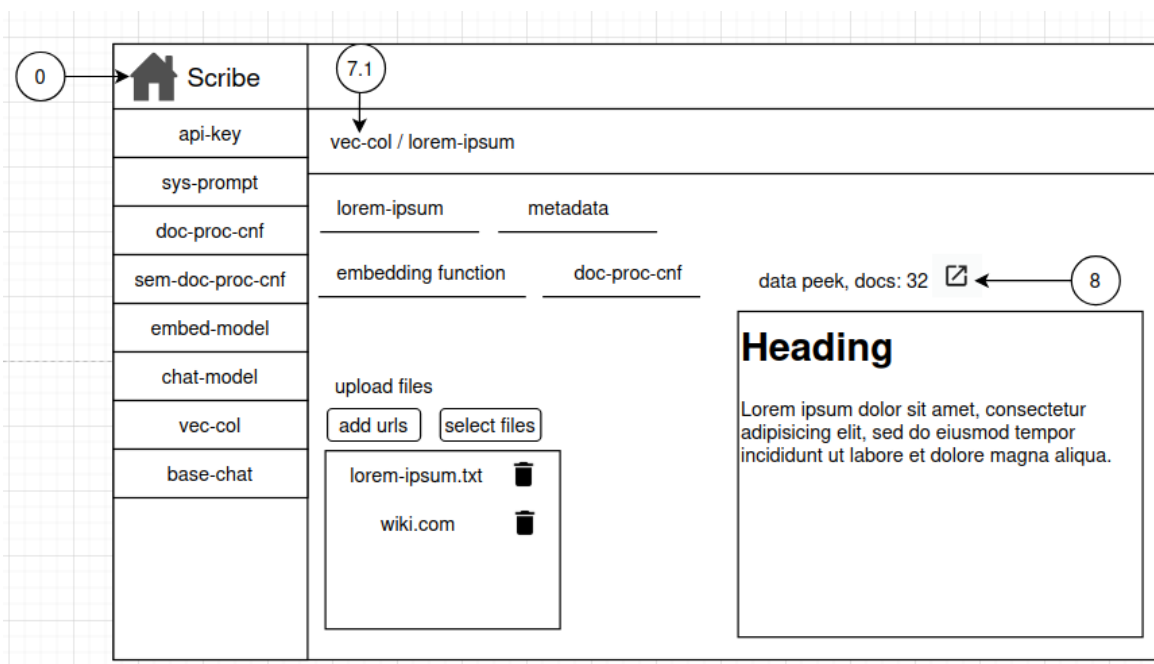


Рисунок Б.1 – Мокап сторінки зміни векторної колекції

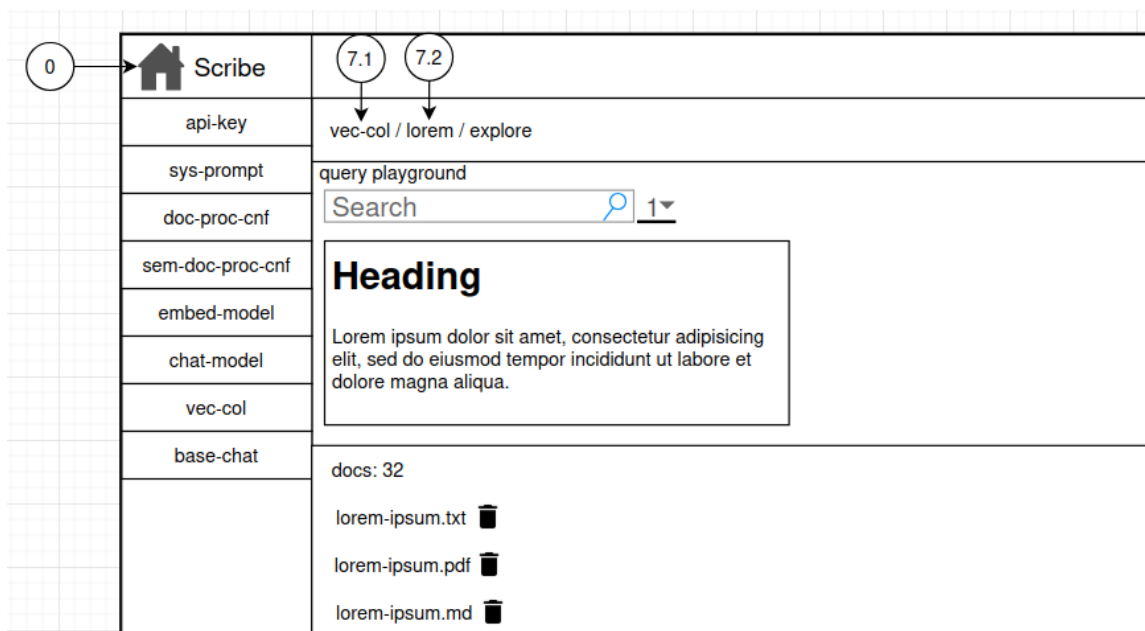


Рисунок Б.2 – Мокап сторінки дитячого майданчика векторної колекції

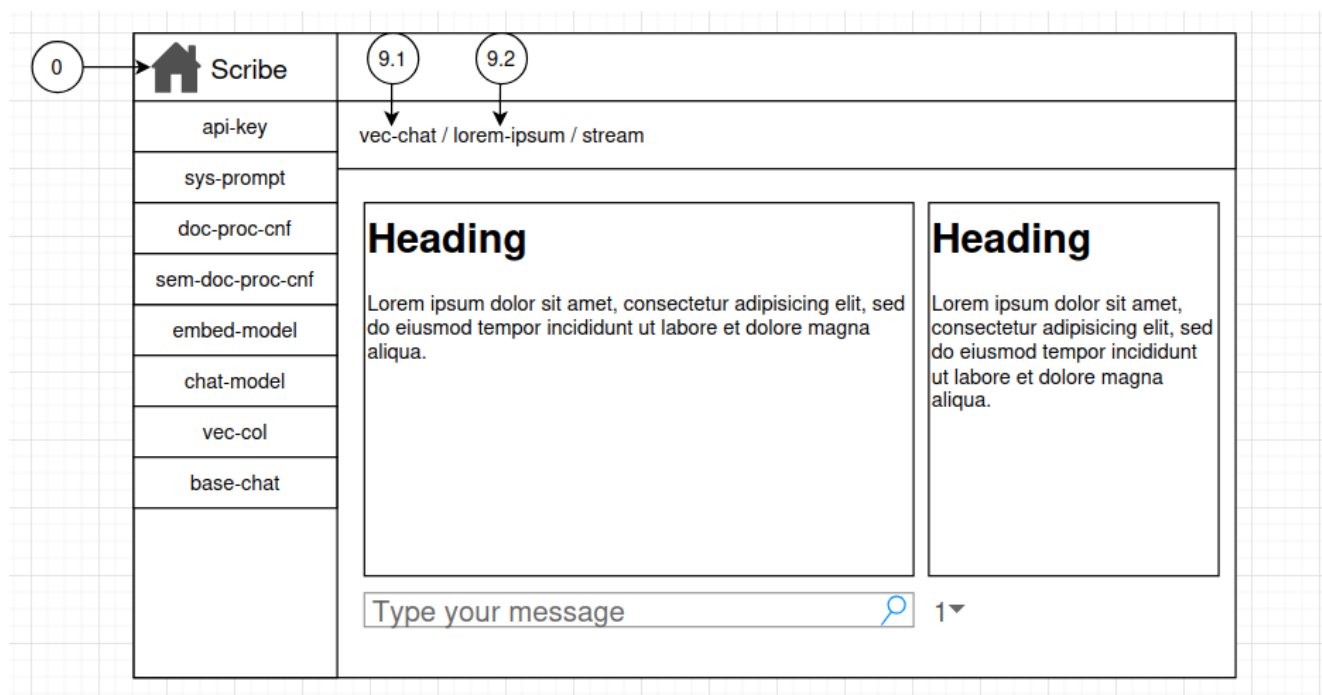


Рисунок Б.3 – Мокап сторінки розмови в чаті

ДОДАТОК В

Протокол проведення функціонального тестування вебресурсу

В табл. В.1 наведено результати проведення функціонального тестування вебресурсу з використанням Retrieval-Augmented Generation з підтримкою української мови.

Таблиця В.1 – Протокол проведення функціонального тестування вебресурсу

Номер тест-кейсу	Очікуваний результат	Фактичний результат	Статус
1	2	3	4
Тест-кейс №1	Директорія logs наявна в .scribe	Було виявлено директорію logs в .scribe	Пройдено
Тест-кейс №2	Всі файли окрім scribe.key та logs видалено	Було видалено всі файли окрім scribe.key та logs	Пройдено
Тест-кейс №3	Файл scribe.key відтворено в .scribe	Файл scribe.key було відтворено в .scribe	Пройдено
Тест-кейс №4	Клієнтська частина доступна через браузер	Було відкрито клієнтську частину через браузер	Пройдено
Тест-кейс №5	З'являється повідомлення про некоректність імені API ключа	Було виявлено повідомлення про некоректність імені API ключа	Пройдено
Тест-кейс №6	Повернення зашифрованого API ключа у UI	Було отримано зашифрований API ключ у UI	Пройдено
Тест-кейс №7	Ім'я API ключа успішно змінено	Ім'я API ключа було успішно змінено	Пройдено
Тест-кейс №8	API ключ успішно видалено	API ключ було успішно видалено	Пройдено
Тест-кейс №9	Підказку успішно введено	Підказку було успішно введено	Пройдено
Тест-кейс №10	Користувач перенаправлений на сторінку з усіма підказками	Було здійснено перенаправлення на сторінку з усіма підказками	Пройдено
Тест-кейс №11	Параметри підказки успішно змінено	Параметри підказки було успішно змінено	Пройдено
Тест-кейс №12	Підказку успішно видалено	Підказку було успішно видалено	Пройдено
Тест-кейс №13	Застосовано сценарій за замовчуванням	Було застосовано сценарій за замовчуванням	Пройдено

Продовження таблиці 3.5

1	2	3	4
Тест-кейс №14	Параметри успішно збережено	Параметри було успішно збережено	Пройдено
Тест-кейс №15	Користувач перенаправлений на сторінку з усіма конфігураціями	Було здійснено перенаправлення на сторінку з усіма конфігураціями	Пройдено
Тест-кейс №16	Параметри успішно змінено	Параметри було успішно змінено	Пройдено
Тест-кейс №17	Конфігурація успішно видалено	Конфігурація була успішно видалена	Пройдено
Тест-кейс №18	Модель обрана	Модель була обрана	Пройдено
Тест-кейс №19	Користувач перенаправлений на сторінку з усіма семантичними конфігураціями	Було здійснено перенаправлення на сторінку з усіма семантичними конфігураціями	Пройдено
Тест-кейс №20	Користувач перенаправлений на сторінку з усіма ембеддінг-моделями	Було здійснено перенаправлення на сторінку з усіма ембеддінг-моделями	Пройдено
Тест-кейс №21	Параметри успішно змінено	Параметри було успішно змінено	Пройдено
Тест-кейс №22	Модель успішно видалено	Модель була успішно видалена	Пройдено
Тест-кейс №23	Модель обрана	Модель була обрана	Пройдено
Тест-кейс №24	Параметри успішно збережено	Параметри було успішно збережено	Пройдено
Тест-кейс №25	Користувач перенаправлений на сторінку з усіма мовними моделями	Було здійснено перенаправлення на сторінку з усіма мовними моделями	Пройдено
Тест-кейс №26	Параметри успішно змінено	Параметри було успішно змінено	Пройдено
Тест-кейс №27	Модель успішно видалено	Модель була успішно видалена	Пройдено
Тест-кейс №28	Функцію виміру відстані успішно збережено	Функцію виміру відстані було успішно збережено	Пройдено
Тест-кейс №29	Користувач перенаправлений на сторінку з усіма векторними колекціями	Було здійснено перенаправлення на сторінку з усіма векторними колекціями	Пройдено

Кінець таблиці 3.5

1	2	3	4
Тест-кейс №30	Колекція успішно видалено	Колекція була успішно видалена	Пройдено
Тест-кейс №31	Вебсторінка успішно завантажено	Вебсторінка була успішно завантажена	Пройдено
Тест-кейс №32	Файли успішно завантажено	Файли були успішно завантажені	Пройдено
Тест-кейс №33	Інформація успішно відображено	Інформація була успішно відображена	Пройдено
Тест-кейс №34	Документи успішно видалено	Документи були успішно видалені	Пройдено
Тест-кейс №35	Кількість результатів успішно вибрано	Кількість результатів була успішно вибрана	Пройдено
Тест-кейс №36	Отримані найближчі частини документів	Було отримано найближчі частини документів	Пройдено
Тест-кейс №37	Відстань між частинами і запитом відображено	Було відображено відстань між частинами і запитом	Пройдено
Тест-кейс №38	З'являється повідомлення про необхідність введення даних	Було виявлено повідомлення про необхідність введення даних	Пройдено
Тест-кейс №39	Векторна колекція додана до чату	Векторну колекцію було додано до чату	Пройдено
Тест-кейс №40	Модель відповідає без включення контекстуальних даних	Було отримано відповідь моделі без включення контекстуальних даних	Пройдено
Тест-кейс №41	Відповідь моделі надходить у вигляді безперервного текстового потоку	Було отримано відповідь моделі у вигляді безперервного текстового потоку	Пройдено
Тест-кейс №42	Модель надає контекстуалізовану відповідь на основі документів з колекції	Було отримано контекстуалізовану відповідь на основі документів з колекції	Пройдено
Тест-кейс №43	Кількість частин; що будуть використані; успішно вибрано	Кількість частин; що будуть використані; було успішно вибрано	Пройдено
Тест-кейс №44	Разом з відповіддю моделі отримано частини документів; що використовувалися для контексту	Було отримано частини документів; що використовувалися для контексту; разом з відповіддю моделі	Пройдено

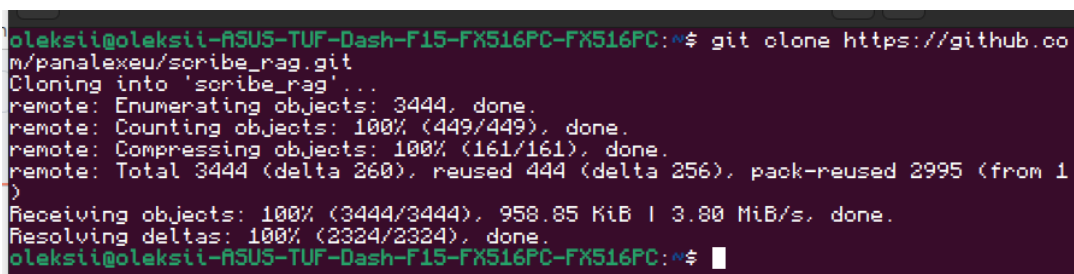
ДОДАТОК Г

Інструкція користувача вебресурсу

Для успішного запуску вебресурсу попередньо повинні бути встановлені:

- 1) git;
- 2) poetry;
- 3) npm.

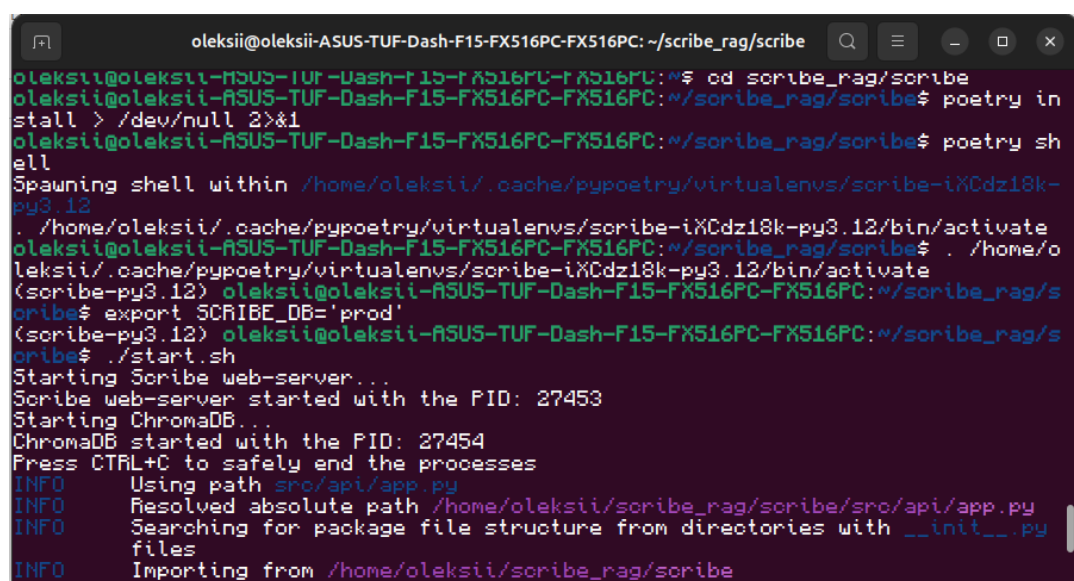
Для встановлення проекту потрібно виконати наступну команду в терміналі (рис. Г.1). Ця команда клонує репозиторій проекту.



```
oleksii@oleksii-ASUS-TUF-Dash-F15-FX516PC-FX516PC:~$ git clone https://github.com/panalexeu/scribe_rag.git
Cloning into 'scribe_rag'...
remote: Enumerating objects: 3444, done.
remote: Counting objects: 100% (449/449), done.
remote: Compressing objects: 100% (161/161), done.
remote: Total 3444 (delta 260), reused 444 (delta 256), pack-reused 2995 (from 1)
Receiving objects: 100% (3444/3444), 958.85 KiB | 3.80 MiB/s, done.
Resolving deltas: 100% (2324/2324), done.
oleksii@oleksii-ASUS-TUF-Dash-F15-FX516PC-FX516PC:~$
```

Рисунок Г.1 – Клонування репозиторію проекту через git

Для запуску серверної частини потрібно виконати наступні команди за допомогою poetry (рис. Г.2).



```
oleksii@oleksii-ASUS-TUF-Dash-F15-FX516PC-FX516PC:~$ cd scribe_rag/scribe
oleksii@oleksii-ASUS-TUF-Dash-F15-FX516PC-FX516PC:~/scribe_rag/scribe$ poetry install
oleksii@oleksii-ASUS-TUF-Dash-F15-FX516PC-FX516PC:~/scribe_rag/scribe$ poetry shell
Spawning shell within /home/oleksii/.cache/pypoetry/virtualenvs/scribe-iXCdz18k-py3.12
. /home/oleksii/.cache/pypoetry/virtualenvs/scribe-iXCdz18k-py3.12/bin/activate
oleksii@oleksii-ASUS-TUF-Dash-F15-FX516PC-FX516PC:~/scribe_rag/scribe$ . /home/oleksii/.cache/pypoetry/virtualenvs/scribe-iXCdz18k-py3.12/bin/activate
(scribe-py3.12) oleksii@oleksii-ASUS-TUF-Dash-F15-FX516PC-FX516PC:~/scribe_rag/scribe$ export SCRIBE_DB='prod'
(scribe-py3.12) oleksii@oleksii-ASUS-TUF-Dash-F15-FX516PC-FX516PC:~/scribe_rag/scribe$ ./start.sh
Starting Scribe web-server...
Scribe web-server started with the PID: 27453
Starting ChromaDB...
ChromaDB started with the PID: 27454
Press CTRL+C to safely end the processes
INFO Using path src/api/app.py
INFO Resolved absolute path /home/oleksii/scribe_rag/scribe/src/api/app.py
INFO Searching for package file structure from directories with __init__.py files
INFO Importing from /home/oleksii/scribe_rag/scribe
```

Рисунок Г.2– Запуск серверної частини вебресурсу

Вищезазначені команди встановлюють залежності, вмикають віртуальне середовище, конфігурують змінні зсередовища, запускать стартовий скрипт, тощо. Після їх виконання серверну частину запущено і ним вже можна користуватись.

Тепер перейдемо до запуску клієнтської частини в окремому терміналі. Для цього потрібно виконати наступні команди (рис. Г.3). Зазначені команди встановлюють залежності і запускають next.js сервер.

```

oleksii@oleksii-ASUS-TUF-Dash-F15-FX516PC-FX516PC: ~/scribe_rag/scribe_ui
oleksii@oleksii-ASUS-TUF-Dash-F15-FX516PC-FX516PC:~$ cd scribe_rag/scribe_ui/
oleksii@oleksii-ASUS-TUF-Dash-F15-FX516PC-FX516PC:~/scribe_rag/scribe_ui$ npm in
stall > /dev/null 2>&1
oleksii@oleksii-ASUS-TUF-Dash-F15-FX516PC-FX516PC:~/scribe_rag/scribe_ui$ npm ru
n dev
> dev
> next dev

▲ Next.js 14.2.13
- Local:      http://localhost:3000

✓ Starting...
✓ Ready in 1272ms
  
```

Рисунок Г.3 – Запуск клієнтської частини вебресурсу

Після встановлення і запуску перейдемо до роботи з графічним інтерфейсом. При першому вході користувача вітає головна порожня сторінка, оскільки ще нічого не було додано (рис. Г.4).

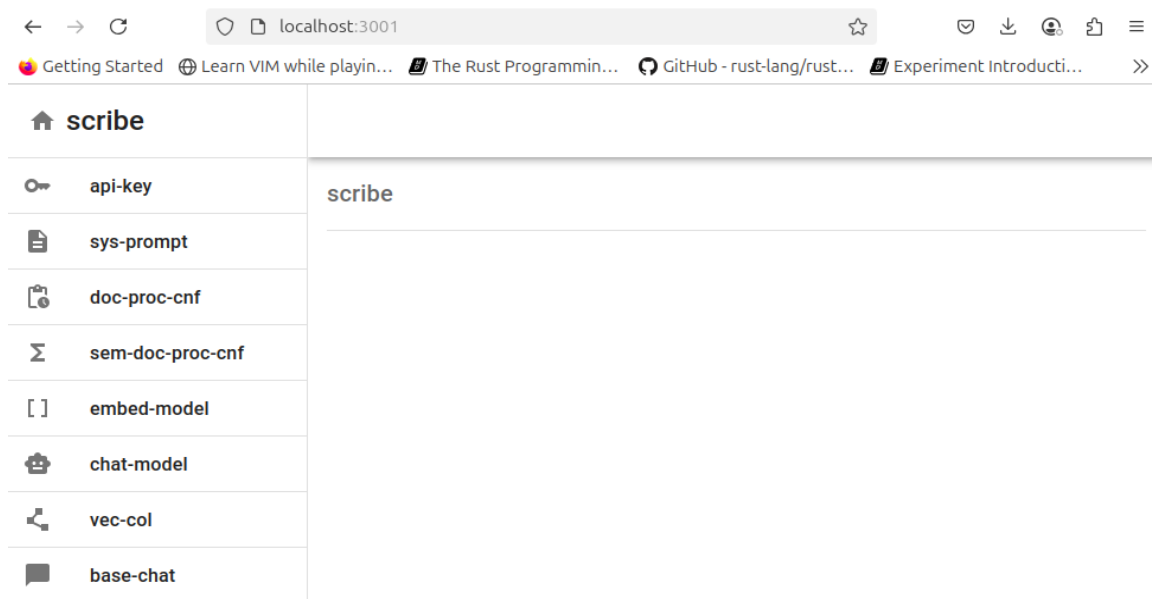


Рисунок Г.4– Головна сторінка

Для початку перейдемо до api-key і створимо OpenAI API ключ, що було отримано через OpenAI API (рис. Г.5).

The screenshot shows a web browser at localhost:3001/api-key/add. The page title is 'api-key / add'. There are two input fields: 'name' containing 'openai-key' and 'api key' containing 'kpZTy0J2vo0rjd1zmZXxEA'. Below these fields is a blue 'SUBMIT' button. The left sidebar contains a list of items: 'scribe' (home), 'api-key' (selected), 'sys-prompt', 'doc-proc-cnf', 'sem-doc-proc-cnf', 'embed-model', 'chat-model', 'vec-col', and 'base-chat'.

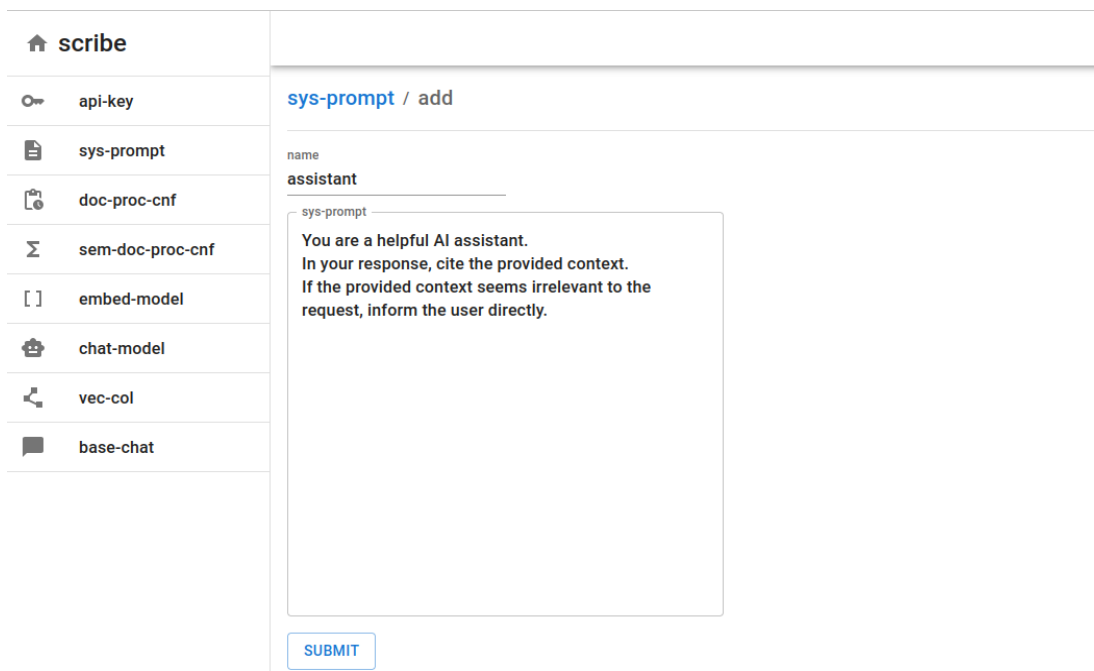
Рисунок Г.5 – Створення API-ключа

Після створення API ключа, користувач перенаправляється на сторінку перегляду ключів (рис. Г.6). Тут можна переглянути, оновити, змінити і додати записи. Подібні сторінку мають і інші сутності.

The screenshot shows the 'api-key' list view. The table has two columns: 'name' and 'api key'. The first row shows 'openai-key' and '05-11-2025, 19:29'. There are icons for deleting and editing the entry. The left sidebar is the same as in Figure Г.5.

Рисунок Г.6 – Сторінка перегляду API ключів

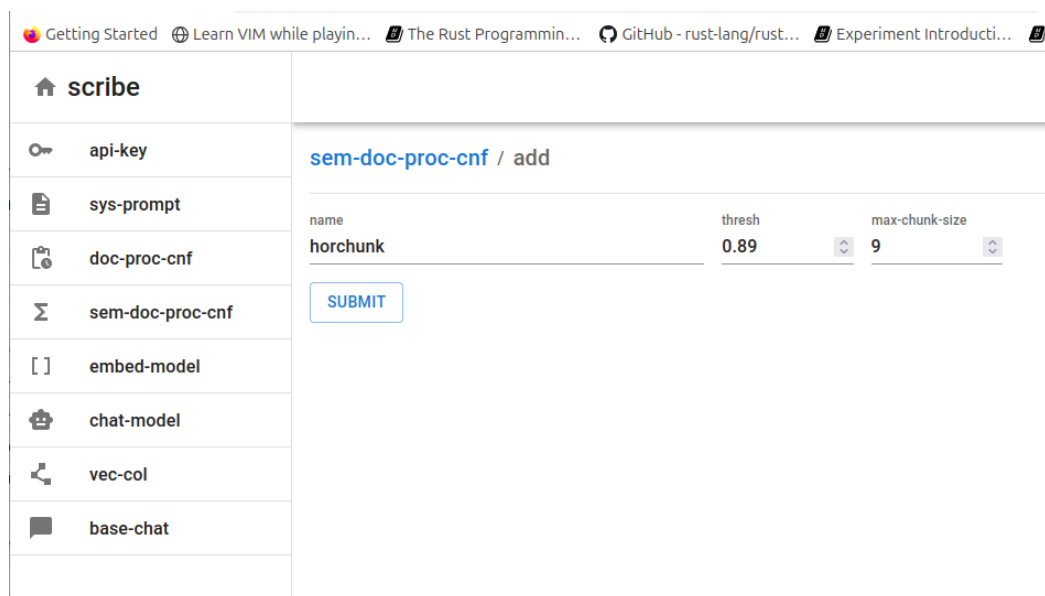
Далі створимо системну підказку з наступними значеннями (рис. Г.7). В самій підказці буде зазначено цитувати контекст і повідомляти, якщо переданий контекст здається непов'язаним.



scribe	
api-key	sys-prompt / add
sys-prompt	name assistant sys-prompt You are a helpful AI assistant. In your response, cite the provided context. If the provided context seems irrelevant to the request, inform the user directly. SUBMIT
doc-proc-cnf	
sem-doc-proc-cnf	
embed-model	
chat-model	
vec-col	
base-chat	

Рисунок Г.7 – Створення системної підказки

Далі створимо конфігурацію для обробки документів (рис Г.8). Оберемо семантичну конфігурацію з параметрами thresh 0,89 і max_chunk_size як 9.



scribe	
api-key	sem-doc-proc-cnf / add
sys-prompt	
doc-proc-cnf	
sem-doc-proc-cnf	name horchunk thresh 0.89 max-chunk-size 9 SUBMIT
embed-model	
chat-model	
vec-col	
base-chat	

Рисунок Г.8 – Створення конфігурації семантичної обробки документів

Далі створимо локальну, україномовну ембедінг-модель xlm-roberta-ua-distilled. Оскільки модель є локальною, створимо API-ключ заглушку (рис. Г.9).

Рисунок Г.9 – Створення API-ключа заглушки

Далі створимо локальну ембедінг-модель зі створеною заглушкою API-ключа (рис. Г.10).

datetime	name	api-key
05-11-2025, 19:29	openai-key	gAAAABoINB5DCsbuJ5hHRAI1-O8Tsa-Svuqxm6NYjN0E_kXBwwRuuzEOmvWJAZ-exrX10YIzEQgjINkoBOIKSd-0ouM7vsZHQ8iD5pLDdB2F-3uahmlmQFxCnAo8JHfidmA8Z
05-11-2025, 19:37	placeholder	gAAAABoINJfc4KYakHQvTVclgBoYi77JuXy8T0YCVohL-1roV1aCEWx5cdCXUIDsYSZ

Рисунок Г.10 – Створення ембедінг-моделі

Далі створимо мовну модель (рис. Г.11). Оберемо gpt-4o-mini і openai-key. З параметрів обмежимо відповідь моделі до 1024 токенів і зменшимо температуру до 0,7 для більш фактичних відповідей.

scribe

api-key

sys-prompt

doc-proc-cnf

sem-doc-proc-cnf

embed-model

chat-model

vec-col

base-chat

chat-model / add

chat-model name:

api key:

base-url:

top-p:

max-tokens:

max-retries:

temperature:

stop-sequence:

[ADD](#) [CLEAR](#)

api-keys

datetime	name	api-key
05-11-2025, 19:29	openai-key	gAAAABoINB5DCsbuJ5hHRAI1-08Tsa-SvuqqxM6NYJN0E_kXBwwRuuzEOmvWJAZ-e_u5hHdmetS4b-OhaCOYjrtLWnWmt1I1SoFDllxrXI0YizEQgjINkoBOIKSd-0ouM7vSZHQ8ID5pLDd82F-3uahmImQFxCnAo8JHfdmA8Z17UneDaeqWkYVXm6SIB-EEzdv5Wz8v8W1y
05-11-2025, 19:37	placeholder	gAAAABoINJfc4KYakHQvTVclgBoYi77JuXy8T0YCVohL-1roV1aCEWx5cdCXUIDsYSZb0TVVyGDKpdqELJxNa25Ai6TwKw36Q==

1-2 of 2 < >

[SUBMIT](#)

Рисунок Г.11 – Створення мовної моделі

Далі перейдемо до створення векторної колекції (рис. Г.12). Для цього необхідно обрати раніше створену локальну ембедінг-модель, яка дозволяє перетворювати текстові дані у векторний простір. Також необхідно визначити метрику для розрахунку схожості між векторами – у нашому випадку доцільно використати косинусну відстань, яка є однією з найпоширеніших і ефективних для порівняння багатовимірних векторів. Це забезпечить більш точне групування схожих об'єктів у колекції та підвищить якість подальшої обробки даних. Налаштування цих параметрів є необхідним для початку роботи з векторною колекцією і обробки документів.

Рисунок Г.12 – Створення векторної колекції

Після створення, оберемо попередньо створену конфігурацію обробки документів, завантажимо до колекції документ, що описує особливості форматування дипломної роботи (рис. Г.13).

Рисунок Г.13 – Завантаження документу до колекції

Після завантаження отримано перегляд перших трьох частин документа і кількість частин загалом (рис. Г.14).

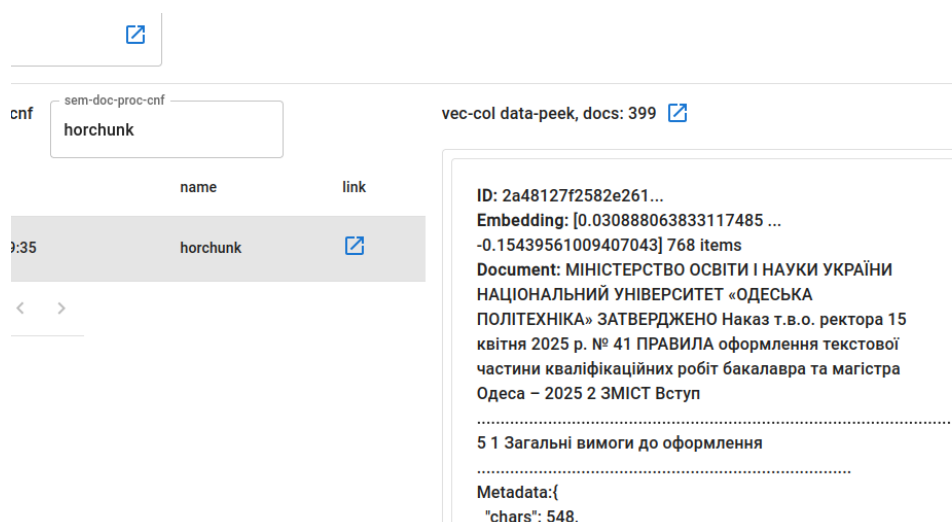


Рисунок Г.14 – Результат успішного завантаження документу до колекції

Ми можемо перевірити, які документи повертається на запит, через використання дитячого майданчика векторної колекції (рис. Г.15). Було введено запит «Як оформлювати формули?» на що було отримано оброблені, релевантні частини документа.

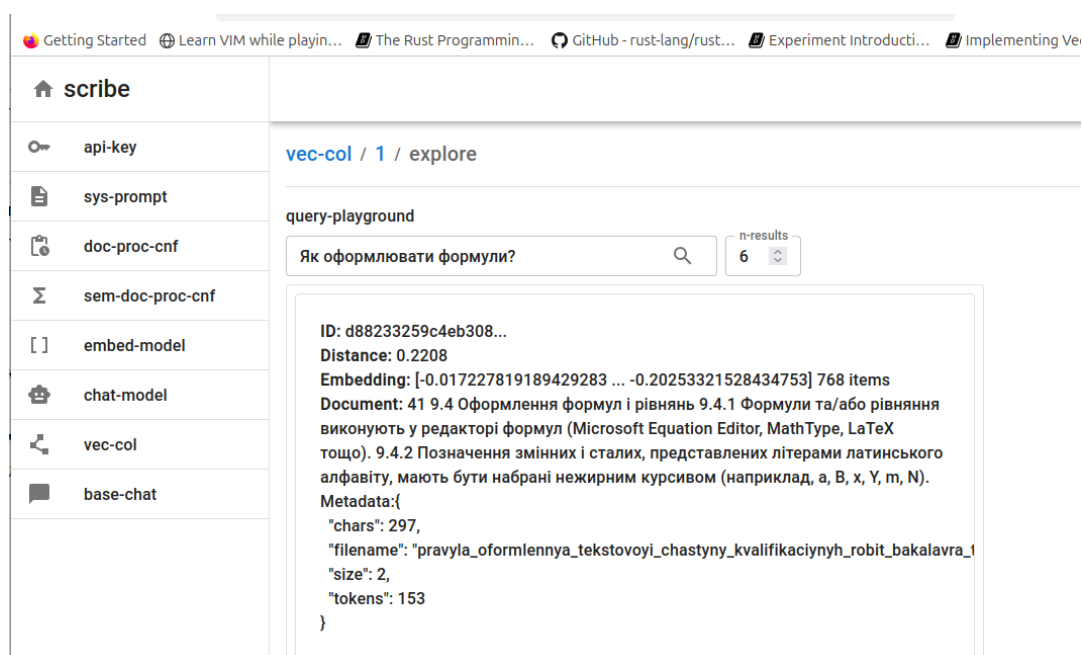


Рисунок Г.15 – Запит до векторної колекції в режимі дитячого майданчика

Перейдемо до останньої частини роботи, а саме створення чату (рис. Г.16). Для цього було обрано попередньо створенні: мовна модель, системна підказка, векторна колекція.

scribe

api-key

sys-prompt

doc-proc-cnf

sem-doc-proc-cnf

embed-model

chat-model

vec-col

base-chat

base-chat / add

name
diploma-formatting

desc
This chat provides diploma formatting guidelines.

sys-prompt
assistant

chat-model
gpt-4o-mini

vec-col
diploma-formatting

date	name	date	name	date	name
05-11-2025, 19:34	assistant	05-11-2025, 19:40	gpt-4o-mini	05-11-2025, 19:42	diploma-formatting

1-1 of 1 < > 1-1 of 1 < > 1-1 of 1 < >

SUBMIT

Рисунок Г.16 – Створення чату

Сторінка перегляду створених чатів має наступний вигляд (рис. Г.17).

scribe

api-key

sys-prompt

doc-proc-cnf

sem-doc-proc-cnf

embed-model

chat-model

vec-col

base-chat

base-chat

05-11-2025, 20:01 diploma-formatting

This chat provides diploma formatting guidelines.

< 1 >

Рисунок Г.17– Перегляд створених чатів

Протестуємо роботу чату через запит «Як оформлювати формули?» (рис. Г.18). Як видно з результатів, було витягнуто релевантні частини документа (права частина сторінки) і додано як контекст до відповіді мовної моделі (центральна частина сторінки).

The screenshot displays the 'scribe' chat application interface. On the left is a sidebar with a list of components: **scribe**, **api-key**, **sys-prompt**, **doc-proc-cnf**, **sem-doc-proc-cnf**, **embed-model**, **chat-model**, **vec-col**, and **base-chat**. The main area is titled **base-chat / 1 / stream**. It shows a chat history with a message from 'diploma-formatting' asking for formula formatting rules. The response from 'assistant' provides a list of four requirements for formula formatting. To the right of the chat history, a panel displays metadata for the retrieved document, including ID, Distance, Embedding, and a list of relevant items. At the bottom, there is a text input field labeled 'Type your message...' and a button to send the message.

diploma-formatting

This chat provides diploma formatting guidelines.

assistant

Оформлення формул і рівнянь регламентується наступними вимогами:

1. **Окремий рядок:** Формули та/або рівняння подають окремим рядком, по центру, без абзацного відступу, безпосередньо після тексту, у якому на них посилаються (пункт 9.1.1).
2. **Позначення змінних:** Літературні позначення змінних і сталих, які використовуються у формулах, мають бути набрані нежирним курсивом (наприклад, a , B , x , Y , m , N) (пункт 9.4.2).
3. **Пояснення позначок:** Пояснення позначок, які входять до формули, подають безпосередньо під формулою. Кожну позначку розміщують на окремому рядку, після неї ставлять тире, а потім пояснення з одиницею фізичної величини (пункт 9.3.1).
4. **Номер формули:** Номер формули розміщують праворуч на рівні формули, у крайньому положенні, у круглих дужках (пункт 9.2.3).

Ці вимоги допоможуть правильно оформити формули у ваших документах.

ID:
d88233259c4eb308...

Distance: 0.2208

Embedding:
[-0.017227819189429283
...
-0.20253321528434753]
768 items

Document: 41 9.4
Оформлення формул і рівнянь 9.4.1
Формули та/або рівняння виконують у редакторі формул (Microsoft Equation Editor, MathType, LaTeX тощо). 9.4.2
Позначення змінних і сталих, представлених

Type your message...

n-results
5

Рисунок Г.18 – Результат запити «Як оформлювати формули»